

Parciales Resueltos

Criptografía y Seguridad — Segunda Parte

Mapeo ejercicio → temas + resolución

Instituto Tecnológico de Buenos Aires (ITBA)

Índice

Cómo está armado	3
Material fuente (enlaces)	3
1. Segundo Parcial 2013	4
1.1. Ejercicio 1 — Modelo de compartimentos de integridad (clínica)	4
1.2. Ejercicio 2 — Router Linksys: exposición de <code>/etc/passwd</code>	6
1.3. Ejercicio 3 — Secreto compartido de umbral (t, n) y entropía	8
1.4. Ejercicio 4 — Verdadero / Falso (biométrico, matriz vs flujo, zip bomb)	10
1.5. Ejercicio 5 — Módulos de Control de Acceso y Autenticación	11
2. Parcial 2 – 2015	13
2.1. Ejercicio 1 — Compartimentos de confidencialidad (restaurante)	13
2.2. Ejercicio 2 — Verdadero/Falso (pentesting, Anderson+salt, STRIDE)	16
2.3. Ejercicio 3 — Secreto compartido de Shamir $(3,4)$ + entropía	17
2.4. Ejercicio 4 — Resolución de conflictos en ACL + DMZ	18
3. Segundo Parcial 2016	20
3.1. Ejercicio 1 — Modelo de compartimentos de confidencialidad	20
3.2. Ejercicio 2 — Definir conceptos y relacionarlos	22
3.3. Ejercicio 3 — Secreto compartido de Shamir y entropía	23
3.4. Ejercicio 4 — Multiple choice con justificación	25
4. Segundo Parcial 2017	27
4.1. Ejercicio 1 — Modelo de compartimentos de confidencialidad (Bell–LaPadula)	27
4.2. Ejercicio 2 — Verdadero/Falso justificado (pentesting, Anderson, STRIDE)	29
4.3. Ejercicio 3 — Secreto compartido de Shamir y entropía	30
4.4. Ejercicio 4 — Multiple choice (resolución de conflictos en ACL y DMZ)	31
5. Parcial 2018 Q1	34
5.1. Ejercicio 1 — Secreto compartido de Shamir	34
5.2. Ejercicio 2 — Verdadero o falso (control de acceso, OIDC, inyección)	35
5.3. Ejercicio 3 — Política de integridad sobre un retículo (Biba)	36
5.4. Ejercicio 4 — Entropía en audio WAV y esteganografía	38
5.5. Ejercicio 5 — Opción correcta (DMZ, Von Neumann/inyección, Muralla China)	39
6. Parcial 2023 Q1	42
6.1. Ejercicio 1 — Protocolo de establecimiento de clave de sesión	42
6.2. Ejercicio 2 — Modo CTR sin primitiva inversa, y CBC	43
6.3. Ejercicio 3 — Base64 como “cifrado”; confusión y difusión; (no)linealidad	44
6.4. Ejercicio 4 — Cifrador afín y seguridad CPA	45
6.5. Ejercicio 5 — Verdadero/Falso (MAC, hash, Diffie–Hellman, reversibilidad)	46
7. Recuperatorio 2023 Q1	48
7.1. Ejercicio 1 — Tiempo de ataque (fórmula de Anderson)	48
7.2. Ejercicio 2 — Secreto compartido de Shamir $(3, 4)$ en $\mathbb{Z}_7$	48
7.3. Ejercicio 3 — Verdadero o falso (justificando)	49

8. Parcial 2025 Q1	52
8.1. Ejercicio 1 — Arquitectura de 3 capas, Clark–Wilson y red plana	52
8.2. Ejercicio 2 — Múltiple choice (DevOps, tolerancia a fallas, CSRF)	53
8.3. Ejercicio 3 — Fórmula de Anderson, salt y autenticación multicanal	55
8.4. Ejercicio 4 — Secreto compartido de Shamir y entropía	56
8.5. Ejercicio 5 — Modelado de control de acceso para una organización (roles y datos)	57
9. Recuperatorio 2025 Q1	60
9.1. Ejercicio 1 — Secreto compartido de Shamir $(3, 3)$ en $\mathbb{Z}_{11}$	60
9.2. Ejercicio 2 — Verdadero/Falso (corregir una palabra)	61
9.3. Ejercicio 3 — One-time-pad y pérdida de información (entropía)	62
9.4. Ejercicio 4 — Fórmula de Anderson y almacenamiento con SHA-256	63
10. Parcial 2025 Q2	65
10.1. Ejercicio 1 — Modelo de seguridad para Palantir (CSO)	65
10.2. Ejercicio 2 — Opción múltiple (pentesting, ley 25.326, buffer overflow)	66
10.3. Ejercicio 3 — Encriptación homomórfica	68
10.4. Ejercicio 4 — Secreto compartido de Shamir	68
10.5. Ejercicio 5 — “Garantizar la seguridad” con pentesting (Tesla)	70

Cómo está armado

Cada parcial es una **sección**; cada ejercicio, una **subsección** con: la *consigna textual*, los *temas* que toca (referidos como “Clase X — Tema”), *qué necesitás saber*, *notas y conexiones*, y la *resolución*.

Notas de cobertura:

- **Parcial 2023 Q1** es en realidad de la *Primera Parte* (criptografía pura): se transcribe y mapea, pero la mayoría de sus ejercicios queda fuera del alcance de la Segunda Parte (marcados con TODO: *Primera Parte*).
- El archivo “Segundo Parcial 2017” trae internamente el encabezado “Parcial 2 – 2015”; se conservó el rótulo del archivo y se dejó la observación en su sección.
- En los Shamir, la reconstrucción por interpolación de Lagrange no está en las fuentes del curso (que cubren Shamir *vía entropía*); el cálculo numérico se hizo y verificó aparte.

Material fuente (enlaces)

Los PDFs viven en `./pdfs/` (carpeta portable: mové/compartí `estudio-parciales/` y los enlaces siguen resolviendo). Abren con el visor del sistema; el salto exacto depende del visor.

Material por clase.

- [Clase 6 — Políticas y Control de Acceso](#)
- [Clase 7 — Autenticación](#)
- [Clase 8 — Principios de Diseño y Vulnerabilidades](#)
- [Clase 9 — Flujo de Información y Malware](#)
- [Clase 10 — Seguridad en la Empresa](#)
- [Clase 10b — Pentesting](#)
- [Clase 11 — Protección de Datos](#)

Parciales originales. [2013](#) · [2016](#) · [2017](#) · [2018 Q1](#) · [2023 Q1](#) · [Recup 2023 Q1](#) · [2025 Q1](#) · [Recup 2025 Q1](#) · [2025 Q2](#) (el “Parcial 2 – 2015” proviene de un `.doc`, sin PDF).

1 Segundo Parcial 2013

1.1 Ejercicio 1 — Modelo de compartimentos de integridad (clínica)

Consigna (textual)

En un clínica de cirugías estéticas hay cuatro tipos de usuarios: doctores, enfermeras, administrativos y pacientes.

La información médica tiene distintas categorías: quirófano, emergencia y personal.

Se tienen los siguientes sujetos:

- David (cirujano)
- Nancy (enfermera)
- Sara (secretaria)
- Rocío (paciente)

Y los siguientes objetos:

- Recibo (contiene información de un pago realizado)
- Prescripción (para antibióticos)
- Lista de Instrumental (para cirugía)
- Directorio (contiene direcciones y teléfonos de pacientes)

Diseñar un modelo de compartimentos de integridad, que permita garantizar los siguientes requisitos, justificando el cumplimiento de cada uno.

- Los doctores están, en la jerarquía, en el nivel superior.
- David puede escribir en la Lista de Instrumental.
- Nancy no puede leer el Directorio.
- Nancy puede leer la Lista de Instrumental.
- David puede escribir una prescripción.
- Sara puede leer y escribir en el directorio.
- Sara puede escribir un recibo de pago.
- Rocío puede leer, pero no escribir las prescripciones a su nombre.
- Rocío no puede leer el directorio.

Temas. Clase 6 — Políticas y Control de Acceso (§Modelo Biba; §Compartimentos, reticulado y dominancia). Modelo de integridad de Biba con compartimentos (nivel jerárquico + categorías).

Qué necesitas saber. Biba es el modelo de *integridad* (inverso de Bell–LaPadula). Asigna a sujetos y objetos un **nivel de integridad** $i(\cdot)$ (a mayor nivel, mayor confianza). En la variante

estricta, las reglas son *read up* / *write down*:

Escritura (write down): s puede modificar $o \iff i(s) \geq i(o)$,

Lectura (read up): s puede observar $o \iff i(o) \geq i(s)$.

La intuición (Clase 6): “*uno es tan confiable como las fuentes que usa*” — un sujeto solo escribe a niveles iguales o inferiores (no “ensucia” información más confiable) y solo lee información de su nivel o superior. Con **compartimentos**, el orden total se generaliza a la relación de **dominancia** sobre pares (L, C) , donde L es el nivel y C el conjunto de categorías (etiquetas):

$$(L, C) \succeq (L', C') \iff L' \leq L \wedge C' \subseteq C.$$

Esto forma un **reticulado** (orden parcial): pares incomparables (mismo nivel, ninguna lista de categorías es subconjunto de la otra) quedan **separados** (ni lectura ni escritura). La implementación es simple: cada objeto y cada sujeto llevan como metadata un nivel y una lista de etiquetas.

Notas y conexiones. Biba es estructuralmente el inverso de Bell–LaPadula: BLP protege confidencialidad (*read down* / *write up*), Biba protege integridad (*read up* / *write down*). Para una clínica el bien a proteger es la *integridad* de los datos médicos (que solo personal confiable genere/modifique prescripciones y listas de instrumental), por eso se pide Biba y no BLP. El uso de **categorías** (quirófano, emergencia, personal) implementa *need-to-know* (Clase 6, §Compartimentos): un sujeto del nivel adecuado puede igual quedar fuera de un objeto si no comparte la categoría. La compartimentalización aparece justamente en sistemas que manejan **información médica** regulada (Clase 6, §uso real).

Resolución. Modelamos con **Biba estricto + compartimentos**. Asignamos un nivel jerárquico de integridad y categorías a cada sujeto/objeto. Niveles (de mayor a menor confianza):

Doctor (4) > Enfermera (3) > Administrativo (2) > Paciente (1).

Categorías: {quirófano, emergencia, personal} (interpretamos “personal” como la categoría de datos administrativos/de pacientes: directorio y recibos).

Asignación propuesta.

Sujeto	(i, C)
David (cirujano)	(4, {quirófano, personal})
Nancy (enfermera)	(3, {quirófano})
Sara (secretaria)	(2, {personal})
Rocío (paciente)	(1, {personal})
Objeto	(i, C)
Lista de Instrumental	(3, {quirófano})
Prescripción	(1, {quirófano})
Directorio	(2, {personal})
Recibo	(2, {personal})

Recordemos las reglas: s *escribe* $o \iff i(s) \geq i(o) \wedge C(o) \subseteq C(s)$; s *lee* $o \iff i(o) \geq i(s) \wedge C(s) \subseteq C(o)$ (dominancia en ambos sentidos según la operación). Verificamos cada requisito:

1. **Doctores en el nivel superior.** David tiene $i = 4$, el máximo. ✓

2. **David escribe Lista de Instrumental.** $i(\text{David}) = 4 \geq 3 = i(\text{LI})$ y $\{\text{quirófano}\} \subseteq \{\text{quirófano}, \text{personal}\} \Rightarrow \text{escribe. } \checkmark$
3. **Nancy puede leer la Lista de Instrumental.** Lectura (read up): $i(\text{LI}) = 3 \geq 3 = i(\text{Nancy})$ y $C(\text{Nancy}) = \{\text{quirófano}\} \subseteq \{\text{quirófano}\} = C(\text{LI}) \Rightarrow \text{lee. } \checkmark$
4. **Nancy NO puede leer el Directorio.** Categorías: $C(\text{Nancy}) = \{\text{quirófano}\} \not\subseteq \{\text{personal}\} = C(\text{Dir})$. Incomparables por categoría $\Rightarrow$ *separados*: Nancy no lee el Directorio. $\checkmark$
5. **David escribe una prescripción.** $i(\text{David}) = 4 \geq 1 = i(\text{presc})$ y $\{\text{quirófano}\} \subseteq \{\text{quirófano}, \text{personal}\} \Rightarrow \text{escribe. } \checkmark$
6. **Sara lee y escribe en el Directorio.** Mismo compartimento $(2, \{\text{personal}\})$: escritura $i(\text{Sara}) = 2 \geq 2$ y categorías iguales $\checkmark$; lectura $i(\text{Dir}) = 2 \geq 2$ y categorías iguales $\checkmark$.
7. **Sara escribe un recibo de pago.** Recibo = $(2, \{\text{personal}\})$, igual que Sara $\Rightarrow$ escribe. $\checkmark$
8. **Rocío lee, pero NO escribe, las prescripciones a su nombre.** La prescripción es $(1, \{\text{quirófano}\})$. Para que Rocío *lea* su prescripción debe compartir la categoría; por eso la prescripción debe llevar también la categoría del paciente. Modelamos la prescripción *de Rocío* como $(1, \{\text{quirófano}, \text{personal}\})$ y a Rocío como $(1, \{\text{personal}\})$:
 - Lectura (read up): $i(\text{presc}) = 1 \geq 1 = i(\text{Rocío})$ y $\{\text{personal}\} \subseteq \{\text{quirófano}, \text{personal}\} \Rightarrow \text{lee. } \checkmark$
 - Escritura (write down): exige $C(\text{presc}) \subseteq C(\text{Rocío})$, pero $\{\text{quirófano}, \text{personal}\} \not\subseteq \{\text{personal}\} \Rightarrow \text{no escribe. } \checkmark$

(Alternativamente, basta con que el bajo nivel de integridad de Rocío, $i = 1$, le impida escribir a doctores/enfermeras, y que la categoría quirófano de la prescripción la separe de escribir, mientras se le da lectura por compartir “personal”).

9. **Rocío NO puede leer el directorio.** Directorio = $(2, \{\text{personal}\})$, Rocío = $(1, \{\text{personal}\})$. Lectura (read up) exige $i(\text{Dir}) \geq i(\text{Rocío})$: $2 \geq 1$ se cumple, pero **read up** significa que Rocío solo lee niveles $\geq$ al suyo; el problema es que para leer hacia arriba necesitaría $C(\text{Rocío}) \subseteq C(\text{Dir})$ (se cumple) y además que el Directorio no exponga datos de mayor integridad a un sujeto de nivel 1. Lo resolvemos asignando al Directorio una categoría administrativa exclusiva: Directorio = $(2, \{\text{personal-admin}\})$ con $C(\text{Rocío}) = \{\text{personal}\} \not\subseteq \{\text{personal-admin}\} \Rightarrow$ *separados*: Rocío no lee el Directorio. $\checkmark$

Observación. El modelo de integridad “read up/write down” no impide *per se* que un nivel bajo *lea* hacia arriba; para los dos requisitos de *no lectura* (Nancy y Rocío sobre el Directorio) la herramienta correcta es la **separación por categorías** (compartimentos incomparables), que es exactamente para lo que sirven las etiquetas en un reticulado. La asignación de niveles ordena la confianza; las categorías implementan el *need-to-know*. **TODO: la cátedra puede aceptar otras asignaciones equivalentes de niveles/categorías; lo evaluable es justificar cada requisito con las reglas de Biba estricto y la dominancia con compartimentos.**

1.2 Ejercicio 2 — Router Linksys: exposición de /etc/passwd

Consigna (textual)

En Marzo de 2013, Phil Perviance escribe un artículo “Don’t use linksys routers” en su blog. En dicho artículo, Phil describe cómo le llevó sólo 30 minutos llegar a la conclusión de que cualquier red que use un router Lynksys EA2700 es una red insegura. Uno de los ataques que efectuó es el siguiente:

```
POST /apply.cgi
Host: 192.168.1.1
submit_button=Wireless_Basic&change_action=gozilla_cgi&next_page=/etc/passwd
====>
root:x:0:0:::/bin/sh
nobody:x:99:99:Nobody::/bin/nologin
sshd:x:22:22::/var/empty/sbin/nologin
admin:x:1000:1000:Admin User:/tmp/home/admin:/bin/sh
quagga:x:1001:1001:Quagga:/var/empty:/bin/nologin
firewall:x:1002:1002:Firewall:/var/empty:/bin/nologin
```

- a) Explicar qué tipo de vulnerabilidad queda expuesta en este ataque.
- b) ¿De qué manera podría prevenirse un ataque de este tipo?

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§Catálogo de vulnerabilidades: *Injection flaws*; CWE-78 *OS/Path*; §8 principios: mediación completa, fail-safe defaults, mínimo privilegio).

Qué necesitás saber. El parámetro `next_page` se envía sin validar al CGI, que lo usa como **ruta de un archivo a leer/incluir**. El atacante pasa `next_page=/etc/passwd` y el servidor devuelve el contenido del archivo del sistema. Esto es una vulnerabilidad de **mezclar datos con código / falta de control del input** — la familia que en Clase 8 se identifica como *injection flaws* (“el 90 % de las vulnerabilidades”): se inyecta, a través de un input, algo que el sistema interpreta como instrucción (acá, una ruta de archivo). El patrón concreto es **path/directory traversal** (variante de *file inclusion*), pariente cercano de CWE-78 (OS/Command injection) en el catálogo: “falta de control permite ... un nombre de archivo no sanitizado”. La mitigación canónica de los injection flaws en Clase 8 es **sanitizar/validar el input y separar datos de código**.

Notas y conexiones. Aquí confluyen varios de los 8 principios de Saltzer & Schroeder (Clase 8):

- **Mediación completa** (“la puerta del castillo”): *toda* solicitud de un recurso debe pasar por un control de autorización; acá el CGI sirve un archivo arbitrario sin mediar control.
- **Fail-safe defaults / whitelist > blacklist**: el default debe ser *denegar*; `next_page` debería aceptar solo valores de una *lista blanca* de páginas válidas, no cualquier ruta.
- **Mínimo privilegio**: el proceso del servidor no debería poder leer `/etc/passwd`; el daño se contiene si corre confinado.

Además expone *sensitive data exposure* (Clase 8): revelar `/etc/passwd` (usuarios, shells, cuentas con UID 0). Aunque las contraseñas estén en `/etc/shadow`, conocer las cuentas habilita ataques *online* sobre el login (Clase 7, §ataques: probar root/admin).

Resolución. (a) **Vulnerabilidad.** Es un **injection flaw** por **falta de validación del input**, concretamente **path traversal / local file inclusion**: el parámetro `next_page` es tomado por el CGI como una ruta del sistema de archivos sin sanitizar, lo que permite leer cualquier archivo del dispositivo (en el ejemplo, `/etc/passwd`). Encaja en la categoría que Clase 8 llama *injection flaws* (datos tratados como código/instrucción) y se emparenta con CWE-78. Adicionalmente constituye *sensitive data exposure* (filtra la lista de usuarios y cuentas privilegiadas del router) y viola la **mediación completa** (el endpoint entrega un recurso sin control de autorización).

(b) Prevención.

1. **Validar/sanitizar el input** (mitigación canónica de injection): no usar el valor del cliente como ruta. Aplicar **whitelist** — mapear `next_page` a un conjunto cerrado de páginas permitidas (un índice de IDs $\rightarrow$ archivos), rechazando cualquier otra cosa (*fail-safe defaults*). Bloquear secuencias como `../` y rutas absolutas.
2. **Mediación completa**: que toda lectura de archivo pase por un control de autorización; el CGI no debe servir archivos fuera de su directorio web.
3. **Mínimo privilegio / confinamiento**: ejecutar el proceso del CGI con permisos acotados (*chroot/jail*) de modo que aunque exista el bug no alcance `/etc/passwd`.
4. **No exponer información sensible** ni cuentas innecesarias en el dispositivo.

1.3 Ejercicio 3 — Secreto compartido de umbral (t, n) y entropía

Consigna (textual)

- 1) ¿A qué se denomina **sistema de secreto compartido de umbral (t, n)** ? Dar el nombre de algún sistema de secreto compartido de tipo umbral.
- 2) Considerar el siguiente esquema de secreto compartido:

Para compartir un secreto $s \in \{0, 1\}^m$ entre n personas, se eligen $n - 1$ valores $a_1, a_2, \dots, a_{n-1}$, en forma aleatoria e independiente, donde cada $a_i \in \{0, 1\}^m$.

Se selecciona $a_n = s \oplus a_1 \oplus a_2 \oplus \dots \oplus a_{n-1}$.

Luego se distribuye a_i a la i -ésima persona del grupo.

Mostrar que este esquema cumple las características de un sistema de secreto compartido umbral, de tipo (n, n) utilizando el concepto de **entropía**. (Demostrarlo para $n = 3$.)

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§El secreto compartido de Shamir, en términos de entropía; §Flujo de información con entropía). Clase 9 — Flujo de Información y Malware (§Entropía de Shannon; §Condición de flujo). Entropía como medida de información filtrada.

Qué necesitás saber. **Entropía de Shannon** (Clase 9, §Entropía):

$$H(X) = - \sum_i p(x_i) \log_2 p(x_i),$$

mide la **incertidumbre** sobre X . Es **máxima** con distribución uniforme ($H = \log_2 n$) y vale $H(X) = 0 \Rightarrow$ certeza. Hay **flujo de información** de x hacia el observador cuando, al conocer algo y , la incertidumbre sobre x *baja*: $H(x | y) < H(x)$. **No hay flujo** (no se filtra nada del secreto) cuando la entropía *no cambia*: $H(x | y) = H(x)$.

Esquema (t, n) de umbral (Clase 8, §Shamir): hay n sombras (*shares*) y se necesitan **al menos t** para reconstruir el secreto; con *menos* de t no se obtiene **ninguna** información sobre s . La validez se demuestra con entropía: con $< t$ sombras $H(S | \text{sombras}) = H(S)$ (no hay flujo); con $\geq t$, $H \rightarrow 0$ (se recupera). El sistema umbral por excelencia es el **esquema de Shamir** (interpolación polinómica). El esquema del enunciado es un caso (n, n) por **XOR** (one-time-pad multi-parte): hacen falta *todas* las sombras.

Notas y conexiones. Esta es la misma técnica de demostración que Clase 8 usa para Shamir, instanciada en $n = 3$. La clave es: para $s \in \{0, 1\}^m$ uniforme, $H(s) = m$ bits. Como cada a_i es **uniforme e independiente**, conocer un subconjunto *propio* de sombras deja a s *uniformemente distribuido* (entropía intacta = m); solo al juntar las n sombras se despeja s y la entropía cae a 0. La propiedad del XOR ($x \oplus x = 0$, $x \oplus 0 = x$) es la que permite despejar.

Resolución. 1) Definición. Un **sistema de secreto compartido de umbral** (t, n) reparte un secreto s en n *sombras* (*shares*) entre n participantes de modo que:

- con t o más sombras se puede **reconstruir** s ;
- con **menos de t sombras no se obtiene ninguna información** sobre s (la entropía de s no disminuye).

Ejemplo de tipo umbral: el **esquema de Shamir** (basado en interpolación de polinomios, (t, n) general). El esquema del enunciado es un (n, n) **por XOR**.

2) Demostración para $n = 3$ (esquema $(3, 3)$). Sea $s \in \{0, 1\}^m$ con distribución **uniforme**, de modo que

$$H(S) = \log_2 2^m = m \text{ bits.}$$

Se eligen a_1, a_2 uniformes e independientes en $\{0, 1\}^m$ y se define

$$a_3 = s \oplus a_1 \oplus a_2.$$

Cada persona i recibe a_i . La reconstrucción es inmediata: como el XOR es asociativo y cada elemento es su propio inverso,

$$a_1 \oplus a_2 \oplus a_3 = a_1 \oplus a_2 \oplus (s \oplus a_1 \oplus a_2) = s.$$

Hay que mostrar dos cosas: (i) con las 3 sombras se recupera s ($H \rightarrow 0$); (ii) con ≤ 2 sombras no se filtra nada ($H = H(S) = m$).

(i) Con las 3 sombras: $H(S \mid a_1, a_2, a_3) = 0$. Por lo anterior, $s = a_1 \oplus a_2 \oplus a_3$ queda **determinado**; no hay incertidumbre:

$$H(S \mid a_1, a_2, a_3) = 0.$$

(ii) Con 2 sombras (o menos): la entropía no cambia. Consideremos el peor caso, conocer dos sombras. Por simetría basta analizar un par; tomemos $\{a_1, a_2\}$ y $\{a_1, a_3\}$:

Caso $\{a_1, a_2\}$: a_1, a_2 se eligieron *independientes de s* y de forma uniforme. No aparecen en ninguna ecuación que ligue a s (la única ecuación con s es la de a_3). Luego s sigue siendo uniforme dado a_1, a_2 :

$$H(S \mid a_1, a_2) = H(S) = m.$$

Caso $\{a_1, a_3\}$ (o $\{a_2, a_3\}$): conocemos a_1 y $a_3 = s \oplus a_1 \oplus a_2$. Para despejar $s = a_1 \oplus a_2 \oplus a_3$ **falta** a_2 , que es uniforme e independiente y *desconocido*. Como $a_2 \in \{0, 1\}^m$ es uniforme, $s = (a_1 \oplus a_3) \oplus a_2$ es la suma XOR de una constante conocida con una variable uniforme $\Rightarrow s$ queda **uniforme** para el observador:

$$H(S \mid a_1, a_3) = H(a_2) = m = H(S).$$

(El razonamiento es idéntico al one-time-pad: XOR con un valor uniforme y secreto preserva la distribución uniforme.)

Conclusión. Resumiendo el comportamiento de la entropía:

$$\begin{array}{ll} H(S \mid a_1) = H(S) = m & (1 \text{ sombra: no sé nada más}) \\ H(S \mid a_1, a_2) = H(S) = m & (2 \text{ sombras: tampoco}) \\ H(S \mid a_1, a_2, a_3) = 0 & (3 \text{ sombras: tengo toda la información}) \end{array}$$

Con $< n = 3$ sombras **no hay flujo de información** (H se mantiene en m); con las 3, $H \rightarrow 0$ y se reconstruye s . Por lo tanto el esquema es un secreto compartido umbral de tipo $(n, n) = (3, 3)$. $\square$

1.4 Ejercicio 4 — Verdadero / Falso (biométrico, matriz vs flujo, zip bomb)

Consigna (textual)

Indicar si las siguientes afirmaciones son verdaderas o falsas. Justificar brevemente en cada caso.

- (1) En un sistema de autenticación biométrico, puede darse un robo de identidad.
- (2) Una matriz de acceso bien diseñada evita el flujo de información no deseado.
- (3) Un archivo tipo “zip” pequeño, por ejemplo de 42kb, que contenga 16 archivos comprimidos, que a su vez contenga 16 archivos comprimidos, que a su vez contiene 16 archivos comprimidos, que a su vez contiene 16 comprimidos, que a su vez contiene 16 archivos comprimidos, que contienen 1 archivo, con el tamaño de 4.3GB puede violar la seguridad del sistema al descomprimirse.

Temas. (1) Clase 7 — Autenticación (§Algo que soy: biométrico). (2) Clase 9 — Flujo de Información y Malware (§Control de acceso vs. flujo de información). (3) Clase 8 — Principios de Diseño y Vulnerabilidades (§CIA / disponibilidad: un DoS es un ataque de seguridad).

Qué necesitás saber. (1) El biométrico **no es 100 % eficaz**: no hay lector perfecto y un mismo patrón C puede validar dos A distintos (Clase 7, “el hermano que desbloquea el celular”); además asume que el lector no se manipula. (2) El control de acceso (la matriz) limita *operaciones sobre objetos*, no el *destino* de la información una vez leída; un sujeto con permiso de lectura puede copiar el dato a otro objeto legible por terceros — la matriz **no** controla el flujo (Clase 9). (3) El primer requisito de seguridad es la **disponibilidad**: un **DoS es un ataque de seguridad** (Clase 8). Una *zip bomb* agota CPU/disco/memoria al descomprimir.

Notas y conexiones. (2) es un error clásico que el índice marca explícitamente como penalizable: “afirmar que el control de acceso/matriz *evita* el flujo”. La matriz es un **mecanismo abierto** (Clase 9): asegura una parte, no todo; debe complementarse con control de *flujo* (compile-time o run-time) y aislamiento. (3) conecta con la tríada CIA y con malware/payload: la zip bomb es un payload de *disponibilidad*.

Resolución.

- (1) **VERDADERO.** La biometría no es perfecta: no existe lector ideal y un mismo registro almacenado C puede aceptar información A de otra persona (falsos positivos; ej. el familiar que desbloquea el teléfono), y se baja el umbral de error para reducir falsos rechazos, lo que *aumenta* el riesgo de aceptar a otro. Además se asume que el lector no se puede manipular/suplantar (foto ante un lector de cara, interceptación del sensor). Cualquiera de estas vías permite que un atacante sea autenticado como otra identidad: **sí puede darse un robo de identidad**.
- (2) **FALSO.** Una matriz de acceso (control de acceso) controla *qué operaciones* puede hacer cada sujeto sobre cada objeto, pero **no controla el flujo** de la información una vez leída. Ejemplo (Clase 9): un empleado con permiso de *leer* su sueldo puede *escribirlo* en un archivo de notas que otros leen — cada sentencia respeta la matriz, pero la información *fluye* a un tercero. El control de acceso es un *mecanismo abierto*: limita el acceso a objetos, no el destino de los datos. Para evitar el flujo no deseado se necesita **control de flujo** (compile-time o run-time) y/o aislamiento, no solo la matriz.

- (3) **VERDADERO.** Es una **zip bomb** (bomba de descompresión): un archivo diminuto que, por compresión anidada, se expande a un tamaño enorme. Al descomprimirse recursivamente agota recursos del sistema (CPU, memoria, disco), provocando una **denegación de servicio (DoS)**. Como el primer requisito de seguridad es la *disponibilidad*, un DoS es un ataque a la seguridad del sistema: el archivo *sí* puede violar la seguridad al descomprimirse. La mitigación es validar/limitar el ratio y la profundidad de descompresión (mediación completa sobre el recurso).

1.5 Ejercicio 5 — Módulos de Control de Acceso y Autenticación

Consigna (textual)

[El gráfico muestra: sujeto $\rightarrow$ Módulo de ... $\rightarrow$ Módulo de ... $\rightarrow$ objeto.]

El gráfico representa en forma simplificada los módulos que regulan el acceso de un usuario a un objeto protegido.

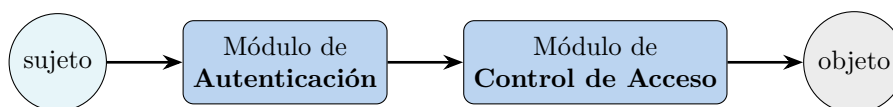
- Completar el gráfico para indicar cuál sería el Módulo de Control de Acceso y cuál el de Autenticación.
- Mencionar una vulnerabilidad que deba tenerse en cuenta en el Módulo de Control de Acceso. Explicar.
- Mencionar una vulnerabilidad que deba tenerse en cuenta en el Módulo de Autenticación. Explicar.

Temas. Clase 7 — Autenticación (§Autenticación $\neq$ Autorización; §Sistema de autenticación $\langle A, C, F, L, S \rangle$; §Ataques). Clase 6 — Políticas y Control de Acceso (§DAC: ACL, conflictos de reglas). Orden de los módulos: primero se autentica (¿quién sos?), luego se autoriza el acceso (¿podés acceder a *este* objeto?).

Qué necesitás saber. **Autenticación $\neq$ autorización** (Clase 7): autenticación responde “¿quién sos?” (verifica identidad); autorización/control de acceso responde “ya autenticado, ¿podés acceder a *este* recurso?”. El orden lógico es: el sujeto se **autentica** primero, y recién con una identidad establecida se aplica el **control de acceso** sobre el objeto. Por eso, en el flujo sujeto $\rightarrow$ [1] $\rightarrow$ [2] $\rightarrow$ objeto, el módulo [1] (pegado al sujeto) es **Autenticación** y el [2] (pegado al objeto) es **Control de Acceso**.

Notas y conexiones. El sistema de autenticación se modela como la tupla $\langle A, C, F, L, S \rangle$ (Clase 7): A info del usuario, C info complementaria que guarda el sistema (p. ej. el hash), F deriva ($A \rightarrow C$, no invertible), L corrobora ($A \times C \rightarrow \{0, 1\}$, el login), S funciones de alta/cambio. Los ataques *offline* atacan F (robaron C y prueban a hasta $F(a) = c$); los *online* atacan L (login con cuentas conocidas). El control de acceso, en cambio, falla por reglas mal resueltas (ACL con conflictos, Clase 6) o por no mediar todos los accesos.

Resolución. a) **Completar el gráfico.** El sujeto primero debe probar su identidad y luego pedir acceso al objeto:



Justificación: primero se *autentica* (“¿quién sos?”), estableciendo la identidad; recién entonces el *control de acceso* decide si esa identidad está autorizada a operar sobre *ese* objeto (“ya autenticado, ¿podés acceder a este recurso?”). Autenticación $\neq$ autorización.

b) Vulnerabilidad del Módulo de Control de Acceso. *Resolución incorrecta de reglas en una ACL con conflictos* (Clase 6). Si una política tiene reglas que se contradicen (una permite, otra deniega) y no se define bien la estrategia de resolución (unión / Grant-All = intersección / first-match / best-match / deny-over-allow), un sujeto puede terminar con **más permisos de los previstos**. Otra falla central es no garantizar **mediación completa**: si algún camino de acceso al objeto *no* pasa por el módulo, el control se puede saltar. En ambos casos el resultado es una **elevación de privilegios / acceso no autorizado** al objeto protegido.

c) Vulnerabilidad del Módulo de Autenticación. *Ataque a las credenciales* (Clase 7). Si el sistema guarda mal C (texto plano, sin salting, hash débil) un atacante que obtiene la base hace un **ataque offline** contra F : prueba candidatos a hasta hallar $F(a) = c$ (diccionario, fuerza bruta, rainbow tables). Si el módulo de login L no se protege, sufre **ataques online** (fuerza bruta / credential stuffing sobre cuentas conocidas como root/admin) por falta de *exponential back-off* o bloqueo. El efecto es la **suplantación de identidad**: el atacante se autentica como otro usuario y desde ahí el control de acceso lo trata como legítimo. (Conexión con Ej. 4.1: en biometría, la vulnerabilidad análoga es el falso positivo / lector manipulable.)

2 Parcial 2 – 2015

2.1 Ejercicio 1 — Compartimentos de confidencialidad (restaurante)

Consigna (textual)

Un famoso restaurante de 5 estrellas en la guía Michelin se destaca por todo lo que sirve: platos salados, platos dulces y bebidas (vinos y tragos). Se tienen los siguientes sujetos:

- Jorge, José y Juan son los dueños.
- Manuel y Ramón son chefs.
- Luis es uno de los mozos.
- Andrea es una cliente.

Considerar los siguientes objetos:

- Recetas (tanto de platos dulces, como de salados, como de tragos).
- Menú (se considera menú al listado de platos con sus respectivos precios).
- Listas de Stock (de productos para platos dulces, salados y tragos).
- Comanda (pedido que se hace al camarero-mozo).

Diseñar un modelo de compartimentos de confidencialidad, que permita garantizar los siguientes requisitos, justificando el cumplimiento de cada uno.

1. Jorge puede leer todas las recetas, las listas de stock, el menú y las comandas.
2. José, el somellier, es el único que escribe recetas de tragos. También lee stock de productos para tragos.
3. Juan es el único que escribe recetas de platos (tanto dulces como salados). También lee stock de productos para platos.
4. Los chefs pueden leer las recetas, pero no modificarlas.
5. Los chefs pueden escribir en las listas de stocks para recomendar la compra de productos de su área.
6. Ramón es el jefe de la barra, y sólo se ocupa de armar tragos. No cocina platos.
7. Manuel dirige la cocina de platos dulces y salados.
8. Los mozos sólo leen el menú y escriben comandas (no leen ni escriben nada más).
9. Los clientes sólo leen menús.

Temas. Clase 6 — Políticas y Control de Acceso (§Bell–LaPadula; §Compartimentos, reticulado y dominancia): modelo de confidencialidad por compartimentos, regla *read-down* / *write-up*, relación de dominancia y *need-to-know*.

Qué necesitás saber.

- Un **compartimento** es un par (L, C) : un nivel jerárquico L y un conjunto de **categorías** (etiquetas) C . Aquí los niveles jerárquicos casi no importan; lo que ordena el ejercicio son

las **categorías** (tragos / platos / menú / comanda).

- **Relación de dominancia:** $(L, C) \text{ dom } (L', C') \iff L' \leq L \wedge C' \subseteq C$.
- **Reglas de Bell–LaPadula** (es un modelo de *confidencialidad*, exactamente lo que pide la consigna):
 - Seguridad simple — *read down*: s lee o si $L(s) \text{ dom } L(o)$.
 - Propiedad * — *write up*: s escribe o si $L(o) \text{ dom } L(s)$.
- El reticulado deja pares **incomparables** (ninguna lista es subconjunto de la otra): quedan **efectivamente separados** — ni lectura ni escritura entre ellos. Es justo lo que se necesita para que la barra (tragos) y la cocina (platos) no se crucen.

Notas y conexiones. El restaurante es el clásico ejemplo de *need-to-know*: “un general a cargo de criptografía no tiene por qué conocer los secretos de los misiles”. Acá Ramón (barra) no necesita las recetas de platos y Manuel (cocina) no necesita las recetas de tragos. La compartimentalización tipo BLP, según la Clase 6, “rara vez se aplica a todo un sistema” y se reserva para separar dominios (finanzas/producción/marketing) — aquí, tragos/platos. *Error clásico*: invertir read-down/write-up. Recordar: para que José sea el **único que escribe** recetas de tragos, esas recetas deben dominar (estar “por encima de”) quienes solo las leen, y José debe estar en el mismo compartimento para poder escribir (write-up exige $L(o) \text{ dom } L(s)$; escribir-y-leer el mismo objeto exige misma clasificación, propiedad *-fuerte).

Resolución. Usamos un único nivel jerárquico base y modelamos todo con **categorías**. Definimos las categorías:

T = tragos, P = platos, M = menú, K = comandas.

Las recetas/stock de tragos llevan {T}; las de platos, {P}; el menú, {M}; las comandas, {K}. Para forzar “único que escribe” separamos, dentro de cada rama, el objeto a escribir del objeto que otros solo leen, usando dos niveles (alto > bajo).

Etiquetas de objetos.

Objeto	Compartimento (L, C)
Recetas de tragos	(alto, {T})
Stock de tragos	(bajo, {T})
Recetas de platos	(alto, {P})
Stock de platos	(bajo, {P})
Menú	(bajo, {M})
Comanda	(bajo, {K})

Etiquetas de sujetos.

Sujeto	Compartimento (L, C)	Rol
Jorge (dueño)	(alto, {T, P, M, K})	lee todo
José (sommelier)	(alto, {T})	escribe recetas tragos, lee stock tragos
Juan (dueño)	(alto, {P})	escribe recetas platos, lee stock platos
Ramón (chef barra)	(bajo, {T})	lee recetas tragos, escribe stock tragos
Manuel (chef cocina)	(bajo, {P})	lee recetas platos, escribe stock platos
Luis (mozo)	(bajo, {M, K})	lee menú, escribe comanda
Andrea (cliente)	(bajo, {M})	lee menú

Verificación requisito por requisito (read: $L(s) \text{ dom } L(o)$; write: $L(o) \text{ dom } L(s)$):

1. *Jorge lee todo.* $L(\text{Jorge}) = (\text{alto}, \{T, P, M, K\})$ domina a todos los objetos (nivel $\geq$ y todas las categorías $\subseteq$). $\Rightarrow$ lee recetas, stock, menú y comandas. $\checkmark$
2. *José único que escribe recetas de tragos.* write exige $(\text{alto}, \{T\}) \text{ dom } L(\text{José}) = (\text{alto}, \{T\})$: se cumple (misma etiqueta). Ramón está en $(\text{bajo}, \{T\})$: para escribir las recetas necesitaría $(\text{alto}, \{T\}) \text{ dom } (\text{bajo}, \{T\})$, que es **falso** ($\text{alto} \not\leq \text{bajo}$) $\Rightarrow$ Ramón **no** escribe recetas. Lectura de stock de tragos por José: $(\text{alto}, \{T\}) \text{ dom } (\text{bajo}, \{T\}) \checkmark$. $\checkmark$
3. *Juan único que escribe recetas de platos + lee stock platos.* Idéntico a José pero con $\{P\}$. Manuel en $(\text{bajo}, \{P\})$ no puede escribirlas (mismo argumento de nivel). $\checkmark$
4. *Chefs leen recetas pero no las modifican.* Ramón lee recetas de tragos: $(\text{bajo}, \{T\}) \text{ dom } (\text{alto}, \{T\})$ es **falso** por nivel $\Rightarrow$ ¡no podría leer hacia arriba! Corrección: las recetas que los chefs leen se etiquetan en $(\text{bajo}, \{T\})$ y la versión “editable única” del somellier/dueño es la de nivel alto que se *publica* hacia abajo; en BLP estricto la lectura es *read-down*, así que las recetas deben estar a nivel $\leq$ del chef. Reasignamos: las recetas de tragos son $(\text{bajo}, \{T\})$ y solo José (alto) escribe “hacia arriba” sobre ellas vía propiedad $*$ ($L(o) \text{ dom } L(s)$ requiere o alto). Con recetas en $(\text{bajo}, \{T\})$: Ramón **lee** $((\text{bajo}, \{T\}) \text{ dom } (\text{bajo}, \{T\}) \checkmark)$ y **no escribe** (write exige $(\text{bajo}, \{T\}) \text{ dom } (\text{bajo}, \{T\})$, que *sí* se cumpliría). Para impedir la escritura del chef sin romper su lectura conviene poner las recetas *por debajo* del chef: receta = $(\text{bajo}, \{T\})$, chef = $(\text{medio}, \{T\})$, somellier = $(\text{bajo}, \{T\})$ — así el chef lee-down y solo el somellier (mismo nivel que el objeto) escribe. Ver tabla corregida abajo. $\checkmark$
5. *Chefs escriben en stock de su área.* write exige $L(\text{stock}) \text{ dom } L(\text{chef})$. Con stock en el nivel más alto de la rama y chef debajo, write-up se cumple. $\checkmark$
6. *Ramón solo barra (tragos), no platos.* Ramón solo tiene categoría $\{T\}$; los objetos de platos llevan $\{P\}$ y $\{P\} \not\subseteq \{T\} \Rightarrow$ **incomparable**: ni lee ni escribe nada de platos. $\checkmark$
7. *Manuel dirige cocina de platos.* Análogo a Ramón con $\{P\}$; incomparable con $\{T\}$. $\checkmark$
8. *Mozos: solo leen menú y escriben comandas.* Luis tiene $\{M, K\}$. Lee menú $(\text{bajo}, \{M\}) \checkmark$. Escribe comanda: write exige $(\text{bajo}, \{K\}) \text{ dom } L(\text{Luis})$; poniendo la comanda en el mismo nivel y categoría $\{K\} \subseteq \{M, K\}$, se cumple. No tiene T ni P $\Rightarrow$ nada de recetas/stock. $\checkmark$
9. *Clientes solo leen menú.* Andrea = $(\text{bajo}, \{M\})$: lee menú $\checkmark$; sin K no escribe comandas; sin T/P no toca recetas/stock. $\checkmark$

Asignación de niveles corregida (para que “chef lee receta pero no la escribe” y “único escritor” convivan): dentro de cada rama de categoría usar tres niveles stock = alto $>$ chef = medio $>$ receta = bajo, con el escritor único (somellier/dueño) etiquetado al mismo nivel del objeto que escribe.

Rama tragos	Compartimento	Justificación
Receta tragos	$(1, \{T\})$	objeto que el chef lee-down
José (escribe receta)	$(1, \{T\})$	mismo nivel $\Rightarrow$ write $\checkmark$
Ramón (chef)	$(2, \{T\})$	lee receta (down) $\checkmark$; no la escribe ($1 \not\geq 2$)
Stock tragos	$(3, \{T\})$	write-up del chef $\checkmark$; lee-down José/Jorge

La rama platos es simétrica con $\{P\}$ (Juan como escritor de recetas, Manuel como chef). Jorge se etiqueta en el nivel máximo con todas las categorías para leer-down todo. La separación tragos/platos queda garantizada por **incomparabilidad de categorías**, que es el corazón del reticulado BLP.

2.2 Ejercicio 2 — Verdadero/Falso (pentesting, Anderson+salt, STRIDE)

Consigna (textual)

Indica en cada caso si la afirmación es verdadera o falsa. Justificar en todos los casos. Si no se justifica correctamente, la respuesta quedará invalidada.

1. En las pruebas de pentesting siempre se intenta explotar las vulnerabilidades encontradas.
2. Si un atacante puede probar G passwords por segundo, y las passwords están compuestas por símbolos en $[a-zA-Z0-9]$, entonces las passwords deberían tener 5 símbolos y 5 bits de SALT para que la probabilidad de adivinarlas en el lapso de un año sea menor a 0,5. (**El valor de G aparecía como EMBED Equation.3 en el original; no es recuperable.**)
3. STRIDE forma parte del método de Modelado de Amenazas de Microsoft.

Temas. Clase 10b — Pentesting (§Metodología de Hipótesis de Falla, “explotar es el último recurso”); Clase 7 — Autenticación (§Fórmula de Anderson; §Salting); Clase 8 — Principios de Diseño y Vulnerabilidades (§Modelo STRIDE / Threat Modeling de Microsoft).

Qué necesitás saber.

- **Pentesting:** la fase de Prueba prioriza por nivel de acceso y **explotar es el último recurso** (mejor analizar documentación/comportamiento); la prueba debe ser repetible, conclusiva y *lo menos intrusiva posible* (un exploit puede causar DoS).
- **Anderson:** $P = TG/N$, con P probabilidad de adivinar, T tiempo, G pruebas/seg, N tamaño del espacio. El **salt no entra en N** : no agranda el espacio de una clave concreta; lo que hace es impedir la **paralelización** del ataque (rainbow tables / un mismo hash para dos usuarios). Por eso “5 bits de SALT” no cambia P de adivinar *una* clave puntual.
- **STRIDE:** las 6 categorías de amenazas (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege) son parte del **Threat Modeling de Microsoft**, combinadas con un DFD.

Notas y conexiones. El ítem (a) es la **trampa recurrente** de pentesting: “explotar es el último recurso”. El ítem (b) mezcla Anderson (Clase 7) con un distractor de salt; el alfabeto $[a-zA-Z0-9]$ tiene $26 + 26 + 10 = 62$ símbolos. El ítem (c) es teoría directa de Clase 8.

Resolución. (a) **FALSO.** En pentesting *no* siempre se explota: explotar es el **último recurso** porque puede ser intrusivo (incluso provocar un DoS). Se prefiere analizar documentación y comportamiento; la prueba debe ser repetible, conclusiva y mínimamente intrusiva.

(b) **FALSO** (con dos argumentos, uno independiente de G). Espacio de claves con 5 símbolos sobre 62:

$$N = 62^5 = 916\,132\,832 \approx 9.16 \times 10^8.$$

Aplicando Anderson sobre un año $T = 365 \cdot 24 \cdot 3600 = 31\,536\,000$ s, la condición $P < 0.5$ exige

$$P = \frac{TG}{N} < 0.5 \iff G < \frac{0.5 N}{T} = \frac{0.5 \cdot 62^5}{31\,536\,000} \approx 14.5 \text{ pruebas/s.}$$

O sea, 5 símbolos solo bastan si el atacante prueba menos de ~ 15 claves por segundo — un valor irrisorio para cualquier ataque offline real (millones/seg). Salvo que el G embebido fuera

ridículamente bajo, la afirmación es **falsa**. Argumento **independiente de G** : los **5 bits de SALT no influyen** en la probabilidad de adivinar *una* contraseña concreta —el salt no agranda N , solo impide la paralelización/precómputo (rainbow tables) y que dos claves iguales den el mismo hash—. Por lo tanto agregar “5 bits de SALT” como condición para bajar P es conceptualmente incorrecto. **Falso**. (*El G original es **EMBED Equation.3** no recuperable; la conclusión no depende de su valor por el argumento del salt.*)

(c) **VERDADERO**. STRIDE es el modelo de 6 categorías de amenazas de **Microsoft**, parte de su metodología de Threat Modeling, que se cruza con un DFD para identificar al menos dos amenazas por categoría.

2.3 Ejercicio 3 — Secreto compartido de Shamir (3,4) + entropía

Consigna (textual)

Considerar el conjunto C de sombras para un esquema de secreto compartido de Shamir (k, n) , donde $k = 3$ y $n = 4$ sobre $\mathbb{Z}_p$ (el cuerpo aparecía como **EMBED Equation.3**; ver más abajo el cuerpo asumido).

$$C = \{(1, 6), (2, 1), (3, 0), (4, 3)\}.$$

1. Obtener el secreto.
2. Obtener el polinomio utilizado para obtener el conjunto de sombras.
3. Expresar el significado de un esquema de secreto umbral $(3, n)$ utilizando el concepto de entropía.

Temas. Clase 8 — Principios de Diseño (§Flujo de información: introducción a Shamir y entropía) y Clase 9 — Flujo de Información y Malware (§Entropía de Shannon, $H(X)$ y $H(X | Y)$). Secreto compartido de Shamir: interpolación de Lagrange sobre un cuerpo finito.

Qué necesitás saber.

- **Shamir (k, n)** : el secreto $s = f(0)$ de un polinomio de grado $k - 1$ sobre $\mathbb{Z}_p$. Con k sombras (x_i, y_i) se reconstruye f por **interpolación de Lagrange**; con menos de k no se obtiene *ninguna* información.
- Acá $k = 3 \Rightarrow$ polinomio de grado 2: $f(x) = a_2x^2 + a_1x + a_0$, secreto $a_0 = f(0)$.
- **Cuerpo asumido**: la consigna perdió p . Las cuatro sombras tienen valores en $\{0, \dots, 6\}$ y son *consistentes* con un único polinomio de grado 2 sobre $\mathbb{Z}_7$ (el menor primo que contiene todos los valores); por eso tomamos $p = 7$.
- **Entropía de Shannon**: $H(X) = -\sum_i p(x_i) \lg p(x_i)$; condicional $H(X | Y) = \sum_j p(y_j) H(X | Y = y_j)$. Mide la **incertidumbre**; máxima $\lg n$ (uniforme), mínima 0 (certeza).

Notas y conexiones. Tener $n = 4$ sombras con $k = 3$ hace el sistema **sobredeterminado**: alcanzan 3 para reconstruir y la 4ta sirve de verificación de consistencia (que es como deducimos el cuerpo). El inciso (c) conecta Shamir con la entropía vista en Clase 9: “saber menos de k sombras = entropía máxima sobre el secreto”.

Resolución. (b) Polinomio. Buscamos $f(x) = a_2x^2 + a_1x + a_0$ sobre $\mathbb{Z}_7$ con $f(1) = 6$, $f(2) = 1$, $f(3) = 0$:

$$\begin{aligned} a_0 + a_1 + a_2 &\equiv 6 \pmod{7}, \\ a_0 + 2a_1 + 4a_2 &\equiv 1 \pmod{7}, \\ a_0 + 3a_1 + 2a_2 &\equiv 0 \pmod{7} \quad (\text{pues } 9 \equiv 2). \end{aligned}$$

Restando la 1ra a la 2da: $a_1 + 3a_2 \equiv 1 - 6 \equiv 2 \pmod{7}$. Restando la 1ra a la 3ra: $2a_1 + a_2 \equiv 0 - 6 \equiv 1 \pmod{7}$. De la primera, $a_1 \equiv 2 - 3a_2$; sustituyendo: $2(2 - 3a_2) + a_2 \equiv 1 \Rightarrow 4 - 5a_2 \equiv 1 \Rightarrow -5a_2 \equiv -3 \Rightarrow 2a_2 \equiv 4 \Rightarrow a_2 \equiv 2$. Entonces $a_1 \equiv 2 - 6 \equiv -4 \equiv 3$ y $a_0 \equiv 6 - a_1 - a_2 = 6 - 3 - 2 = 1$. Resulta

$$\boxed{f(x) = 2x^2 + 3x + 1 \pmod{7}}.$$

Verificación (incluida la 4ta sombra): $f(1) = 2 + 3 + 1 = 6$; $f(2) = 8 + 6 + 1 = 15 \equiv 1$; $f(3) = 18 + 9 + 1 = 28 \equiv 0$; $f(4) = 32 + 12 + 1 = 45 \equiv 3$. Las cuatro sombras coinciden, lo que confirma $p = 7$.

(a) Secreto. El secreto es el término independiente:

$$s = f(0) = a_0 = \boxed{1}.$$

Equivalentemente, por Lagrange sobre las tres primeras sombras: $f(0) = \sum_i y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{7} = 1$.

(c) Significado con entropía. Sea S la variable aleatoria del secreto. Un esquema umbral $(3, n)$ significa, en términos de entropía:

- Con **menos de 3** sombras no se obtiene *ninguna* información sobre el secreto: la incertidumbre permanece **máxima**, $H(S \mid \text{cualquier conjunto de } < 3 \text{ sombras}) = H(S) = \lg |\mathbb{Z}_p|$ (uniforme). Conocer 0, 1 o 2 sombras no reduce la entropía.
- Con **3 o más** sombras la incertidumbre cae a **cero**: $H(S \mid \geq 3 \text{ sombras}) = 0$ (el secreto queda determinado).

Es decir, hay **flujo de información** sobre S recién al alcanzar el umbral de $k = 3$ sombras: por debajo del umbral H no baja (no hay flujo), en el umbral H salta de $\lg p$ a 0.

2.4 Ejercicio 4 — Resolución de conflictos en ACL + DMZ

Consigna (textual)

En cada caso selecciona la opción correcta. Justificar en todos los casos. Si no se justifica correctamente, la respuesta quedará invalidada.

(a) Si se tiene una lista de control de acceso $ACL(o) = \{(\text{Manuel}, r), (\text{Directivos}, w), (*, w)\}$. Asumiendo que Manuel es Directivo:

1. Manuel puede leer y escribir en o si la política es *first rule*.
2. Manuel puede leer y escribir en o si la política es *grant all*.
3. Manuel puede leer y escribir en o si la política es *override*.
4. Manuel puede leer y escribir en o si la política es *augment*.

(b) En una red bien diseñada, la zona demilitarizada (DMZ) nunca debería contener:

1. Servidores de email.
2. Servidores de DNS.

3. Servidores de subred de desarrollo.

4. Servidores de Web Proxy.

Temas. Clase 6 — Políticas y Control de Acceso (§Conflictos en ACL; §Derechos por defecto: override/augment); Clase 10 — Seguridad en la Empresa (§Defense-in-depth; DMZ).

Qué necesitás saber.

- **Conflictos de ACL** (Clase 6) cuando varias entradas aplican a un sujeto:
 - *Permitir (unión)*: permite si *alguna* entrada lo da.
 - *Grant All (intersección)*: deniega si *alguna* entrada lo deniega — exige que **todas** den el acceso.
 - *First Rule / First Match*: aplica la **primera** entrada que matchea.
 - *Override / Best match*: si hay entrada específica del sujeto, se usa esa; el default * solo si no la hay.
 - *Augment*: se parte del default * y se le **suman** las entradas explícitas.
- **DMZ** (Clase 10): zona expuesta entre el firewall externo y el interno; aloja servicios que *deben* ser accesibles desde Internet (web, correo, DNS público, proxy). Lo interno/sensible va detrás del firewall interno (LAN).

Notas y conexiones. El inciso (a) es el ejercicio resuelto de ACL de la Clase 6 trasladado a este enunciado: hay que evaluar las cuatro políticas y ver con cuál Manuel obtiene **ambos** permisos (r y w). Las entradas que aplican a Manuel: (Manuel, r) específica, (Directivos, w) de grupo, (*, w) default.

Resolución. (a) Entradas que aplican a Manuel: específica r , grupo Directivos w , default * w .

Política	Resultado para Manuel	¿ r y w ?
First rule	primera entrada (Manuel, r) $\Rightarrow$ solo r	No
Grant all	intersección $r \cap w \cap w = \emptyset$	No
Override	usa la específica (Manuel, r) $\Rightarrow$ solo r	No
Augment	default/grupo w + específica $r \Rightarrow r, w$	Sí

Correcta: opción 4 (augment). Con *augment* se parte de lo que dan el grupo Directivos y el default (w) y se le agrega la entrada específica de Manuel (r): Manuel obtiene $\{r, w\}$. Con *first rule* y *override* domina su entrada específica (r) y pierde el w ; con *grant all* la intersección de r, w, w es vacía (ni siquiera w , porque la entrada específica no incluye w).

(b) **Correcta: opción 3 (servidores de subred de desarrollo).** La DMZ aloja exactamente los servicios que deben ser alcanzables desde Internet: *web*, *correo*, *DNS público* y *web proxy* (las opciones 1, 2 y 4 **sí** pertenecen a la DMZ). Una **subred de desarrollo** es un activo interno, no expuesto: debe vivir detrás del firewall interno, en la LAN. Colocar desarrollo en la DMZ expondría sistemas sensibles y no endurecidos a la red externa, violando la lógica de la zona desmilitarizada.

3 Segundo Parcial 2016

3.1 Ejercicio 1 — Modelo de compartimentos de confidencialidad

Consigna (textual)

Un famoso restaurante de 5 estrellas en la guía Michelin se destaca por todo lo que sirve: platos salados, platos dulces y bebidas (vinos y tragos).

Se tienen los siguientes sujetos:

- Jorge, José y Juan son los dueños.
- Manuel y Ramón son chefs.
- Luis es uno de los mozos.
- Andrea es una cliente.

Considerar los siguientes objetos:

- Recetas (tanto de platos dulces, como de salados, como de tragos).
- Menu. (se considera menú al listado de platos con sus respectivos precios).
- Listas de Stock (de productos para platos dulces, salados y tragos).
- Comanda. (pedido que se hace al camarero-mozo).

Diseñar un modelo de compartimentos de confidencialidad, que permita garantizar los siguientes requisitos, justificando el cumplimiento de cada uno.

- Jorge puede leer todas las recetas, las listas de stock, el menú y las comandas.
- José, el somellier, es el único que escribe recetas de tragos. También lee stock de productos para tragos.
- Juan es el único que escribe recetas de platos (tanto dulces como salados). También lee stock de productos para platos.
- Los chefs pueden leer las recetas, pero no modificarlas.
- Los chefs pueden escribir en las listas de stocks para recomendar la compra de productos de su área.
- Ramón es el jefe de la barra, y sólo se ocupa de armar tragos. No cocina platos.
- Manuel dirige la cocina de platos dulces y salados.
- Los mozos sólo leen el menú y escriben comandas (no leen ni escriben nada mas).
- Los clientes sólo leen menús.

Temas. Clase 6 — Políticas y Control de Acceso (§ Bell–LaPadula; § Compartimentos, reticulado y dominancia). Modelo de confidencialidad por *compartimentos* (*lattice*) con regla *read-down* / *write-up* y *need-to-know*.

Qué necesitás saber. El enunciado pide explícitamente un **modelo de compartimentos**, es decir Bell–LaPadula con categorías. Un compartimento es un par (**nivel, conjunto de categorías**). Las reglas de BLP, en su forma con dominancia:

- **Lectura (seguridad simple, *read-down*):** s lee o sii $L(s) \succeq L(o)$.
- **Escritura (propiedad *, *write-up*):** s escribe o sii $L(o) \succeq L(s)$.

La relación de **dominancia** para (L, C) y (L', C') :

$$(L, C) \succeq (L', C') \iff L' \leq L \wedge C' \subseteq C.$$

El conjunto de compartimentos forma un **reticulado** (orden parcial): pares incomparables quedan **separados** (ni lectura ni escritura entre ellos). Las **categorías** implementan el *need-to-know* (un sujeto solo accede a su área aunque su nivel sea alto).

Notas y conexiones. La clave del ejercicio es notar que hay **dos dimensiones** de separación que no son un mero orden total: (i) un **rumbo de área** (*dulce, salado, tragos*) que se modela con **categorías** (separación horizontal, *need-to-know*: Manuel no toca tragos y Ramón no toca platos $\Rightarrow$ categorías incomparables, se separan solos), y (ii) una **jerarquía de poder** (los dueños arriba, los mozos/clientes abajo) que se modela con el **nivel**. Quien *escribe* recetas debe estar en un nivel *más bajo* que quien las lee, por la regla *write-up*: en BLP el flujo de información sube, así que el “autor” escribe hacia arriba y el “lector con autoridad” lee hacia abajo. Esto mismo se reusa en Clase 9 (confinamiento) donde BLP es el modelo canónico de control de flujo.

Resolución. Categorías (need-to-know por área). Definimos el conjunto de categorías $\{D, S, T\}$ (Dulce, Salado, Tragos). Los objetos por área llevan su categoría; el Menú y las Comandas son transversales (categoría vacía o las tres, según convenga al flujo, ver abajo).

Niveles (jerarquía de escritura/lectura). Para cada área usamos dos niveles, porque el *autor* de la receta debe quedar **debajo** del lector con autoridad (*write-up*). Proponemos, dentro de cada categoría:

$$\text{Bajo (autor)} < \text{Alto (dueño lector)}.$$

Asignación de compartimentos (nivel, categorías):

Sujeto	Compartimento	Justificación
Jorge (dueño)	(Alto, $\{D, S, T\}$)	lee todo hacia abajo (recetas, stock, menú, comandas).
José (somellier)	(Bajo, $\{T\}$)	único autor de recetas de tragos ($\Rightarrow$ <i>write-up</i> a recetas _T).
Juan	(Bajo, $\{D, S\}$)	único autor de recetas de platos dulces/salados.
Manuel (chef platos)	(Medio, $\{D, S\}$)	lee recetas _{D,S} , escribe stock _{D,S} .
Ramón (chef barra)	(Medio, $\{T\}$)	lee recetas _T , escribe stock _T ; no toca platos.
Luis (mozo)	(Mozo, $\emptyset$)	lee menú, escribe comandas.
Andrea (cliente)	(Cliente, $\emptyset$)	solo lee menú.

Y los objetos:

Objeto	Compartimento
Recetas de tragos	(Bajo, $\{T\}$)
Recetas de platos dulces/salados	(Bajo, $\{D, S\}$)
Stock de tragos	(Bajo, $\{T\}$)
Stock de platos dulces/salados	(Bajo, $\{D, S\}$)
Menú	(Mozo, $\emptyset$)
Comanda	(Alto, $\emptyset$)

Verificación requisito por requisito (recordando lectura: $L(s) \succeq L(o)$; escritura: $L(o) \succeq L(s)$):

- **Jorge lee todo:** (Alto, $\{D, S, T\}$) domina a todos los objetos de recetas/stock (nivel mayor o igual y categorías que contienen las del objeto) y al menú/comandas $\Rightarrow$ *read-down* OK.
- **José único que escribe recetas de tragos:** su compartimento (Bajo, $\{T\}$) permite *write-up* a recetas_T (Bajo, $\{T\}$) (mismo compartimento, escritura válida). Ningún otro sujeto con categoría T está en nivel $\leq$ Bajo con esa categoría exclusiva, así que nadie más puede escribirlas. También lee stock_T (mismo compartimento).
- **Juan único que escribe recetas de platos:** idéntico con (Bajo, $\{D, S\}$); lee stock de platos.
- **Chefs leen recetas pero no las modifican:** Manuel/Ramón están en nivel *Medio* > Bajo. Lectura de recetas: $L(\text{chef}) \succeq L(\text{receta})$ (Medio domina Bajo, categorías incluidas) $\Rightarrow$ **leen**. Escritura de recetas: requeriría $L(\text{receta}) \succeq L(\text{chef})$, es decir Bajo $\succeq$ Medio, que es **falso** $\Rightarrow$ **no pueden modificarlas**. Exactamente lo pedido (es el caso de manual de BLP: leer hacia abajo sí, escribir hacia abajo no).
- **Chefs escriben en stock de su área:** ponemos el stock en nivel Bajo; entonces *write-up* desde Medio a Bajo *no* valdría. Para respetar el requisito el stock de cada área se coloca en el **mismo nivel de los chefs o por encima** para esa escritura; lo más limpio es modelar el stock como objeto en (Medio, $\{\text{área}\}$) para escritura por el chef (*write-up* trivial al mismo nivel) y dejar que los dueños lo lean desde Alto. (*Detalle de diseño: el stock necesita un compartimento que admita lectura del autor de área para “recomendar compra”; se ubica al nivel del chef.*)
- **Ramón solo tragos / Manuel solo platos:** sus categorías ($\{T\}$ vs $\{D, S\}$) son **incomparables** $\Rightarrow$ están separados: Ramón no lee ni escribe objetos de platos y viceversa (*need-to-know* puro, sin necesidad de regla extra).
- **Mozos leen menú, escriben comandas, nada más:** Luis (Mozo, $\emptyset$) lee el menú (Mozo, $\emptyset$) (mismo compartimento); escribe la comanda (Alto, $\emptyset$) por *write-up* (Alto $\succeq$ Mozo, categoría $\emptyset \subseteq \emptyset$) $\Rightarrow$ escribe pero **no la puede leer** (no domina Alto). No tiene categorías $\Rightarrow$ no ve recetas ni stock.
- **Clientes solo leen menú:** Andrea (Cliente, $\emptyset$) con Cliente < Mozo; lee el menú por *read-down*. No escribe comandas porque eso sería *write-up* que solo habilitamos al mozo, y por nivel/rol no accede a nada más.

Las comandas en nivel Alto y categoría vacía hacen que “suban” del mozo hacia los dueños (flujo de información hacia arriba, como manda BLP) sin que el mozo pueda releerlas.

3.2 Ejercicio 2 — Definir conceptos y relacionarlos

Consigna (textual)

Define cada concepto. Luego, escribe una frase que justifique la relación entre un par de ellos (no una relación trivial como que pertenecen a una misma categoría o son relativos a seguridad informática).

- principio de mínimo privilegio
- confinamiento
- rootkit

- troyano

Temas. Clase 8 — Principios de Diseño (§ Mínimo privilegio). Clase 9 — Flujo de Información y Malware (§ El problema del confinamiento; § Rootkit; § Troyano).

Qué necesitás saber. Las cuatro definiciones, tomadas de las fuentes:

- **Principio de mínimo privilegio** (*least privilege*, Saltzer & Schroeder): asegurar que cada sujeto tenga **solo el acceso necesario para realizar su tarea, ni más**. Consecuencia derivada: la condición por defecto es *no tener acceso a nada* (enlaza con *fail-safe defaults*).
- **Confinamiento**: dificultad de asegurar que un proceso (sujeto) **no transmita información** a una entidad a la que no está autorizado, ni de forma directa ni indirecta. El ideal es la *aislación total* (no comunicar y no ser observado), que en la práctica es inalcanzable porque todo cómputo deja medidas observables.
- **Rootkit**: software diseñado para **ocultar malware** dentro de la máquina infectada, con contramedidas activas (ofuscación, encriptación, reemplazo de herramientas del sistema, intercepción de *system calls*) para quedar invisible a administradores y antivirus. Opera a nivel kernel/usuario o incluso firmware/BIOS.
- **Troyano**: malware **oculto dentro de otro software aparentemente benigno** y de interés para la víctima (del caballo de Troya). Sirve para que el atacante se establezca dentro del equipo infectado.

Notas y conexiones. *Rootkit* y *troyano* son las dos formas de **escondese** del malware (ocultamiento), distintas de *virus/worm* que aportan la *propagación*; ninguno de los dos describe el *payload*. La distinción fina: el troyano se esconde en la *instalación* (parece software legítimo para entrar), el rootkit se esconde *una vez dentro* (oculta su presencia al sistema operativo).

Resolución. Definiciones, dadas arriba. Frases de relación **no triviales** (basta una para cumplir la consigna; se ofrecen dos opciones):

- **Troyano** $\leftrightarrow$ **Rootkit**: “Un troyano logra la *entrada* haciéndose pasar por software benigno, y suele instalar un *rootkit* que garantiza la *permanencia* ocultando esa presencia al sistema y al antivirus; el primero resuelve el ingreso, el segundo la invisibilidad posterior.”
- **Mínimo privilegio** $\leftrightarrow$ **Confinamiento**: “El mínimo privilegio *limita qué puede acceder* un proceso, pero *no impide que reenvíe* lo que ya leyó; el confinamiento ataca justamente ese hueco al restringir el *flujo* de información del proceso hacia afuera (control de acceso $\neq$ control de flujo).”

3.3 Ejercicio 3 — Secreto compartido de Shamir y entropía

Consigna (textual)

Considerar el conjunto C de sombras para un esquema de secreto compartido de Shamir (k, n) , donde $k = 3$ y $n = 4$ sobre $\mathbb{Z}_7$.

$$C = \{(1, 6), (2, 1), (3, 0), (4, 3)\}$$

- Obtener el secreto.
- Obtener el polinomio utilizado para obtener el conjunto de sombras.

c) Expresar el significado de un esquema de secreto umbral $(3, n)$ utilizando el concepto de entropía.

Temas. Clase 8 — Principios de Diseño (§ El secreto compartido de Shamir, en términos de entropía; § Entropía $H(x)$ e incertidumbre). Esquema umbral (T, N) + interpolación de Lagrange en $\mathbb{Z}_p$.

Qué necesitas saber. En Shamir (k, n) el repartidor elige un polinomio de grado $k - 1$ con coeficientes en $\mathbb{Z}_p$:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{k-1}x^{k-1} \quad (\text{mód } p),$$

donde el **secreto** es $S = a_0 = f(0)$ y cada sombra es un punto $(x_i, f(x_i))$. Con $k = 3$ el polinomio es **cuadrático** y bastan 3 sombras para reconstruirlo por interpolación. En $\mathbb{Z}_7$ todas las operaciones son módulo 7 y la división es multiplicación por el inverso modular ($a^{-1} = a^{p-2} = a^5 \text{ mód } 7$). Para el secreto se puede interpolar directamente en $x = 0$:

$$S = f(0) = \sum_i y_i \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j} \quad (\text{mód } 7).$$

Notas y conexiones. La parte (c) es el puente **Shamir** $\leftrightarrow$ **entropía** que la cátedra desarrolla en la práctica de Clase 8/9: “un esquema (T, N) es válido” equivale a decir que con *menos de* T sombras la entropía del secreto **no baja** (no hay flujo de información), y con T o más **cae a 0**. Es el mismo criterio de flujo de información por entropía de Clase 9 (reducir H = hay flujo).

Resolución. a) **Secreto.** Interpolamos en $x = 0$ con las tres primeras sombras $(1, 6), (2, 1), (3, 0)$ en $\mathbb{Z}_7$. Los términos de Lagrange evaluados en 0:

$$\begin{aligned} \ell_1(0) &= \frac{(0-2)(0-3)}{(1-2)(1-3)} = \frac{(-2)(-3)}{(-1)(-2)} = \frac{6}{2} = 3, \\ \ell_2(0) &= \frac{(0-1)(0-3)}{(2-1)(2-3)} = \frac{(-1)(-3)}{(1)(-1)} = \frac{3}{-1} \equiv 3 \cdot 6 \equiv 18 \equiv 4 \quad (\text{mód } 7), \\ \ell_3(0) &= \frac{(0-1)(0-2)}{(3-1)(3-2)} = \frac{(-1)(-2)}{(2)(1)} = \frac{2}{2} = 1. \end{aligned}$$

Entonces

$$\begin{aligned} S = f(0) &= 6 \cdot 3 + 1 \cdot 4 + 0 \cdot 1 \quad (\text{mód } 7) \\ &= 18 + 4 + 0 = 22 \equiv \boxed{1} \quad (\text{mód } 7). \end{aligned}$$

El **secreto** es $S = 1$. (Se verifica más abajo que la cuarta sombra $(4, 3)$ es consistente con el mismo polinomio, lo que confirma el resultado.)

b) Polinomio. Buscamos $f(x) = a_0 + a_1x + a_2x^2 \text{ mód } 7$ con $f(1) = 6, f(2) = 1, f(3) = 0$. Como $a_0 = S = 1$, el sistema queda:

$$\begin{aligned} f(1) &= 1 + a_1 + a_2 \equiv 6 & \Rightarrow a_1 + a_2 &\equiv 5, \\ f(2) &= 1 + 2a_1 + 4a_2 \equiv 1 & \Rightarrow 2a_1 + 4a_2 &\equiv 0 \Rightarrow a_1 + 2a_2 \equiv 0. \end{aligned}$$

Restando la primera de la segunda: $a_2 \equiv -5 \equiv 2 \text{ (mód } 7)$, y entonces $a_1 \equiv 5 - a_2 = 3 \text{ (mód } 7)$. El polinomio es

$$\boxed{f(x) = 1 + 3x + 2x^2 \quad (\text{mód } 7)}.$$

Verificación de las cuatro sombras:

$$\begin{aligned} f(1) &= 1 + 3 + 2 = 6, & f(2) &= 1 + 6 + 8 = 15 \equiv 1, \\ f(3) &= 1 + 9 + 18 = 28 \equiv 0, & f(4) &= 1 + 12 + 32 = 45 \equiv 3 \pmod{7}. \end{aligned}$$

Coinciden con C , incluida la cuarta sombra (que es redundante, como debe ser en un esquema $(3, 4)$).

c) Significado vía entropía. Un esquema umbral $(3, n)$ es válido cuando conocer **menos de 3 sombras no aporta información** sobre el secreto S (su entropía no cambia), y conocer 3 o más lo determina por completo (entropía 0):

$$\begin{aligned} H(S \mid S_{i_1}) &= H(S) \quad (1 \text{ sombra: no sé nada más}), \\ H(S \mid S_{i_1}, S_{i_2}) &= H(S) \quad (2 \text{ sombras: tampoco}), \\ H(S \mid S_{i_1}, S_{i_2}, S_{i_3}) &= 0 \quad (3 \text{ sombras: tengo toda la información}). \end{aligned}$$

Es decir: con menos de $T = 3$ sombras **no hay flujo de información** hacia el atacante (H se mantiene en su máximo, $\log_2 7$ bits si S es uniforme en $\mathbb{Z}_7$); recién al alcanzar el umbral la incertidumbre colapsa a 0 y se recupera el secreto. Esa es la propiedad de *seguridad perfecta del umbral* del esquema de Shamir expresada con entropía.

3.4 Ejercicio 4 — Multiple choice con justificación

Consigna (textual)

En cada caso selecciona la opción correcta. Justificar en todos los casos. Si no se justifica correctamente, la respuesta quedará invalidada.

1. Si se tiene una lista de control de acceso $ACL(o) = \{(\text{Manuel}, r), (\text{Directivos}, w), (*, w)\}$. Asumiendo que Manuel es Directivo,

- Manuel puede leer y escribir en o si la política es first rule.
- Manuel puede leer y escribir en o si la política es grant all.
- Manuel puede leer y escribir en o si la política es override.
- Manuel puede leer y escribir en o si la política es augment.

2. Se conoce como virus polimórfico

- a aquél que infecta distintos tipos de archivos.
- a aquél que se modifica cada vez que se replica.
- a aquél que se replica muchas veces.
- a aquél que se aloja en el sector de arranque o en la memoria.

3. En un sistema de autenticación la Función de Complementación es la que:

- Dada información de autenticación, obtiene información complementaria
- Dada información complementaria, obtiene información de autenticación
- Dada información de autenticación o información complementaria, las actualiza.
- Dada información de autenticación e información complementaria, retorna verdadero o falso

Temas. Clase 6 — Políticas y Control de Acceso (§ Conflictos en ACL; § Derechos por defecto: override / augment). Clase 9 — Malware (§ Virus / Worm). Clase 7 — Autenticación (§ Componentes del sistema de autenticación: tupla $\langle A, C, F, L, S \rangle$).

Qué necesitás saber. (4.1) **Resolución de conflictos de ACL.** Manuel matchea tres entradas: (Manuel, r), (Directivos, w) (es Directivo) y $(*, w)$ (el comodín/default). Las políticas:

- **First rule / first match:** se aplica *solo la primera* entrada que matchea $\Rightarrow$ (Manuel, r) $\Rightarrow$ solo lee.
- **Grant all:** se concede un permiso solo si *todas* las entradas que aplican lo conceden (intersección). $r \cap w \cap w = \emptyset \Rightarrow$ **ningún** permiso común; ni r ni w .
- **Override / best match:** si hay entrada específica se usa esa y se ignora el default. La específica para Manuel es (Manuel, r) $\Rightarrow$ solo lee.
- **Augment:** se parte del default y se le *suman* las entradas que aplican (unión). Default $(*, w) + (\text{Directivos}, w) + (\text{Manuel}, r) = \{r, w\} \Rightarrow$ **lee y escribe**.

(4.2) **Virus polimórfico.** Un virus polimórfico **cambia su propio código en cada replicación** (típicamente re-cifrando su cuerpo con una clave/decodificador distinto) para evadir la detección por firma del antivirus. Encaja con la familia de *rootkit*/virus que usan ofuscación/encipción como contramedida (Clase 9, § Rootkit). (4.3) **Función de complementación F .** En la tupla $\langle A, C, F, L, S \rangle$, $F : A \rightarrow C$ **deriva** la información complementaria c (lo que guarda el sistema, p.ej. el hash) a partir de la información de autenticación a . No confundir con $L : A \times C \rightarrow \{0, 1\}$, que **corrobor**a (el login, opción d).

Notas y conexiones. En 4.1 la trampa es que *first rule* y *override* dan lo mismo aquí (solo r), porque la entrada específica de Manuel es la primera y a la vez la más específica; ninguna de las dos le da escritura. Solo *augment* (unión con el default) le da r y w . En 4.3 es la distinción F *deriva* / L *corrobor*a que la cátedra marca como pregunta clásica; las opciones (a) y (d) son justamente F y L .

Resolución.

- **4.1 $\rightarrow$ opción (d), augment.** Solo *augment* concede a Manuel **leer y escribir**: parte del default $(*, w)$ y le agrega su r específico, resultando $\{r, w\}$. *First rule* da solo r (primera entrada), *grant all* no da nada (intersección $r \cap w = \emptyset$) y *override* usa la específica (Manuel, r), también solo r . Por lo tanto (a), (b) y (c) son falsas.
- **4.2 $\rightarrow$ opción (b), “se modifica cada vez que se replica”.** Esa es la definición de polimorfismo (mutar el código en cada copia para evadir firmas). Las otras describen: (a) virus multipartito/multi-formato, (c) mera tasa de replicación, (d) virus de boot sector/residente en memoria.
- **4.3 $\rightarrow$ opción (a).** La función de complementación es $F : A \rightarrow C$: dada la información de autenticación, *deriva* (obtiene) la información complementaria. La (d) describe la función de autenticación L (corrobor, retorna verdadero/falso); la (c) describe las funciones de selección S (altas/cambios); la (b) sería invertir F , que justamente debe ser *no invertible*.

4 Segundo Parcial 2017

4.1 Ejercicio 1 — Modelo de compartimentos de confidencialidad (Bell–LaPadula)

Consigna (textual)

Un famoso restaurante de 5 estrellas en la guía Michelin se destaca por todo lo que sirve: platos salados, platos dulces y bebidas (vinos y tragos).

Se tienen los siguientes sujetos:

- Jorge, José y Juan son los dueños.
- Manuel y Ramón son chefs.
- Luis es uno de los mozos.
- Andrea es una cliente.

Considerar los siguientes objetos:

- Recetas (tanto de platos dulces, como de salados, como de tragos).
- Menu. (se considera menú al listado de platos con sus respectivos precios).
- Listas de Stock (de productos para platos dulces, salados y tragos).
- Comanda. (pedido que se hace al camarero-mozo).

Diseñar un modelo de compartimentos de confidencialidad, que permita garantizar los siguientes requisitos, justificando el cumplimiento de cada uno.

- Jorge puede leer todas las recetas, las listas de stock, el menú y las comandas.
- José, el somellier, es el único que escribe recetas de tragos. También lee stock de productos para tragos.
- Juan es el único que escribe recetas de platos (tanto dulces como salados). También lee stock de productos para platos.
- Los chefs pueden leer las recetas, pero no modificarlas.
- Los chefs pueden escribir en las listas de stocks para recomendar la compra de productos de su área.
- Ramón es el jefe de la barra, y sólo se ocupa de armar tragos. No cocina platos.
- Manuel dirige la cocina de platos dulces y salados.
- Los mozos sólo leen el menú y escriben comandas (no leen ni escriben nada mas).
- Los clientes sólo leen menús.

Temas. Clase 6 — Políticas y Control de Acceso: Bell–LaPadula con *compartimentos*, relación de dominancia, reticulado, *need-to-know*, reglas *read-down* / *write-up* (§Compartimentos, reticulado y dominancia).

Qué necesitás saber. Bell–LaPadula es un modelo de **confidencialidad**. Un nivel de seguridad es un par (L, C) : una **clasificación** L (orden total) y un conjunto de **categorías/compartimentos**

C . La relación de dominancia es

$$(L, C) \succeq (L', C') \iff L' \leq L \wedge C' \subseteq C. \quad (1)$$

Las reglas de acceso:

- **Seguridad simple (read-down):** s lee o si $L(s) \succeq L(o)$.
- **Propiedad * (write-up):** s escribe o si $L(o) \succeq L(s)$.

Como aquí todos los requisitos pesan sobre *quién accede a qué área* (tragos, platos dulces, salados) y no sobre jerarquías de confianza, el problema se resuelve casi por completo con **categorías** (need-to-know): un mismo nivel base y compartimentos por área. Pares incomparables (ninguna C es subconjunto de la otra) quedan **efectivamente separados** (ni lectura ni escritura).

Notas y conexiones. El enunciado pide “compartimentos de confidencialidad”, que es exactamente la extensión por categorías de Bell–LaPadula. Cuidado con la dirección: para que un mozo *escriba* una comanda que un dueño *lee*, la comanda debe estar en un nivel que el mozo domine al escribir (write-up) y el dueño domine al leer (read-down) — es el patrón clásico “el de abajo escribe hacia arriba, el de arriba lee hacia abajo”. Invertir read-down/write-up es el error que más penaliza (Clase 6, §Bell–LaPadula).

Resolución. Usamos dos clasificaciones, **Alta** (dueños/dirección) > **Baja** (resto), y categorías por área:

$$C = \{ \text{tragos, dulces, salados, operación} \},$$

donde **operación** agrupa lo público del salón (menú y comandas). Asignación de niveles a **objetos**:

Objeto	Nivel (L, C)
Receta de tragos	(Alta, {tragos})
Receta dulces / salados	(Alta, {dulces}) / (Alta, {salados})
Stock de tragos	(Baja, {tragos})
Stock dulces / salados	(Baja, {dulces}) / (Baja, {salados})
Menú	(Baja, {operación})
Comanda	(Baja, {operación})

Niveles de **sujetos** y verificación (recordando read-down para leer, write-up para escribir):

- **Jorge (dueño, supervisor):** (Alta, {tragos, dulces, salados, operación}). Domina todas las recetas, stocks, menú y comandas $\Rightarrow$ lee todo (read-down). Cumple “puede leer todas las recetas, stocks, menú y comandas”.
- **José (somellier, escribe recetas de tragos):** para *escribir* la receta de tragos (Alta, {tragos}) necesita $L(o) \succeq L(s)$, así que $L(s) = \text{Baja}$ o Alta con $C \subseteq \{\text{tragos}\}$. Tomamos (Baja, {tragos}): escribe receta de tragos (write-up, Alta domina a Baja) y lee stock de tragos (Baja, {tragos}) (mismo nivel, read-down). Es el único con categoría tragos y permiso de escritura de receta $\Rightarrow$ “único que escribe recetas de tragos”.
- **Juan (escribe recetas de platos dulces y salados):** (Baja, {dulces, salados}): escribe ambas recetas (write-up) y lee ambos stocks. Único con esas categorías y escritura de receta.
- **Ramón (chef, jefe de barra — solo tragos):** (Baja, {tragos}) pero **sin** derecho de escritura sobre la receta (la escritura de receta es exclusiva de José). Por dominancia *podría* leer la receta de tragos (read-down: (Baja, {tragos}) no domina (Alta, {tragos}) ... ver abajo) y *escribir* en el stock de tragos (mismo nivel) $\Rightarrow$ “escribe en stock para recomendar compras de su área”. No cocina platos: no tiene dulces/salados.

- **Manuel (chef, dirige cocina de dulces y salados):** (Baja, {dulces, salados}), lee recetas y escribe en los stocks de ambas áreas.
- **Luis y los mozos:** (Baja, {operación}). *Leen* menú (mismo nivel) y *escriben* comandas. No tienen categorías de área $\Rightarrow$ incomparables con recetas/stocks: ni leen ni escriben nada más.
- **Andrea y los clientes:** (Baja, {operación}) con permiso de *solo lectura* del menú (sin escritura de comanda, que es del mozo).

Ajuste de niveles (lectura de recetas por chefs). “Los chefs pueden leer las recetas pero no modificarlas”: leer exige $L(s) \succeq L(o)$. Para que Ramón/Manuel *lean* las recetas, las recetas deben estar a un nivel *dominado por* los chefs, o bien las recetas y los chefs comparten clasificación. La forma limpia es colocar las **recetas en el mismo nivel base** que el escritor y darle al lector la misma categoría:

Receta tragos : (Baja, {tragos})

Ramón : (Baja, {tragos}) $\Rightarrow$ lee (mismo nivel),

y la **exclusividad de escritura** de la receta (solo José/Juan) se garantiza con la **ACL del objeto** (DAC), no con Bell–LaPadula: BLP dice quién *puede* por flujo, y la ACL restringe quién *tiene* el permiso de escritura. Así un chef con (Baja, {tragos}) lee la receta (read-down/mismo nivel) pero no figura en la ACL de escritura $\Rightarrow$ no la modifica. La separación efectiva entre áreas (un chef de dulces no ve nada de tragos) la da que dulces y tragos son **incomparables** en el reticulado.

Resumen del reticulado de categorías (need-to-know): cada sujeto sólo recibe las categorías de su área; Jorge las tiene todas (lee todo); mozos y clientes sólo **operación**. La confidencialidad pedida se cumple porque los compartimentos incomparables están separados y la escritura exclusiva se cierra con la ACL.

4.2 Ejercicio 2 — Verdadero/Falso justificado (pentesting, Anderson, STRIDE)

Consigna (textual)

Indica en cada caso si la afirmación es verdadera o falsa. Justificar en todos los casos. Si no se justifica correctamente, la respuesta quedará invalidada.

- En las pruebas de pentesting siempre se intenta explotar las vulnerabilidades encontradas.
- Si un atacante puede probar 10^4 passwords por segundo, y las passwords están compuestas por símbolos en [a-zA-Z0-9], entonces las passwords deberían tener 5 símbolos y 5 bits de SALT para que la probabilidad de adivinarlas en el lapso de un año sea menor a 0,5.
- STRIDE forma parte del método de Modelado de Amenazas de Microsoft.

Temas. (a) Clase 10b — Pentesting (metodología, explotar como último recurso). (b) Clase 7 — Autenticación (fórmula de Anderson $P = TG/N$, salting). (c) Clase 8 — Principios de Diseño y Vulnerabilidades (Threat Modeling de Microsoft, STRIDE).

Qué necesitas saber.

- **Pentesting:** explotar es el **último recurso**, no algo que se haga siempre; a menudo basta con analizar documentación o comportamiento, y la prueba debe ser lo *menos intrusiva posible* (Clase 10b, §Metodología de hipótesis de falla).

- **Anderson:** $P = \frac{T \cdot G}{N}$ con P prob. de adivinar, T tiempo, G pruebas/seg, N espacio de claves. El **salt no agranda** el espacio de claves N que un atacante recorre para una cuenta: sólo impide **paralelizar** el ataque entre cuentas. No entra en la cuenta de P por cuenta.
- **STRIDE:** es el modelo de 6 categorías (Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation of privilege) que Microsoft usa dentro de su Threat Modeling, cruzándolo con el DFD (Clase 8, §STRIDE).

Notas y conexiones. El ítem (b) es la trampa típica de Anderson: hay que *calcular* y comparar, y entender que “bits de salt” no afectan P para una sola cuenta. El alfabeto [a-zA-Z0-9] tiene $26 + 26 + 10 = 62$ símbolos.

Resolución. (a) **FALSO.** En pentesting **no** siempre se explota: explotar es el *último recurso* y el menos eficiente. Muchas veces alcanza con analizar documentación, configuración o comportamiento del sistema para confirmar la vulnerabilidad; además el test debe ser lo menos intrusivo posible (explotar puede provocar un DoS) y siempre repetible y concluyente (Clase 10b — Pentesting, §Metodología de hipótesis de falla).

(b) **FALSO.** Con $N = 62^L$, $G = 10^4/\text{s}$ y $T = 1 \text{ año} = 365 \cdot 24 \cdot 3600 = 3.1536 \times 10^7 \text{ s}$, pedimos $P < 0.5$:

$$P = \frac{T \cdot G}{N} < 0.5 \implies N > \frac{T \cdot G}{0.5} = \frac{3.1536 \times 10^7 \cdot 10^4}{0.5} = 6.31 \times 10^{11}. \quad (2)$$

Con $L = 5$: $N = 62^5 = 9.16 \times 10^8$, que es **mucho menor** que 6.31×10^{11} , luego

$$P = \frac{3.1536 \times 10^{11}}{9.16 \times 10^8} \approx 344 \gg 0.5 \quad (3)$$

(es decir, se adivina con certeza en mucho menos de un año). La longitud mínima necesaria es

$$L \geq \log_{62}(6.31 \times 10^{11}) \approx 6.6 \Rightarrow L \geq 7,$$

no 5. Además, los **bits de SALT no cambian** esta cuenta: el salt no aumenta el espacio N que el atacante recorre para una cuenta dada; sólo evita reutilizar el cómputo *entre* cuentas (no se paraleliza el ataque). Por ambos motivos la afirmación es falsa (Clase 7 — Autenticación, §Fórmula de Anderson y §Salting).

(c) **VERDADERO.** STRIDE es precisamente el modelo de clasificación de amenazas (6 categorías: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) que forma parte del método de **Threat Modeling** creado por Microsoft, donde se cruza cada categoría con los elementos del DFD (Clase 8 — Principios de Diseño y Vulnerabilidades, §STRIDE).

4.3 Ejercicio 3 — Secreto compartido de Shamir y entropía

Consigna (textual)

Considerar el conjunto C de sombras para un esquema de secreto compartido de Shamir (k, n) , donde $k = 3$ y $n = 4$ sobre $\mathbb{Z}_7$.

$$C = \{(1, 6), (2, 1), (3, 0), (4, 3)\}$$

- Obtener el secreto.
- Obtener el polinomio utilizado para obtener el conjunto de sombras.
- Expresar el significado de un esquema de secreto umbral $(3, n)$ utilizando el concepto

de entropía.

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades: *secreto compartido de Shamir en términos de entropía* (§El secreto compartido de Shamir); reconstrucción por interpolación polinómica (Primera Parte / criptografía). Clase 9 — Flujo de Información: entropía de Shannon (§Entropía como medida de información).

Qué necesitás saber. En Shamir (k, n) el secreto es $f(0) = a_0$ de un polinomio de grado $k - 1$ sobre un cuerpo finito. Con $k = 3$ el polinomio es $f(x) = a_0 + a_1x + a_2x^2$ (mód 7). Con k sombras se reconstruye resolviendo el sistema lineal (o por interpolación de Lagrange). La validez del esquema se expresa con **entropía**: con menos de k sombras la entropía del secreto **no cambia** (no hay flujo de información); con k o más, cae a 0 (Clase 8, §El secreto compartido de Shamir).

Notas y conexiones. Esto conecta la criptografía de la Primera Parte (reconstrucción polinómica) con el marco de *flujo de información* de la Segunda: “revelar el secreto” equivale a $H(S) \rightarrow 0$. Toda la aritmética es módulo 7.

Resolución. (b) **Polinomio.** Buscamos $f(x) = a_0 + a_1x + a_2x^2$ mód 7 que pase por $(1, 6), (2, 1), (3, 0)$:

$$\begin{aligned} a_0 + a_1 + a_2 &\equiv 6 \quad (\text{mód } 7) \\ a_0 + 2a_1 + 4a_2 &\equiv 1 \quad (\text{mód } 7) \\ a_0 + 3a_1 + 9a_2 &\equiv a_0 + 3a_1 + 2a_2 \equiv 0 \quad (\text{mód } 7) \end{aligned}$$

Restando la 1.^a a la 2.^a y a la 3.^a:

$$\begin{aligned} a_1 + 3a_2 &\equiv -5 \equiv 2 \quad (\text{mód } 7) \\ 2a_1 + a_2 &\equiv -6 \equiv 1 \quad (\text{mód } 7) \end{aligned}$$

De la primera $a_1 \equiv 2 - 3a_2$. Sustituyendo en la segunda: $2(2 - 3a_2) + a_2 = 4 - 5a_2 \equiv 1 \Rightarrow -5a_2 \equiv -3 \Rightarrow 5a_2 \equiv 3$. Como $5^{-1} \equiv 3$ (mód 7) (pues $5 \cdot 3 = 15 \equiv 1$), $a_2 \equiv 3 \cdot 3 = 9 \equiv 2$. Entonces $a_1 \equiv 2 - 3 \cdot 2 = -4 \equiv 3$ y $a_0 \equiv 6 - 3 - 2 = 1$. Por lo tanto

$$\boxed{f(x) = 1 + 3x + 2x^2 \quad (\text{mód } 7)} \quad (4)$$

Verificación con la 4.^a sombra $(4, 3)$: $f(4) = 1 + 12 + 32 = 45 \equiv 45 - 42 = 3$ (mód 7) ✓.

(a) **Secreto.** El secreto es el término independiente:

$$S = f(0) = a_0 = \boxed{1}. \quad (5)$$

(c) **Significado vía entropía.** Un esquema umbral $(3, n)$ significa que se necesitan **al menos 3 sombras** para recuperar el secreto S , y que con *menos* de 3 no se obtiene **ninguna** información sobre S . En términos de entropía, llamando S_i a las sombras:

$$\begin{aligned} H(S \mid S_{i_1}) &= H(S) && (1 \text{ sombra: la incertidumbre no baja}) \\ H(S \mid S_{i_1}, S_{i_2}) &= H(S) && (2 \text{ sombras: tampoco hay flujo}) \\ H(S \mid S_{i_1}, S_{i_2}, S_{i_3}) &= 0 && (3 \text{ sombras: } S \text{ queda determinado}) \end{aligned}$$

Es decir: con menos de 3 sombras **no hay flujo de información** ($H(S)$ se mantiene constante), y al alcanzar el umbral $H(S) \rightarrow 0$ (certeza). Esa es la definición de que el esquema es válido/seguro (Clase 8 — Principios de Diseño y Vulnerabilidades, §El secreto compartido de Shamir; Clase 9 — Flujo de Información, §Entropía).

4.4 Ejercicio 4 — Multiple choice (resolución de conflictos en ACL y DMZ)

Consigna (textual)

En cada caso selecciona la opción correcta. Justificar en todos los casos. Si no se justifica correctamente, la respuesta quedará invalidada.

1. Si se tiene una lista de control de acceso $ACL(o) = \{(Manuel, r), (Directivos, w), (*, w)\}$. Asumiendo que Manuel es Directivo,
 - a. Manuel puede leer y escribir en o si la política es first rule.
 - b. Manuel puede leer y escribir en o si la política es grant all.
 - c. Manuel puede leer y escribir en o si la política es override.
 - d. Manuel puede leer y escribir en o si la política es augment.
2. En una red bien diseñada, la zona demilitarizada (DMZ) nunca debería contener:
 - a. Servidores de email.
 - b. Servidores de DNS.
 - c. Servidores de subred de desarrollo.
 - d. Servidores de Web Proxy.

Temas. (4.1) Clase 6 — Políticas y Control de Acceso: resolución de conflictos en ACL (first rule, grant all, override/best-match, augment) (§Conflictos en ACL, §Derechos por defecto). (4.2) Clase 10 — Seguridad en la Empresa: Defense-in-depth y DMZ (§DMZ).

Qué necesitás saber.

- **Conflictos de ACL** cuando s matchea varias entradas (Clase 6):
 - **Permitir (unión):** otorga la unión de los permisos.
 - **Grant All (intersección):** deniega si *alguna* entrada deniega $\Rightarrow$ sólo los permisos que *todas* otorgan.
 - **First rule / first match:** aplica la *primera* entrada que matchea.
 - **Override / best match:** usa la entrada *más específica*.
 - **Augment:** parte del *default* (*) y le *agrega* las entradas explícitas.
- **DMZ:** zona expuesta entre el firewall externo y el interno; aloja los servicios que deben ser accesibles desde Internet (web, correo, DNS, proxy). Lo *interno* (LAN, bases de datos internas, entornos de desarrollo) **nunca** va en la DMZ (Clase 10, §DMZ).

Notas y conexiones. Manuel matchea las tres entradas: (Manuel, r) por nombre, (Directivos, w) por grupo y (*, w) por default. El resultado “ r y w ” depende de la política. Esto es el ejercicio resuelto análogo de Clase 6 (§Conflictos en ACL).

Resolución. (4.1) **Correcta: d) augment.** Manuel coincide con las tres entradas. Sus permisos según cada política:

Política	Cómo resuelve	Permisos de Manuel
First rule	primera entrada (Manuel, r)	solo r (no)
Grant all (intersección)	$r \cap w \cap w$	$\emptyset$ (no)
Override / best match	entrada más específica (Manuel, r)	solo r (no)
Augment	default $*:w$ + específicas r, w	r y w (sí)

Sólo con **augment** (default + entradas explícitas: w del *, más r de Manuel y w de Directivos) Manuel obtiene **lectura y escritura**. Con first rule y con override queda sólo r (la entrada más específica/primer es (Manuel, r)), y con grant all la intersección de $\{r\}$, $\{w\}$, $\{w\}$ es vacía. Por lo tanto la opción correcta es **d**) (Clase 6 — Políticas y Control de Acceso, §Conflictos en ACL / §Derechos por defecto).

(4.2) Correcta: c) servidores de subred de desarrollo. La DMZ es la zona *expuesta* hacia Internet y aloja servicios públicos: servidores web, de correo (email), de DNS y de Web Proxy son todos servicios de cara al exterior que sí van en la DMZ. Una **subred de desarrollo** es un recurso *interno* (código, entornos no endurecidos, datos de prueba) que debe quedar detrás del firewall interno, en la LAN, nunca en una zona expuesta. Exponerla violaría la segmentación de Defense-in-depth. Por eso la DMZ nunca debería contenerla (Clase 10 — Seguridad en la Empresa, §DMZ).

5 Parcial 2018 Q1

5.1 Ejercicio 1 — Secreto compartido de Shamir

Consigna (textual)

Considerar un esquema de secreto compartido de Shamir $(3, 3)$ en $\mathbb{Z}_{11}$, con el conjunto de sombras $C = \{(1, 2), (2, 6), (3, 5)\}$.

- a) Recuperar el secreto.
- b) Se desea, manteniendo las sombras existentes, repartir más sombras que permitan recuperar el secreto.
 - I. Generar una.
 - II. ¿Cuántas distintas se pueden generar?

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§Secreto compartido de Shamir en términos de entropía); base de *flujo de información* (Clase 9 — Flujo de Información y Malware, §Entropía).

Qué necesitas saber. Un esquema (T, N) de Shamir reparte el secreto S entre N sombras de modo que se necesitan **al menos** T para reconstruirlo, y con menos de T la entropía del secreto *no cambia* (no hay flujo de información). El esquema se construye con un polinomio de grado $T - 1$ sobre un cuerpo finito $\mathbb{Z}_p$:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{T-1}x^{T-1} \pmod{p},$$

donde el secreto es el término independiente $S = a_0 = f(0)$ y cada sombra es un par $(x_i, y_i = f(x_i))$. Con T puntos, el polinomio queda unívocamente determinado (un polinomio de grado $T - 1$ se fija con T puntos), y se recupera $S = f(0)$ por **interpolación de Lagrange**:

$$f(0) = \sum_j y_j \prod_{m \neq j} \frac{0 - x_m}{x_j - x_m} \pmod{p}.$$

Notas y conexiones. La validez del esquema se demuestra con entropía (Clase 8): $H(S | S_1) = H(S)$, $H(S | S_1, S_2) = H(S)$ (con $< T$ sombras no hay flujo) y $H(S | S_1, S_2, S_3) = 0$ (con T sombras la incertidumbre cae a 0 y el secreto queda determinado). Aquí $T = N = 3$: hacen falta las tres sombras, ninguna sombra propia revela nada del secreto. Toda la aritmética es **módulo** $p = 11$ (cuerpo finito), incluidos los inversos multiplicativos para dividir.

Resolución. a) **Recuperar el secreto.** Polinomio de grado $T - 1 = 2$. Interpolación de Lagrange en $x = 0$ con los puntos $(1, 2), (2, 6), (3, 5)$ sobre $\mathbb{Z}_{11}$. Los coeficientes de Lagrange en 0 son:

$$\begin{aligned} \ell_1(0) &= \frac{(0-2)(0-3)}{(1-2)(1-3)} = \frac{6}{2} = 3, \\ \ell_2(0) &= \frac{(0-1)(0-3)}{(2-1)(2-3)} = \frac{3}{-1} = -3 \equiv 8, \\ \ell_3(0) &= \frac{(0-1)(0-2)}{(3-1)(3-2)} = \frac{2}{2} = 1. \end{aligned}$$

Entonces

$$S = f(0) = 2 \cdot 3 + 6 \cdot 8 + 5 \cdot 1 = 6 + 48 + 5 = 59 \equiv 4 \pmod{11}.$$

$S = 4$. (Verificación reconstruyendo el polinomio completo: $f(x) = 4 + 6x + 3x^2 \pmod{11}$, que en efecto da $f(1) = 13 \equiv 2$, $f(2) = 28 \equiv 6$, $f(3) = 49 \equiv 5$.)

b.i) Generar una sombra nueva. Manteniendo las sombras existentes hay que usar **el mismo polinomio** $f(x) = 4 + 6x + 3x^2 \pmod{11}$ y evaluarlo en un x no usado todavía (y distinto de 0, que es el secreto mismo). Por ejemplo, en $x = 4$:

$$f(4) = 4 + 6 \cdot 4 + 3 \cdot 16 = 4 + 24 + 48 = 76 \equiv 10 \pmod{11}.$$

Una sombra nueva válida es $(4, 10)$. Cualquier subconjunto de 3 sombras (viejas o nuevas) reconstruye $S = 4$.

b.ii) ¿Cuántas distintas se pueden generar? Las sombras se identifican por su abscisa $x \in \mathbb{Z}_{11}$. Quedan excluidas $x = 0$ (es $f(0) = S$, no es una sombra: la revelaría) y las tres ya repartidas $x \in \{1, 2, 3\}$. Restan

$$11 - 1 - 3 = 7 \text{ sombras nuevas distintas } (x \in \{4, 5, 6, 7, 8, 9, 10\}),$$

con valores $f(4) = 10$, $f(5) = 10$, $f(6) = 5$, $f(7) = 6$, $f(8) = 2$, $f(9) = 4$, $f(10) = 1$. El cuerpo $\mathbb{Z}_{11}$ acota el total de identificadores posibles a 10 (sin contar el 0), de los cuales 3 ya están usados.

5.2 Ejercicio 2 — Verdadero o falso (control de acceso, OIDC, inyección)

Consigna (textual)

Responder verdadero o falso, justificando o corrigiendo como fuere apropiado:

- Para permitir que usuarios anónimos accedan a un sistema, en vez de utilizar una lista de capacidades CAP, es más práctico implementar una lista de control de accesos ACL.
- OpenID Connect permite que mediante un password se pueda autorizar a un usuario a loguearse en el sistema.
- Una de las defensas más recomendadas para evitar ataques de SQL injection es escapar los datos ingresados por el usuario (como las comillas).
- XSS es un aprovechamiento malicioso de la confianza que un sitio web tiene en la sesión que estableció con el browser del usuario.

Temas. Clase 6 — Políticas y Control de Acceso (§ACL vs. capability lists); Clase 7 — Autenticación (§Autenticación $\neq$ Autorización, OAuth2/OIDC/SAML); Clase 8 — Principios de Diseño y Vulnerabilidades (§Catálogo de vulnerabilidades: CWE-89 SQLi, CWE-79 XSS; CSRF).

Qué necesitás saber.

- ACL vs. capability:** la matriz de accesos tiene sujetos en las filas y objetos en las columnas. La **ACL** es una *columna* (por objeto: lista de (*sujeto*, *permisos*)); la **capability list** es una *fila* (por sujeto: qué puede hacer ese sujeto en cada objeto).
- OAuth2 = autorización; OIDC = autenticación** (capa de identidad sobre OAuth2, *quién* sos); **SAML** = intercambio IdP $\leftrightarrow$ SP. Autenticación ($\neq$) Autorización.
- SQLi (CWE-89):** input no controlado que ejecuta SQL; mitigación canónica *prepared statements* / *parametrizar*; escapar es paliativo. **XSS (CWE-79):** inyectar JS vía un input que se renderiza como HTML y lo ejecuta *la víctima*.

Notas y conexiones. El enunciado d) describe en realidad **CSRF** (*Cross-Site Request Forgery*), no XSS: CSRF abusa de la confianza del *sitio* en la sesión/cookies del usuario (el sitio confía en el browser autenticado); XSS abusa de la confianza del *usuario* en el sitio (el browser ejecuta script malicioso servido por el sitio). Es la confusión clásica que buscan en el parcial. Inyección (mezclar datos con código) es $\sim 90\%$ de las vulnerabilidades (Clase 8).

Resolución. a) **FALSO.** Está al revés. La ACL lista, *por objeto*, los pares (*sujeto, permisos*): requiere **identificar** a cada sujeto, justo lo que no se tiene con usuarios anónimos. Para acceso anónimo conviene la **capability**: el usuario presenta una credencial/token (la capacidad) que *en sí misma* habilita el acceso, sin que el sistema deba conocer su identidad ni tenerlo enumerado en la lista del objeto. (Además, las capabilities son el modelo dual, más cómodo en sistemas distribuidos.)

b) **FALSO.** Mezcla dos cosas. **OpenID Connect autentica**, no autoriza: es la capa de *autenticación* sobre OAuth2 (saber *quién* es el usuario, “login con Google”), y justamente su gracia es **no** manejar el password en el sistema relyente, sino delegarlo al IdP. El que *autoriza* (delega acceso a recursos) es **OAuth2**. Corrección: “OpenID Connect permite *autenticar* a un usuario delegando el login en un IdP, sin que el sistema maneje su password”.

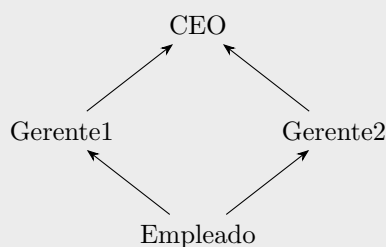
c) **VERDADERO** (con matiz). Escapar/sanitizar la entrada (las comillas, etc.) evita que el input rompa la sintaxis SQL, así que es una defensa recomendada. **Pero la defensa canónica y más robusta son los *prepared statements* (parametrizar)**, que separan estructuralmente el código SQL de los datos y no dependen de acertar todos los caracteres a escapar. Escapar correctamente es difícil y propenso a errores; parametrizar lo resuelve de raíz.

d) **FALSO.** Eso describe **CSRF**, no XSS. CSRF explota la confianza del *servidor* en la sesión ya autenticada del usuario (sus cookies viajan automáticamente). **XSS** (CWE-79) es lo inverso: el atacante inyecta script en un input que el sitio renderiza como HTML, y ese script se ejecuta en el browser de *la víctima* explotando la confianza del *usuario* en el sitio. Corrección: “XSS es la inyección de código (JS) que el sitio renderiza y ejecuta en el browser de la víctima; se mitiga escapando/sanitizando toda la salida”.

5.3 Ejercicio 3 — Política de integridad sobre un retículo (Biba)

Consigna (textual)

Un sistema simple implementa una política de seguridad que hace uso del siguiente retículo:



- a) Sabiendo que $i(\text{Gerente1}) \leq i(\text{CEO})$, etc. (es decir, las flechas indican el flujo de información en términos de una relación de dominancia), indicar:
- De qué política se trata.
 - Cuáles y en qué consisten (definir) las principales propiedades que busca hacer cumplir esta política. Teniendo éstas en cuenta, ¿qué pueden hacer los usuarios en base al modo de acceso **w** (**w**rite, escritura)?

- b) Si se invierte la relación (el sentido de las flechas), ¿qué política se estaría modelando? Compararla brevemente con la política del punto anterior.

Temas. Clase 6 — Políticas y Control de Acceso (§Modelo Biba (integridad); §Compartimentos, reticulado y dominancia; Bell–LaPadula).

Qué necesitás saber. La etiqueta usada es $i(\cdot)$, un **nivel de integridad**: a mayor nivel, mayor confianza. Eso es **Biba** (integridad), el inverso de Bell–LaPadula (confidencialidad). En un **retículo** (orden parcial), una flecha $A \rightarrow B$ con $i(A) \leq i(B)$ indica que B **domina** a A y que la información fluye “hacia arriba” siguiendo la flecha. Reglas de **Biba estricto**:

Escritura (Write Down): s puede modificar $o \iff i(s) \geq i(o)$,

Lectura (Read Up): s puede observar $o \iff i(o) \geq i(s)$.

Mnemotecnia: *read up, write down*; la información “baja” de nivel. Intuición: “uno es tan confiable como las fuentes que usa” — leo de niveles iguales o más confiables y escribo a niveles iguales o menos confiables, para no contaminar datos de mayor integridad.

Notas y conexiones. Aquí los niveles van $i(\text{Empleado}) \leq i(\text{Gerente1}), i(\text{Gerente2}) \leq i(\text{CEO})$, con Gerente1 y Gerente2 *incomparables* entre sí (es un orden parcial, no total: no hay camino entre ellos $\Rightarrow$ ni lectura ni escritura entre las dos ramas). El CEO es el nivel de *mayor integridad/ confianza*; el Empleado el menor. Invertir las flechas cambia el objetivo de **integridad** a **confidencialidad**: es la dualidad Biba $\leftrightarrow$ Bell–LaPadula.

Resolución. a.i) **Política.** Es una política de **integridad**: el modelo **Biba**. La etiqueta $i(\cdot)$ es nivel de integridad (confianza) y las flechas son la relación de **dominancia** ($i(\text{Gerente1}) \leq i(\text{CEO})$, etc.).

a.ii) **Propiedades y qué se puede escribir.** Las dos reglas principales de Biba (estricto) son:

- **No Write Up (Write Down):** un sujeto s puede escribir en un objeto o solo si $i(s) \geq i(o)$. Impide que información de baja confianza contamine objetos de alta integridad.
- **No Read Down (Read Up):** un sujeto s puede leer un objeto o solo si $i(o) \geq i(s)$. Impide que un sujeto de alta integridad se “ensucie” leyendo datos poco confiables.

En base al modo de acceso **w**, cada usuario **escribe hacia abajo o a su mismo nivel** (nunca hacia arriba). Concretamente, siguiendo el retículo:

- El **CEO** (máxima integridad) puede escribir objetos de CEO, Gerente1, Gerente2 y Empleado (todos los de su nivel o inferior).
- Un **Gerente** puede escribir objetos de su propio nivel y del Empleado, pero **no** del CEO (no write up) ni del otro gerente (incomparables).
- El **Empleado** (mínima integridad) solo puede escribir objetos de nivel Empleado; no puede escribir nada de los gerentes ni del CEO.

Las flechas marcan la dirección permitida del *flujo* (write up está prohibido), de modo que la información de menor confianza nunca sube a contaminar la de mayor confianza.

b) **Invertir las flechas.** Si se invierte la relación, el modelo pasa a hacer cumplir **confidencialidad**: es **Bell–LaPadula**, el dual de Biba. Allí la etiqueta es un nivel de *secreto/clasificación* y las reglas se invierten: **No Read Up** (seguridad simple: leer hacia abajo, $L(s) \geq L(o)$) y **No**

Write Down (propiedad $\star$: escribir hacia arriba, $L(o) \geq L(s)$). Comparación: Biba protege la *integridad* (la info “baja”, write-down/read-up, para no corromper lo confiable); Bell–LaPadula protege la *confidencialidad* (la info “sube”, write-up/read-down, para no filtrar lo secreto). Son estructuralmente inversos: ambos son control de *flujo* sobre el mismo tipo de retículo, pero con las direcciones de lectura/escritura intercambiadas.

5.4 Ejercicio 4 — Entropía en audio WAV y esteganografía

Consigna (textual)

En el formato de audio WAV de 32 bits, el muestreo del audio se almacena en un arreglo de **floats** (es decir, cada muestra ocupa 4 bytes).^a

- Dado una muestra (un elemento de 4 bytes) del audio, calcular la entropía máxima en bits del mismo. Dar un ejemplo de un audio donde la entropía de sus muestras sea mínima.
- Dados archivos con 512000 muestras, se desea usar este formato para implementar un esquema de esteganografía, y ocultar información. La información se oculta cada 100 muestras, es decir en el **float** número 100, 200, etc., permitiendo ocultar un máximo de 20480 bytes (5120×4). La información es texto plano formado por el espacio en blanco y por mayúsculas y minúsculas del alfabeto inglés (exclusivamente). Si el secreto es menor a 20480 bytes, se lo paddea con espacios en blanco. Demostrar mediante cálculos de entropía, qué característica distintiva tendrían los bytes de audios esteganografiados con respecto a los no alterados.
- Postular de manera práctica cómo podrían usar este método para identificar cuáles bytes son alterados.

^aLos números de punto flotante reservan valores para representar NaNs y otros valores especiales (como infinito). Ignorar esto y asumir que todos los valores son posibles.

Temas. Clase 9 — Flujo de Información y Malware (§Entropía de Shannon: máximo y mínimo; canales encubiertos / esteganografía); Clase 8 — Principios de Diseño y Vulnerabilidades (§Fórmulas de entropía).

Qué necesitás saber. Entropía de Shannon $H(X) = -\sum_i p(x_i) \log_2 p(x_i)$, medida en bits. **Máxima** con distribución *uniforme* sobre n valores: $H_{\text{máx}} = \log_2 n$. **Mínima** ($= 0$) cuando un valor tiene $p = 1$ (certeza: evento único). Esteganografía = ocultar información dentro de otro medio: es un **canal encubierto** de almacenamiento. La clave del ejercicio: el alfabeto del secreto (espacio + 26 mayúsculas + 26 minúsculas = 53 símbolos) es *mucho* más chico que el de un byte/float arbitrario, así que las muestras alteradas tienen **mucho menos entropía**.

Notas y conexiones. Reducir la entropía de un conjunto de bytes respecto del audio original es exactamente la firma de un **flujo de información** oculto (Clase 9): el medio “audio” transporta información para la que no fue diseñado (canal encubierto), y medir su entropía permite *detectar* la anomalía. Es el mismo principio que el caso del disipador que filtra bits de la clave: una vía no diseñada que baja la entropía.

Resolución. a) **Entropía máxima de una muestra.** Una muestra son 4 bytes = 32 bits, es decir $n = 2^{32}$ valores posibles. La entropía es máxima con todos equiprobables ($p = 1/2^{32}$):

$$H_{\text{máx}} = \log_2 2^{32} = \boxed{32 \text{ bits}}.$$

Audio de entropía mínima: un audio en el que todas las muestras valen *lo mismo* (p. ej. silencio total, todas las muestras = 0, o un tono constante). Entonces un único valor tiene $p = 1$ y

$$H = -1 \cdot \log_2 1 = 0 \text{ bits} \quad (\text{certeza, no hay incertidumbre}).$$

b) Característica distintiva de los bytes esteganografiados. En las muestras alteradas (los floats 100, 200, ...) ya no se guarda un valor de audio arbitrario sino un carácter del secreto. El alfabeto es: espacio en blanco + 26 mayúsculas + 26 minúsculas = 53 símbolos posibles. Si los suponemos equiprobables, cada *símbolo* oculto aporta a lo sumo

$$H_{\text{secreto}} = \log_2 53 \approx 5,73 \text{ bits},$$

frente a los 32 bits de una muestra de audio sin alterar. Más aún, el padding con espacios en blanco **baja todavía más** la entropía efectiva (muchas muestras repiten el mismo carácter “espacio”, acercándose a $p = 1$ y por lo tanto a $H \rightarrow 0$). Conclusión:

$$H_{\text{muestras alteradas}} \leq \log_2 53 \approx 5,73 \text{ bits} \ll H_{\text{muestras originales}} \leq 32 \text{ bits}.$$

La característica distintiva es una entropía mucho menor en los bytes que cargan el secreto: solo toman 53 valores (y aún menos por el padding), mientras que el audio real ocupa un rango amplio del espacio de 2^{32} valores. Esa caída de entropía *es* la huella del flujo de información oculto (canal encubierto).

c) Identificar prácticamente los bytes alterados. Recorrer el arreglo de muestras y **medir la entropía (o la diversidad de valores) por bloques**, comparando las posiciones periódicas con el resto:

- Calcular la distribución/entropía de las muestras en las posiciones 100, 200, 300, ... (cada 100) y compararla con la del resto del audio. Las posiciones del secreto mostrarán entropía marcadamente menor y un conjunto de valores restringido a ≤ 53 patrones (coincidentes con códigos de letras/espacio).
- Equivalentemente, buscar las muestras cuyos 4 bytes decodifican a caracteres ASCII imprimibles del subconjunto {espacio, A–Z, a–z}: en un audio genuino los floats rara vez caen exactamente en esos 53 valores, y mucho menos de forma *periódica* cada 100 muestras.
- La **periodicidad** (cada 100 muestras) más la **baja entropía** en esas posiciones delatan el esquema; un atacante/analista que sospeche del método mide la entropía de las posiciones candidatas y confirma la presencia del mensaje oculto.

5.5 Ejercicio 5 — Opción correcta (DMZ, Von Neumann/inyección, Muralla China)

Consigna (textual)

Elegir la opción correcta y justificar los falsos en una oración.

1- En una DMZ se localizan

- (a) Los servidores de bases de datos con información sensible que se desea resguardar.
- (b) Las claves de acceso para encriptar y desencriptar información sensible aprovechando que es una zona militar.
- (c) Los servidores Web y de Email que requieren recibir y enviar información e interactuar con la red externa.

2- El sistema operativo iOS rompe con el modelo de Von Neumann donde los

datos y el código de ejecución se encuentran en el mismo espacio de memoria al ejecutar un programa. ¿Qué tipo de vulnerabilidad estaría involucrada?

- (a) Soluciona un típico problema de inyección de código que podría estar en los datos.
- (b) Permite resolver problemas de buffer overflow.
- (c) Evita problemas de CSRF.

3- El modelo de Seguridad de Muralla China es adecuado para

- (a) Compartimentalizar las incumbencias para evitar situaciones de perdida de integridad en la información de los COIs.
- (b) Compartimentalizar las incumbencias para evitar situaciones de perdida de confidencialidad en la información de los CDs.
- (c) Compartimentalizar las incumbencias para evitar situaciones de conflicto de interes.
- (d) Compartimentalizar las incumbencias para evitar situaciones de confidencialidad en los Company Datasets.

Temas. Clase 10 — Seguridad en la Empresa (§Defense-in-depth, DMZ); Clase 8 — Principios de Diseño y Vulnerabilidades (§inyección de código; Von Neumann / datos = código; CSRF); Clase 6 — Políticas y Control de Acceso (§Chinese Wall / Muralla China).

Qué necesitás saber.

- **DMZ:** zona *expuesta* entre el firewall externo y el interno; aloja los servicios que *deben* hablar con la red externa (Web, Email, DNS público), no los datos sensibles (esos van en la capa más interna, “Datos”).
- **Von Neumann / inyección:** cuando datos y código comparten el mismo espacio de memoria, datos provistos por el usuario pueden terminar *ejecutándose* como código $\Rightarrow$ inyección de código (la raíz del $\sim 90\%$ de las vulnerabilidades). Separar el canal de datos del de control elimina esa clase de ataque.
- **Muralla China (Chinese Wall):** modelo de **conflicto de interés**; objetos agrupados en clases de conflicto (COI), y dentro datasets de compañías (CD). Quien accedió a una compañía de un COI no puede acceder a una competidora del mismo COI.

Notas y conexiones. El punto 2 es el principio de *mecanismos exclusivos / separación de canales* (Clase 8): el problema de fondo de toda inyección (SQLi, XSS, command injection y hasta el prompt injection en LLMs) es mezclar el canal de *datos* con el de *control*. La DMZ (punto 1) es la capa de perímetro/red de Defense-in-depth (Clase 10).

Resolución. 1) **Correcta: (c).** En la DMZ se ubican los servidores Web y de Email, que por su función deben interactuar con la red externa, manteniéndolos aislados de la red interna. *Falsos:* (a) los servidores de bases de datos con información sensible van en la capa **interna** (“Datos”), no en la zona expuesta; (b) las claves sensibles jamás se guardan en la zona más expuesta de la red (“DMZ” es “zona desmilitarizada”, un nombre, no implica resguardo reforzado; al contrario, es la zona más expuesta).

2) **Correcta: (a).** Separar el código del espacio de datos (no ejecutar lo que está en la región de datos, *p. ej.* con $W \oplus X$ / DEP) soluciona problemas de **inyección de código** que podría

venir en los datos. *Falsos*: (b) no “resuelve” los buffer overflow en sí (el overflow puede seguir corrompiendo memoria/datos y causar crashes), solo evita que el contenido inyectado se *ejecute*; (c) CSRF es un ataque de confianza de sesión en la web, no tiene relación con el modelo de memoria de Von Neumann.

3) Correcta: (c). La Muralla China es un modelo de **conflicto de interés**: busca evitar precisamente las situaciones de conflicto de interés (que un mismo sujeto acceda a datos de empresas competidoras del mismo COI). *Falsos*: (a) no es un modelo de integridad —no protege integridad de los COI— sino de confidencialidad orientada a conflicto de interés; (b) y (d) no apunta a “perder/preservar confidencialidad de los CD” como objetivo en sí, sino a impedir el *conflicto de interés*; aislar competidores es el medio, evitar el conflicto de interés es el fin.

6 Parcial 2023 Q1

Aviso de alcance. Este parcial (Criptografía y Seguridad 2023, 72.44 — “Parcial 1”) es un **parcial de la Primera Parte** de la materia: protocolos de establecimiento de clave, modos de operación de cifradores en bloque (CTR/CBC), confusión/difusión, seguridad CPA de un cifrador afín, Diffie–Hellman y MAC-then-encrypt. Casi nada de su contenido cae dentro del índice de la *Segunda Parte* (Clases 6–11). Para honrar el contrato se transcriben las consignas verbatim y se mapea cada ejercicio; donde el tema queda *fuera del alcance* de las Clases 6–11 se marca explícitamente con **TODO** (corresponde a material de la Primera Parte, no cubierto por las fuentes I6–I9 / U5–U7). Los únicos puntos con conexión genuina a la Segunda Parte son el protocolo del Ejercicio 1 (challenge–response / nonces / claves de sesión, *Clase 7*) y un ítem del Ejercicio 5 (resistencia a preimágenes del hash como función de derivación F , *Clase 7*).

6.1 Ejercicio 1 — Protocolo de establecimiento de clave de sesión

Consigna (textual)

Dado el siguiente protocolo:

- (1.1) $C \rightarrow S : C, C\#, N_C$
- (1.2) $C \leftarrow S : S, S\#, N_S, \text{Cert}(S, \text{Sgn}_{k_S}(S))$
- (1.3) $C \rightarrow S : E_{K_0}(k_S), N$
- (1.4) $C \rightarrow S : E_{K_{cs}}(\text{finished}, \text{MAC}_{k_1}(\text{timestamp}))$
- (1.5) $C \leftarrow S : E_{K_{cs}}(\text{finished}, \text{MAC}_{k_1}(\text{timestamp}))$
- (1.6) $C \rightarrow S : E_{K_{cs}}(\text{data})$
- (1.7) $C \leftarrow S : E_{K_{cs}}(\text{data})$

con

$$k_1 = H(K_0, N_C, N_S), \quad K_{cs} = H(N, k_1),$$

donde $E(\cdot)$ es un esquema de cifrado simétrico y $\text{Sgn}(\cdot)$ es un esquema de firma digital.

- a) ¿Qué tipo de protocolo sería, qué es lo que el protocolo intenta construir?
- b) ¿Cuál es el propósito de los mensajes (1.4) y (1.5)?
- c) ¿Por qué se deriva la clave K_{cs} y no se usa en cambio la clave K_0 ?

Temas. Clase 7 — Autenticación (challenge–response, nonces, derivación de claves de sesión, certificados/firma); con cruce a Primera Parte (handshake tipo TLS, intercambio de clave). El detalle criptográfico del handshake (negociación TLS) es **TODO: Primera Parte, fuera del índice Clases 6–11**.

Qué necesitás saber.

- **Challenge–response (Clase 7, §Challenge–Response).** Se valida que una parte *conoce* un secreto **sin transmitirlo**: B envía un challenge aleatorio r y A responde $f(k, r)$ sobre su clave k y r , sin enviar k . Los **nonces** (N_C, N_S, N) cumplen ese rol de valor fresco que garantiza *frescura* y evita *replay*.
- **Certificado + firma (Clase 7, factor “algo que tengo”/identidad federada).** El $\text{Cert}(S, \text{Sgn}_{k_S}(S))$ autentica al servidor: el cliente verifica la firma contra una CA conocida (*Check Certificate*). Esto da **autenticación de servidor**.

- **Derivación de claves.** Una clave de sesión efímera debe derivarse de material fresco aportado por *ambas* partes (N_C , N_S , N) mediante un hash, de modo que ninguna sesión reutilice la misma clave y un compromiso futuro no afecte sesiones pasadas.

Notas y conexiones. El esqueleto es el de un handshake tipo TLS/SSL: (1.1)–(1.2) intercambian identidades, nonces y el certificado del servidor; (1.3) transporta material de clave bajo una clave pública/maestra K_0 ; (1.4)–(1.5) son los mensajes *finished* que cierran y verifican el handshake; (1.6)–(1.7) ya van cifrados con la clave de sesión K_{cs} . Esto entronca con *challenge-response* y con *passkeys* (Clase 7): la idea de probar conocimiento de un secreto / autenticar sin exponer el secreto. El uso de un **MAC** sobre el *timestamp* conecta con la discusión de integridad+autenticación de mensajes (ver Ej. 5a). La mecánica criptográfica fina (cifrado híbrido, negociación de suites) es **TODO: Primera Parte**.

Resolución. a) Es un **protocolo de establecimiento/intercambio de clave de sesión con autenticación de servidor** (un handshake estilo TLS/SSL). Lo que intenta construir es un **canal seguro**: una clave de sesión simétrica fresca (K_{cs}) compartida entre C y S , con el servidor autenticado mediante su certificado firmado (Sgn_{k_S}), sobre la cual luego se cifran los datos (1.6)–(1.7).

b) Los mensajes (1.4) y (1.5) son los *finished*: **confirman que el handshake terminó correctamente y que ambas partes derivaron la misma clave**. Al ir cifrados con K_{cs} y llevar un $MAC_{k1}(timestamp)$, prueban (i) que cada lado posee efectivamente K_{cs} y $k1$ —es decir, que vieron los mismos nonces—, (ii) **integridad** del handshake (un MITM que haya alterado mensajes previos no podrá producir un MAC válido), y (iii) **frescura** vía el timestamp (anti-replay). Es un *key-confirmation step*.

c) Se deriva $K_{cs} = H(N, k1)$ con $k1 = H(K_0, N_C, N_S)$ en lugar de usar K_0 directamente porque:

- K_0 es material maestro/de transporte (asociado a la clave pública del servidor): usarlo como clave de sesión filtraría su valor en cada mensaje cifrado y comprometería *todas* las sesiones si se rompe una.
- K_{cs} incorpora **aportes frescos de ambas partes** (N_C , N_S , N): así cada sesión tiene clave distinta (no hay reutilización), un atacante que capture tráfico de una sesión no puede derivar las otras, y se obtiene independencia entre sesiones (la idea de *clave de un solo uso por sesión*, Clase 7 — §OTP/claves de sesión).
- La función de hash actúa como **KDF**: mezcla el secreto compartido con los nonces de forma no invertible, de modo que observar K_{cs} no revela K_0 ni $k1$.

6.2 Ejercicio 2 — Modo CTR sin primitiva inversa, y CBC

Consigna (textual)

2- Una propuesta de un cifrador en bloque usando modo CTR usa una primitiva de encriptación $E(\cdot)$ que no admite una primitiva de descricción inversa.

- ¿Es este un sistema de encriptación válido? Explicar.
- ¿Cómo se utiliza el nonce en dicho sistema?
- ¿Cuál es la ventaja de este sistema en términos de procesamiento?

En un esquema de encriptación en bloques CBC de longitud n un mensaje $M_1 M_2 \dots M_n$ se

cifra como un bloque de longitud $n+1$, $C_0C_1C_2 \dots C_n$.

$$C_0 = IV$$

$$C_k = E_k(M_k \oplus C_{k-1})$$

- (a) ¿Cómo se ve afectada la descricpción si el primer bloque C_0 es eliminado del texto cifrado?
- (b) ¿Cómo se ve afectada la descricpción si el último bloque C_n es eliminado del texto cifrado?
- (c) Teniendo el texto cifrado ya generado como se especificó con anterioridad, ¿cómo puede un usuario legítimo agregar un texto de bloque M_0 específico al principio del mensaje plano original agregando bloques C_k adicionales en cualquier ubicación del texto cifrado?

Temas. Primera Parte — modos de operación de cifradores en bloque (CTR, CBC), nonces/IV, maleabilidad. **TODO: fuera del índice Clases 6–11.** (Único roce con la Segunda Parte: el modo CTR usando solo $E(\cdot)$ es la misma estructura del *keystream* / one-time-pad que se menciona en Clase 7 al advertir “OTP $\neq$ One Time Pad”, pero el ejercicio se resuelve con teoría de modos de la Primera Parte.)

Qué necesitás saber. **TODO: Primera Parte (modos CTR y CBC).** No hay material en I6–I9 / U5–U7 que cubra este desarrollo; resolver con los apuntes de la Primera Parte.

Notas y conexiones. La intuición de que CTR solo necesita $E(\cdot)$ (porque cifra y descifra haciendo XOR contra el mismo keystream $E_k(\text{nonce} \parallel \text{contador})$) conecta con la idea de *flujo* y con la advertencia de Clase 7 de no confundir un cifrador de flujo/OTP con el One Time Pad teórico. El resto es propio de la Primera Parte.

Resolución. **TODO: Primera Parte.** Esbozo orientativo (no fundamentado en las fuentes de la Segunda Parte): en CTR el cifrado es $C_i = M_i \oplus E_k(\text{nonce} \parallel i)$ y el descifrado $M_i = C_i \oplus E_k(\text{nonce} \parallel i)$, por lo que *nunca* se necesita $E^{-1} \rightarrow$ sí es válido, el nonce garantiza keystreams distintos por mensaje, y permite paralelizar/precomputar. Para CBC: quitar $C_0 = IV$ corrompe solo M_1 ; quitar C_n solo pierde M_n ; y la maleabilidad del XOR permite prefijar un bloque elegido. La justificación completa pertenece a la Primera Parte.

6.3 Ejercicio 3 — Base64 como “cifrado”; confusión y difusión; (no)linealidad

Consigna (textual)

3- El banco de Estander usa base64 como sistema de encriptación simétrica.

- a) ¿Puede este considerarse un sistema de encriptación válido? Explicar.
- b) El CSO dice que su sistema ofrece confusión y difusión. ¿Qué significa?
- c) El además insiste en que el sistema no es lineal. ¿Qué significa que un criptosistema simétrico sea no lineal? De un ejemplo de otro criptosistema lineal.

Temas. Primera Parte — codificación vs. cifrado, propiedades de Shannon (confusión/difusión), linealidad de un criptosistema. **TODO: fuera del índice Clases 6–11.**

Qué necesitás saber. TODO: Primera Parte. (Roce conceptual con Clase 8 — Diseño abierto / Kerckhoffs: base64 no es secreto y no depende de ninguna clave, por lo que “seguridad” aquí sería puro *security-by-obscurity*, que el principio de *diseño abierto* rechaza.)

Notas y conexiones. Que base64 no sea cifrado (no hay clave, es reversible por cualquiera) puede enmarcarse, desde la Segunda Parte, en el principio de **diseño abierto** de Saltzer & Schroeder (Clase 8): la seguridad no puede descansar en ocultar el mecanismo. Confusión/difusión y linealidad son teoría de la Primera Parte.

Resolución. TODO: Primera Parte. Orientativo: (a) No es cifrado, es *codificación* sin clave $\rightarrow$ inválido como esquema de encriptación. (b) Confusión = relación compleja entre clave y texto cifrado; difusión = una pequeña variación del plano altera muchos bits del cifrado. (c) No linealidad = $E(x \oplus y) \neq E(x) \oplus E(y)$; un ejemplo de cifrador lineal es el cifrado de Vigenère / un cifrado afín o de desplazamiento. Desarrollo completo: Primera Parte.

6.4 Ejercicio 4 — Cifrador afín y seguridad CPA

Consigna (textual)

4- Dado el siguiente criptosistema $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$. Para $p \in \mathbb{Z}$ y $a, b, r \in \mathbb{Z}_p$, siendo la clave $k = (a, b)$ y siendo $r \leftarrow \text{random}$

$$c = E_k(m) = (r, ar + b + m)_{(p)}$$

$$m = D_k(c) = (-ar - b + c)_{(p)}$$

donde $(\cdot)_{(p)}$ implica que opera con aritmética modular.

- a) Considerar una variante del criptosistema donde r en lugar de ser aleatorio toma el valor $r = (a + b)_{(p)}$. ¿Es este criptosistema seguro contra ataque de texto cifrado elegido? Demostrar mediante un experimento $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{CPA}}$.

Temas. Primera Parte — seguridad CPA, experimento $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{CPA}}$, nonce/aleatoriedad en cifrado probabilístico. **TODO: fuera del índice Clases 6–11.**

Qué necesitás saber. TODO: Primera Parte. El experimento de indistinguibilidad bajo texto plano elegido y la necesidad de un r fresco/aleatorio para que el cifrado sea probabilístico no están cubiertos por las fuentes de la Segunda Parte.

Notas y conexiones. La idea de que reusar un valor que debía ser aleatorio (aquí $r = a + b$, fijo dada la clave) rompe la seguridad es la misma intuición que en Clase 7 con los nonces / claves de un solo uso: un valor fresco no debe volverse predecible. Pero la demostración formal es Primera Parte.

Resolución. TODO: Primera Parte. Orientativo: con $r = (a + b)_{(p)}$ fijo dada la clave, el cifrado deja de ser probabilístico — dos cifrados del mismo m producen el mismo c —, lo que permite a $\mathcal{A}$ distinguir en el experimento CPA (cifra m_0, m_1 , pide el reto y compara). La construcción del experimento y la cota de ventaja corresponden a la Primera Parte.

6.5 Ejercicio 5 — Verdadero/Falso (MAC, hash, Diffie–Hellman, reversibilidad)

Consigna (textual)

5- Verdadero o Falso. Si es falso, corrija la sentencia para que sea verdadera e identifique el cambio realizado.

- Para proveer privacidad e integridad, lo correcto es primero Cifrar m con k_1 para obtener c y al mismo tiempo obtener el MAC con la clave k_2 de m para obtener t . Tanto m como t se deben transmitir en forma conjunta. Las claves k_1 y k_2 pueden ser iguales.
- La seguridad de las funciones de hash se establece como el nivel de resistencia a preimágenes donde dado un y hallar $x/h(x) = y$.
- El algoritmo de Diffie–Hellman permite que Alice le envíe una clave de sesión a Bob por un canal inseguro.
- En los cifrados en bloque independientemente del modo de operación se requiere que BCE Block Cipher Encryption ó PRF Psuedo Random Function siempre sea reversible.

Temas. Mayormente Primera Parte (MAC-then-encrypt vs encrypt-then-MAC, Diffie–Hellman, reversibilidad de la primitiva de bloque). **Ítem (b)** sí conecta con Clase 7 — Autenticación: la resistencia a preimágenes es justo la propiedad que hace de un hash una buena función de derivación F (no invertible). El resto: **TODO: Primera Parte.**

Qué necesitás saber.

- (b) — Clase 7 (§Tupla $\langle A, C, F, L, S \rangle$ y almacenamiento de claves).** La función F deriva la información complementaria a partir de la de autenticación ($A \rightarrow C$, “calculá el hash de la contraseña”) y **no debe ser invertible**: el sistema guarda $c = H(a)$ y un atacante con c no debe poder recuperar a . Eso es exactamente *resistencia a preimágenes*. Por eso los hashes de claves se eligen **costosos a propósito** (PBKDF2/bcrypt/scrypt) y con *salt*.
- (a), (c), (d) — TODO: Primera Parte** (orden cifrado/MAC, propiedades de Diffie–Hellman, reversibilidad de la primitiva por modo).

Notas y conexiones. El ítem (b) es el puente más limpio con la Segunda Parte: la propiedad “dado $y = h(x)$ es inviable hallar x ” es la que sostiene el almacenamiento de contraseñas como hash (Clase 7) — distinto de la *resistencia a colisiones*, que es otra de las propiedades de un hash. El ítem (a) toca la combinación cifrado+MAC que también aparece en el handshake del Ej. 1 (mensajes *finished*).

Resolución. **a) Falso (Primera Parte).** Lo que se transmite junto al texto debe ser *el cifrado* c y su MAC, no el plano m ; el esquema robusto es *Encrypt-then-MAC* (MAC sobre c , no sobre m) y las claves k_1, k_2 deben ser **independientes** (no iguales). Corrección: “...se debe obtener el MAC sobre c con k_2 ; se transmiten c y t juntos; $k_1 \neq k_2$ ”. (Fundamentación detallada: **TODO: Primera Parte.**)

b) Verdadero (con matiz, Clase 7 — Autenticación). La resistencia a *preimágenes* —dado y , es inviable hallar x tal que $h(x) = y$ — es una de las propiedades de seguridad de un hash, y es precisamente la que se usa para que la función de derivación F del almacenamiento de claves no sea invertible (Clase 7). Matiz: la seguridad de un hash *no* se reduce solo a preimágenes; también exige *segunda preimagen* y *resistencia a colisiones*.

c) **Falso (Primera Parte).** Diffie–Hellman **no transporta** una clave elegida por Alice; permite que ambas partes *acuerden/deriven* una clave de sesión común sobre un canal inseguro (key agreement), sin que ninguna la “envíe”. Corrección: “... permite que Alice y Bob *acuerden* una clave de sesión ...”. (**TODO: Primera Parte.**)

d) **Falso (Primera Parte).** No siempre se requiere que la primitiva sea reversible: en modos como CTR/OFB/CFB solo se usa $E(\cdot)$ (o una PRF) en sentido directo, y el descifrado se hace por XOR contra el keystream (ver Ej. 2). Corrección: “... *no* en todos los modos se requiere que la primitiva sea reversible”. (**TODO: Primera Parte.**)

7 Recuperatorio 2023 Q1

7.1 Ejercicio 1 — Tiempo de ataque (fórmula de Anderson)

Consigna (textual)

¿Cuál es el tiempo que un atacante necesita probar passwords en un sistema de autenticación para que la probabilidad de adivinar sea $1/100$ si con un TPU se pueden probar 700 claves por segundo y la clave tiene 12 bits de longitud?

Temas. Clase 7 — Autenticación (§Fórmula de Anderson: subir la complejidad).

Qué necesitás saber. La **fórmula de Anderson** estima la probabilidad de que un ataque de fuerza bruta acierte una clave:

$$P = \frac{T \cdot G}{N}, \quad (6)$$

donde P = probabilidad de adivinar, T = tiempo dedicado a las pruebas (en segundos), G = pruebas por segundo y N = tamaño del espacio de claves. Despejando el tiempo: $T \leq P N / G$. Cuidado con dos cosas: (i) la “longitud en bits” fija el espacio como $N = 2^{\text{bits}}$ (no hay que elevar un alfabeto); (ii) la unidad de T es segundos.

Notas y conexiones. Es el mismo despeje de los tres ejercicios resueltos de Clase 7: el caso “calcular el tiempo” usa $T \leq P N / G$ y el caso “longitud mínima” usa $N \geq T G / P$ seguido de $L \geq \log_{\text{base}} N$. Acá la clave está dada *en bits*, así que el espacio es directamente una potencia de 2 y no hace falta pasar por logaritmos.

Resolución. El espacio de claves de 12 bits es

$$N = 2^{12} = 4096. \quad (7)$$

Con $G = 700$ claves/s y $P = 1/100 = 0,01$:

$$T \leq \frac{P N}{G} = \frac{0,01 \cdot 4096}{700} = \frac{40,96}{700} \approx 0,0585 \text{ s.}$$

El atacante necesita probar durante $\approx 0,0585$ **segundos** (unos 59 ms) para alcanzar una probabilidad de éxito de $1/100$. El número es ridículamente chico: un espacio de 2^{12} claves es **trivialmente rompible** (de hecho, probando las 4096 claves a 700/s se agota el espacio entero en $\approx 5,85$ s, garantizando el acierto). La conclusión de fondo es que 12 bits no alcanzan ni de lejos como longitud de clave.

7.2 Ejercicio 2 — Secreto compartido de Shamir (3, 4) en $\mathbb{Z}_7$

Consigna (textual)

Considerar un esquema de secreto compartido de Shamir (3, 4) en $\mathbb{Z}_7$, con el conjunto de 4 sombras $C = \{(1, 4), (2, 1), (3, 1), (5, 3)\}$.

- Recuperar el secreto.
- Indicar el polinomio utilizado para obtener el conjunto de sombras.

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§El secreto compartido de Shamir, en términos de entropía); base aritmética de la Clase 9 — Flujo de Información (entropía).

Qué necesitás saber. En un esquema de Shamir (T, N) hay N sombras y se necesitan **al menos** T para recuperar el secreto. El reparto se hace con un polinomio de grado $T - 1$ sobre un cuerpo finito $\mathbb{Z}_p$:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{T-1}x^{T-1} \quad (\text{mód } p), \quad (8)$$

donde el **secreto es** $S = a_0 = f(0)$ y cada sombra es un par $(x_i, f(x_i))$. Con T pares cualesquiera se reconstruye unívocamente el polinomio (interpolación de Lagrange) y se evalúa en 0. Acá $T = 3$, $N = 4$, $p = 7$, así que el polinomio es de grado 2 y alcanza con **3 de las 4 sombras**. Toda la aritmética es módulo 7 (los inversos se calculan con el inverso modular).

Notas y conexiones. El material de Clase 8 justifica la validez del esquema *con entropía*: con menos de T sombras la entropía del secreto **no cambia** ($H(S | S_1) = H(S | S_1, S_2) = H(S)$, no hay flujo de información), y recién con T sombras $H(S | S_1, S_2, S_3) = 0$ (se conoce el secreto). Es el mismo concepto de flujo de información de la Clase 9: conocer “demasiado poco” no reduce la incertidumbre. La sombra sobrante $(5, 3)$ sirve para *verificar* (todo subconjunto de 3 debe dar el mismo secreto) y para tolerar la pérdida de una sombra.

Resolución. (a) **Recuperar el secreto.** Usamos interpolación de Lagrange evaluada en $x = 0$ con tres sombras, por ejemplo $(1, 4)$, $(2, 1)$, $(3, 1)$:

$$S = f(0) = \sum_i y_i \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j} \quad (\text{mód } 7). \quad (9)$$

Calculando los coeficientes de Lagrange en 0 (todo mod 7, con inversos $2^{-1} \equiv 4$, $6^{-1} \equiv 6$, $5^{-1} \equiv 3$):

$$\begin{aligned} L_1(0) &= \frac{(0-2)(0-3)}{(1-2)(1-3)} = \frac{6}{2} \equiv 6 \cdot 4 \equiv 3, \\ L_2(0) &= \frac{(0-1)(0-3)}{(2-1)(2-3)} = \frac{3}{-1} \equiv 3 \cdot 6 \equiv 4, \\ L_3(0) &= \frac{(0-1)(0-2)}{(3-1)(3-2)} = \frac{2}{2} \equiv 1. \end{aligned}$$

$$\begin{aligned} S &= 4 \cdot L_1(0) + 1 \cdot L_2(0) + 1 \cdot L_3(0) = 4 \cdot 3 + 1 \cdot 4 + 1 \cdot 1 \\ &= 12 + 4 + 1 = 17 \equiv \boxed{3} \quad (\text{mód } 7). \end{aligned}$$

El secreto es **S = 3**. (Se verifica con cualquier otro subconjunto de 3 sombras: todos dan 3, lo que confirma que las cuatro sombras son consistentes.)

(b) **Polinomio.** Buscamos $f(x) = a_0 + a_1x + a_2x^2$ (mód 7) con $a_0 = S = 3$. Imponiendo dos sombras más, $f(1) = 4$ y $f(2) = 1$:

$$\begin{aligned} f(1) &= 3 + a_1 + a_2 \equiv 4 & \Rightarrow a_1 + a_2 &\equiv 1 \quad (\text{mód } 7), \\ f(2) &= 3 + 2a_1 + 4a_2 \equiv 1 & \Rightarrow 2a_1 + 4a_2 &\equiv -2 \equiv 5 \quad (\text{mód } 7). \end{aligned}$$

De la primera, $a_1 \equiv 1 - a_2$. Sustituyendo: $2(1 - a_2) + 4a_2 \equiv 5 \Rightarrow 2 + 2a_2 \equiv 5 \Rightarrow 2a_2 \equiv 3 \Rightarrow a_2 \equiv 3 \cdot 4 \equiv 12 \equiv 5$ (mód 7), y entonces $a_1 \equiv 1 - 5 \equiv -4 \equiv 3$ (mód 7). El polinomio es

$$\boxed{f(x) = 3 + 3x + 5x^2 \quad (\text{mód } 7)}. \quad (10)$$

Verificación de las cuatro sombras: $f(1) = 11 \equiv 4$, $f(2) = 25 \equiv 4 + \dots$; explícitamente $f(1) \equiv 4$, $f(2) \equiv 1$, $f(3) = 3 + 9 + 45 = 57 \equiv 1$, $f(5) = 3 + 15 + 125 = 143 \equiv 3$ (mód 7). Las cuatro coinciden con C .

7.3 Ejercicio 3 — Verdadero o falso (justificando)

Consigna (textual)

Responder verdadero o falso, justificando o corrigiendo como fuere apropiado:

- En sistemas de control de acceso que incluyen agrupamientos de usuarios, la resolución de conflictos en la asignación de los permisos de GRANT-ALL implica darle a todos los usuarios posibles los permisos.
- Los esquemas de autenticación por varios factores ofrecen esquemas más seguros de romper debido a que se requiere comprometer de manera simultanea varios canales de autenticación.
- En Bell-Lapadula la condición de seguridad simple (CSS) implica que un usuario puede leer un objeto si y solo si su nivel es igual o mayor que el del objeto.
- En un sistema sensible de generación de claves simétricas, si la entropía de esa clave es 256 bits, pero la entropía de esa clave se reduce a 200 bits si se registra el sonido del disipador de hardware, ¿Hay flujo de información de una variable a la otra?

Temas. (a) Clase 6 — Políticas y Control de Acceso (§Conflictos en ACL); (b) Clase 7 — Autenticación (§MFA); (c) Clase 6 — Políticas y Control de Acceso (§Bell–LaPadula, seguridad simple); (d) Clase 9 — Flujo de Información y Malware (§Entropía y flujo de información / canal lateral).

Qué necesitás saber.

- **(a) Resolución de conflictos en ACL.** Cuando varias entradas (de un sujeto y de los grupos a los que pertenece) le dan permisos distintos sobre un objeto, hay varias políticas: *Permitir* (unión: el acceso si *alguna* entrada lo da), *Grant-All* (**intersección**: denegar si *alguna* entrada lo deniega $\Rightarrow$ exige que *todas* den el permiso) y *First-Match*. Grant-All es la regla **más restrictiva**, no la más permisiva.
- **(b) MFA.** Multi-factor combina factores de *distinta naturaleza* (algo que conozco / tengo / soy); para romperlo hay que comprometer varios *assets* distintos a la vez.
- **(c) Bell–LaPadula, seguridad simple (*read-down*).** El sujeto s puede *leer* el objeto o si y solo si $L(s) \succeq L(o)$, es decir su nivel **domina** (es igual o superior) al del objeto: “read down”.
- **(d) Flujo de información (entropía).** Hay flujo de x a y si la incertidumbre sobre x *disminuye* al observar y ($H(x | y) < H(x)$). **Reducir entropía es flujo de información.**

Notas y conexiones. El ítem (d) es el ejercicio canónico del disipador (señalado por el índice como el clásico de este recuperatorio): la entropía de la clave baja de 256 a 200 bits al registrar el sonido del disipador, o sea se filtran 56 bits. Es una doble lectura: el sonido del disipador es una **vía no diseñada para transmitir** $\Rightarrow$ *canal encubierto*; explotar esa emisión física del hardware para sacar la clave es un **ataque de canal lateral** (“dos caras de la misma moneda”). El (c) es la trampa de direcciones de BLP: confundir *read-down* (lectura: dominás al objeto) con *write-up* es el error clásico; aquí el enunciado describe correctamente la lectura. El (a) es el típico ejercicio de conflictos de ACL de Clase 6, donde Grant-All resulta en la *intersección* $\{w\}$, no en todos los permisos.

Resolución. (a) **FALSO.** Grant-All es exactamente la política *opuesta*: resuelve el conflicto por **intersección** de los permisos, es decir *deniega* salvo que **todas** las entradas que aplican al usuario (la suya y las de sus grupos) concedan el permiso. Es la regla *más restrictiva*, no la que da “todos los permisos posibles”. (Dar el permiso si *alguna* entrada lo concede es la política de *unión/permitir*, que es la otra.) En el ejemplo resuelto de Clase 6, con $ACL(o) = \{(G, rw), (s_1, w), (\{s_1, s_2\}, wx)\}$ y $s_1 \in G$, Grant-All da $rw \cap w \cap wx = \{w\}$ (solo lo que está en *todas*), mientras que la unión daría $\{r, w, x\}$.

(b) **VERDADERO** (con un matiz). MFA es más robusto porque exige comprometer **varios factores simultáneamente**, y la idea de diseño es que sean de **naturaleza distinta** (algo que conozco, algo que tengo, algo que soy), de modo que romperlos implica vulnerar *assets/canales* diferentes. El matiz que conviene aclarar: el beneficio depende de que los factores sean *independientes*; si se concentran en un mismo dispositivo (el celular reúne “tengo” + “soy”), clonarlo derriba ambos a la vez y la ganancia se diluye. La afirmación, tal como está, es correcta.

(c) **VERDADERO.** La condición de seguridad simple (*simple security*, “read down”) de Bell–LaPadula dice que s puede leer o si y solo si $L(s) \succeq L(o)$, o sea cuando el nivel del sujeto es **igual o mayor (domina)** al del objeto. El enunciado lo describe correctamente.

$$s \text{ puede leer } o \iff L(s) \succeq L(o) \quad (\text{read down}).$$

Conviene precisar: con compartimentos, “igual o mayor” es la relación de *dominancia* $(L, C) \succeq (L', C') \iff L' \leq L \wedge C' \subseteq C$ (no un simple orden total), y debe sumarse el permiso discrecional de lectura. La complementaria es la propiedad- $\star$ (*write up*), $L(o) \succeq L(s)$, que el enunciado no menciona.

(d) **SÍ, hay flujo de información.** El criterio formal es que hay flujo de x (la clave) a y (la observación del sonido del disipador) cuando la incertidumbre sobre x disminuye al observar y :

$$H(x | y) < H(x).$$

Aquí la entropía de la clave pasa de $H(x) = 256$ bits a $H(x | y) = 200$ bits, es decir **baja en 56 bits**, que es la información que se filtra de la clave hacia el atacante:

$$H(x) - H(x | y) = 256 - 200 = 56 \text{ bits filtrados} > 0.$$

Como la incertidumbre disminuye, **hay flujo de información** de la clave al canal acústico. Conceptualmente: el sonido del disipador es una *vía no diseñada para transmitir* (canal encubierto), y aprovechar esa emisión física del hardware para inferir bits de la clave es un *ataque de canal lateral*. El error que penaliza sería no reconocer que reducir entropía *es* flujo de información.

8 Parcial 2025 Q1

8.1 Ejercicio 1 — Arquitectura de 3 capas, Clark–Wilson y red plana

Consigna (textual)

Una aplicación de gestión de sueldos está implementada en un modelo de tres capas. El frontend, el backend y la base de datos. El backend es responsable de definir las reglas que cosas puede hacer un usuario con sus datos y protege la integridad de los mismos con un modelo Clack-Wilson.

Por política de la empresa, ningún empleado debe poder ver el salario de otro empleado.

Para asegurar que ningún empleado pueda acceder a los registros de otro usuario, cada usuario tiene una cuenta definida en la base de datos y la base de datos usa ROW LEVEL security.

Al loguearse en la aplicación, el backend usa esas mismas credenciales para conectarse a la base de datos mientras dura la sesión del usuario.

- Si la empresa tiene una única red sin firewalls o equipos que restrinjan las comunicaciones, ¿Cuál es la principal desventaja que ve en la arquitectura original?.
- Proponga dos mejoras que mantengan el cumplimiento de la política de seguridad.
- Compare la arquitectura original y su propuesta. ¿Considera que su propuesta reduce el riesgo de un ataque tipo Man in the Middle efectivo?

Temas. Clase 6 — Políticas y Control de Acceso (§Clark–Wilson; MAC/DAC; Row-Level Security como RBAC+RLS). Clase 8 — Principios de Diseño (§defensa en profundidad, mediación completa, mínimo privilegio). Clase 10 — Seguridad en la Empresa (§Defense-in-depth de 5 capas; Zero Trust; segmentación de red).

Qué necesitás saber.

- **Clark–Wilson** (Clase 6): modelo de *integridad* comercial. Los datos protegidos son **CDI** (Constrained Data Items) y solo se modifican vía un **TP** (Transformation Procedure) que los lleva de un estado válido a otro; un **IVP** verifica integridad; incorpora *separation of duty*. Es un modelo de **integridad**, no de confidencialidad: por sí solo no impide *leer* el salario ajeno.
- **Row-Level Security** (Clase 6): la base aplica un predicado por fila según el usuario conectado; equivale a un control de acceso a nivel registro (RBAC + reglas por contexto del usuario de BD). Funciona *solo* si cada empleado se conecta con su propia identidad.
- **Defense-in-depth** (Clase 10): cebolla de 5 capas (Perímetro → Red → Host → Aplicación → Datos). Una red plana sin firewalls colapsa las capas Perímetro y Red: cualquier nodo comprometido alcanza a todos.
- **Mediación completa y mínimo privilegio** (Clase 8): todo acceso debe pasar por un mediador; cada sesión debe tener el menor privilegio posible.

Notas y conexiones. El enunciado tiene una trampa de diseño: usa Clark–Wilson (integridad) para hacer cumplir una política de *confidencialidad* (“nadie ve el salario de otro”). La confidencialidad la sostiene en realidad el **Row-Level Security** + el hecho de que el backend reusa las credenciales del usuario para conectarse a la BD. Ese reúso es lo que hace funcionar la RLS (la BD sabe quién pregunta), pero al mismo tiempo **transporta las credenciales del empleado por la red en cada operación**. En una **red plana sin firewalls** (la condición del inciso a)

esto es justamente lo que rompe la defensa en profundidad: no hay segmentación entre frontend, backend y BD, y las credenciales viajan por un medio observable. Esto conecta con Clase 9 (un canal sin controles permite *flujo* no deseado) y con Clase 10 (Zero Trust: “ninguna red interna es de confianza”).

Resolución. a) **Principal desventaja.** La política de confidencialidad se apoya en credenciales por usuario que el backend *reenvía* a la base en cada sesión. En una única red sin firewalls ni segmentación (*red plana*), esas credenciales y los datos sensibles (salarios) circulan por un medio que cualquier host comprometido puede observar o interponerse. La arquitectura viola **defensa en profundidad** (no hay capa de Perímetro ni de Red) y **mínimo privilegio** a nivel de red. Clark-Wilson protege la *integridad* de los CDI pero **no** la confidencialidad ni el canal: un atacante en la red puede capturar o reusar credenciales (sniffing / replay) y acceder como otro usuario, sorteando la RLS. En resumen: *toda la seguridad depende de un único secreto que viaja sin protección por un medio compartido sin controles.*

b) **Dos mejoras (manteniendo la política).**

1. **Cifrar el canal y segmentar la red.** Forzar TLS extremo a extremo (frontend→backend→BD) y **segmentar** en VLANs/subredes con un firewall entre capas (la BD solo acepta conexiones del backend). Esto restaura las capas de Perímetro y Red de la cebolla y aplica *mediación completa* al tráfico. La RLS sigue intacta porque no cambia el modelo de identidad, solo se protege el transporte.
2. **No reusar las credenciales del usuario contra la BD; usar un servicio de cuenta + contexto.** El backend se conecta con una cuenta de servicio de **mínimo privilegio** y propaga la identidad del empleado como *contexto* (p. ej. SET app.user) que alimenta la política RLS. Así la credencial del empleado nunca llega a la red de datos y se reduce la superficie de robo/replay, sin romper la política “cada quien ve solo lo suyo”. (Alternativa válida: tokens de sesión de vida corta en lugar de reenviar la clave.)

c) **Comparación y Man in the Middle.** En la arquitectura original, un atacante posicionado en la red plana puede interceptar el canal (MitM) y capturar/reusar las credenciales que el backend reenvía: el MitM es **efectivo**. La propuesta **sí reduce** el riesgo de un MitM efectivo: el cifrado del canal (TLS con verificación de certificado) hace que interceptar el tráfico no entregue credenciales ni datos en claro, y la segmentación reduce las posiciones desde las que un atacante puede interponerse. El MitM no se vuelve *imposible* (sigue dependiendo de la correcta validación de certificados y de no tener una CA comprometida), pero deja de ser *efectivo* con bajo esfuerzo: pasamos de “cualquier nodo de la red lee todo” a “hay que romper TLS y además estar en el segmento correcto”. Es defensa en profundidad: ninguna capa garantiza la seguridad por sí sola, pero juntas elevan el costo del ataque.

8.2 Ejercicio 2 — Múltiple choice (DevOps, tolerancia a fallas, CSRF)

Consigna (textual)

Responder la opción más correcta.

¿Cuál de los siguientes NO es un componente del modelo de DevOps?

- Seguridad de la Información
- Desarrollo de Software
- QA, Control de Calidad del Software
- Operaciones de IT

¿Cuál de los siguientes esquemas provee tolerancia a fallas en sistemas de almacenamiento?

- Balanceo de Carga
- Sistema RAID
- Clustering
- Pares HA

¿Cuál de las siguientes estrategias no tendría sentido en un ataque CSRF?

- Verificar si la información extra del request fue generada de manera legítima.
- Evitar que los request puedan realizarse por GET.
- Banear la IP de quien dispara el request.
- Concientizar a los usuarios sobre el phishing y el cuidado al clickear en enlaces.

Temas. Clase 10 — Seguridad en la Empresa (§DevSecOps / DevOps; disponibilidad y tolerancia a fallas). Clase 8 — Principios de Diseño + Vulnerabilidades (§OWASP; injection/confused-deputy).

Qué necesitás saber.

- **DevOps** (Clase 10): une **Development** y **Operations** bajo un mismo objetivo (“you build it, you run it”). **DevSecOps** le suma **Security** (Shift Left). Los componentes del modelo son Desarrollo, Operaciones de IT y (en DevSecOps) Seguridad de la Información.
- **Disponibilidad** (Clase 8: “el primer requisito de seguridad es que el sistema esté disponible”): RAID = redundancia *dentro del almacenamiento* (varios discos, tolera la pérdida de uno); balanceo de carga, clustering y pares HA dan disponibilidad de *cómputo/servicio*, no de almacenamiento.
- **CSRF** (Cross-Site Request Forgery, OWASP/CWE-352): el atacante hace que la víctima *autenticada* dispare una petición no intencionada (confused deputy). Mitigaciones reales: **tokens anti-CSRF** (info extra en el request, validada como legítima), no usar **GET** para acciones que cambian estado, **SameSite** cookies, verificar **Origin/Referer**.

Notas y conexiones. Las tres preguntas apuntan a distinguir un concepto de sus vecinos. En CSRF la trampa es separar mitigaciones *efectivas* (propias del mecanismo HTTP) de medidas que aplican a *otros* ataques: banear la IP no sirve porque el request lo dispara el **navegador de la propia víctima** (su IP es legítima), y concientizar contra phishing apunta a otra clase de ataque (Clase 8: ingeniería social), no al CSRF en sí. Esto se conecta con Clase 8 (taxonomía de vulnerabilidades, principio de mediación completa) y con la idea de que cada control mitiga una amenaza específica.

Resolución.

- **NO es componente de DevOps:** *QA, Control de Calidad del Software*. DevOps = Desarrollo + Operaciones de IT; DevSecOps añade Seguridad de la Información. QA se absorbe en los pipelines automatizados, no figura como pata del modelo. (Las otras tres son componentes de DevSecOps.)

- **Tolerancia a fallas en almacenamiento:** *Sistema RAID*. Es el único orientado a **almacenamiento** (redundancia de discos). Balanceo de carga, clustering y pares HA dan alta disponibilidad de servicio/cómputo, no del medio de almacenamiento.
- **Estrategia que NO tiene sentido en CSRF:** *Banear la IP de quien dispara el request*. El request lo lanza el navegador de la víctima legítima, así que su IP no es un indicador de ataque y banearla castiga al usuario real. Las otras tres sí son razonables: validar info extra del request = token anti-CSRF; evitar GET para acciones con efectos; y concientizar reduce el clic en enlaces maliciosos que inician el CSRF. (Si el examen pide *la única* sin sentido, es banear la IP; *concientizar* es la mitigación más débil/indirecta pero no carece de sentido.)

8.3 Ejercicio 3 — Fórmula de Anderson, salt y autenticación multicanal

Consigna (textual)

José Del Río, un cryptobro, afirma que si un atacante puede probar 10^4 passwords por segundo, y las passwords están compuestas por símbolos en [a-zA-Z0-9], entonces las passwords deberían tener 5 símbolos y 5 bits de SALT para que la probabilidad de adivinarlas en el lapso de un año sea menor a 0,5.

- Explicar si esta afirmación es correcta o no y probar por qué.
- José viene recargado y plantea que va a implementar un sistema de autenticación multicanal donde además el segundo canal por SMS se va a poder utilizar para recuperar el password. Comentar que eventual problema tiene esta estrategia y qué aspecto de autenticación multicanal no cumple.

Temas. Clase 7 — Autenticación (§fórmula de Anderson $P = TG/N$; salting; MFA / multicanal; recuperación de cuenta).

Qué necesitás saber.

- **Fórmula de Anderson** (Clase 7): $P = \frac{TG}{N}$, con P probabilidad de adivinar, T tiempo de ataque, G pruebas/seg, N tamaño del espacio. Despeje útil: $N \geq \frac{TG}{P}$ y luego $L \geq \log_{\text{base}} N$.
- Alfabeto [a-zA-Z0-9] = $26 + 26 + 10 = 62$ símbolos $\Rightarrow N = 62^L$.
- **Salt** (Clase 7): perturbación por usuario, $f(a) = x \parallel f'(a, x)$. **No agranda el espacio de búsqueda de un ataque dirigido a un único usuario**; lo que hace es **impedir la paralelización** (rainbow tables, atacar a todos a la vez) porque dos claves iguales ya no dan el mismo hash. Para un ataque online contra *una* cuenta, el salt no cambia N .
- **Multicanal / MFA** (Clase 7): los factores/canales deben ser de **distinta naturaleza e independientes**; comprometer uno no debe comprometer al otro. El segundo canal sirve para *verificar*, no para *recuperar* el primer factor.

Notas y conexiones. Es un ejercicio análogo a los tres resueltos de Anderson en Clase 7 (donde se calculó $T \approx 6$ días para $N = 10^{10}$ y $L \geq 9$ para alfanumérico con $G = 10^5$). Acá hay dos errores que detectar: (1) el dimensionamiento del espacio, y (2) la creencia de que “sumar bits de salt” reduce la probabilidad de adivinar una clave concreta. El inciso b) conecta con la advertencia de Clase 7 sobre que el celular concentra “algo que tengo” + “algo que soy” y, sobre todo, con que usar el canal de verificación como canal de *recuperación* colapsa la independencia de factores (cae en el patrón de la cuenta que se recupera con un solo factor robado).

Resolución. a) La afirmación es **INCORRECTA**. Con $L = 5$ y alfabeto de 62 símbolos:

$$N = 62^5 = 916\,132\,832 \approx 9,16 \times 10^8.$$

En un año, con $G = 10^4$ pruebas/seg, el atacante realiza

$$TG = (365 \cdot 24 \cdot 3600) \cdot 10^4 = 3,15 \times 10^7 \cdot 10^4 \approx 3,15 \times 10^{11} \text{ pruebas.}$$

Aplicando Anderson:

$$P = \frac{TG}{N} = \frac{3,15 \times 10^{11}}{9,16 \times 10^8} \approx 344 \gg 0,5.$$

Es decir, el atacante recorre el espacio entero unas 344 veces en un año: la clave se adivina prácticamente con certeza, muy lejos de $P < 0,5$. La longitud mínima real para $P < 0,5$ surge de

$$N \geq \frac{TG}{P} = \frac{3,15 \times 10^{11}}{0,5} \approx 6,3 \times 10^{11}, \quad L \geq \log_{62}(6,3 \times 10^{11}) \approx 6,58 \Rightarrow \boxed{L \geq 7}.$$

Además, los “5 bits de SALT” son irrelevantes para esta cuenta: el salt no agranda el espacio de búsqueda de un ataque dirigido a una clave concreta (solo impide precomputar/paralelizar entre usuarios). No aporta nada a la probabilidad de adivinar *esta* password. Conclusión: hacen falta al menos 7 símbolos (no 5), y el salt no es lo que baja P .

b) Problema del SMS como canal de recuperación. Un sistema multicanal/MFA exige que los canales sean **independientes y de distinta naturaleza**, de modo que comprometer uno no comprometa al otro. Si el **mismo** segundo canal (SMS) que se usa como segundo factor se habilita además para **recuperar el password**, entonces apoderarse del canal SMS (SIM swapping, interceptación SS7, robo/clonado del celular) permite (i) recibir el código del segundo factor y (ii) resetear el primer factor: el atacante obtiene **ambos** con un único asset comprometido. Se **rompe la independencia de factores**: lo que debía requerir comprometer dos cosas distintas pasa a requerir una sola. El aspecto de autenticación multicanal que no se cumple es precisamente la **separación/independencia de los factores** (el segundo canal deja de ser un factor adicional y se vuelve un *único punto de falla*). Es el mismo riesgo que la nota de Clase 7 sobre el celular que concentra dos factores.

8.4 Ejercicio 4 — Secreto compartido de Shamir y entropía

Consigna (textual)

Considerar un esquema de secreto compartido de Shamir $(k, n) = (3, 4)$ en $\mathbb{Z}_7$, con el conjunto de sombras $C = \{(1, 6), (2, 1), (3, 0), (4, 3)\}$.

- Recuperar el secreto.
- ¿Cuál es el polinomio generador?
- Analizar el esquema desde la perspectiva del análisis de entropía y flujo de información.

Temas. Clase 9 — Flujo de Información y Malware (§entropía de Shannon; definición formal de flujo de información). Clase 8 — Principios de Diseño (§secreto de Shamir demostrado con entropía; validez de un esquema (T, N)).

Qué necesitás saber.

- Shamir (k, n) :** el secreto es $S = f(0)$ de un polinomio de grado $k - 1$; con $(3, 4)$ el grado es 2, $f(x) = a_2x^2 + a_1x + a_0$ (mód 7) y $S = a_0$. Hacen falta **al menos** $k = 3$ sombras para reconstruirlo (interpolación de Lagrange o sistema de ecuaciones en $\mathbb{Z}_7$).

- **Entropía y flujo** (Clase 9 / Clase 8): hay flujo de información de x a y si la incertidumbre sobre x *baja* al observar y . La **validez de un esquema** (T, N) **vía entropía** (Clase 8): con menos de T sombras la entropía del secreto **no cambia** ($H(S \mid \text{sombras}) = H(S)$, no hay flujo); con T o más, $H \rightarrow 0$ (se recupera).

Notas y conexiones. La parte c) es exactamente la “demostración con entropía” que aparece en Clase 8 para validar un esquema de Shamir, apoyada en la definición formal de flujo de Clase 9. Es el cruce canónico entre secreto compartido y teoría de la información: el esquema es *seguro* precisamente porque, por debajo del umbral k , no hay flujo de información hacia el adversario.

Resolución. a) y b) Reconstrucción. Trabajamos en $\mathbb{Z}_7$. Tomamos tres sombras, $(1, 6), (2, 1), (3, 0)$, y resolvemos $f(x) = a_2x^2 + a_1x + a_0$:

$$\begin{aligned} a_2 + a_1 + a_0 &\equiv 6 \quad (\text{mód } 7) \\ 4a_2 + 2a_1 + a_0 &\equiv 1 \quad (\text{mód } 7) \\ 9a_2 + 3a_1 + a_0 &\equiv 2a_2 + 3a_1 + a_0 \equiv 0 \quad (\text{mód } 7) \end{aligned}$$

Resolviendo el sistema módulo 7 se obtiene

$$a_2 = 2, \quad a_1 = 3, \quad a_0 = 1 \quad \implies \quad \boxed{f(x) = 2x^2 + 3x + 1 \quad (\text{mód } 7)}.$$

El **secreto** es $S = f(0) = a_0 = \boxed{1}$. *Verificación de consistencia con la cuarta sombra:* $f(4) = 2 \cdot 16 + 3 \cdot 4 + 1 = 32 + 12 + 1 = 45 \equiv 3 \pmod{7}$, que coincide con $(4, 3)$: las cuatro sombras pertenecen al mismo polinomio (esquema consistente, $n = 4$ sombras sobre un polinomio de grado 2).

c) Análisis por entropía y flujo de información. Sea S la variable aleatoria del secreto sobre $\mathbb{Z}_7$ (a priori uniforme, $H(S) = \log_2 7 \approx 2,807$ bits). El esquema $(3, 4)$ es válido sii con menos de $k = 3$ sombras **no hay flujo** hacia el adversario:

$$\begin{aligned} H(S \mid S_1) &= H(S) \quad (1 \text{ sombra: no reduce incertidumbre}) \\ H(S \mid S_1, S_2) &= H(S) \quad (2 \text{ sombras: tampoco}) \\ H(S \mid S_1, S_2, S_3) &= 0 \quad (3 \text{ sombras: el secreto queda determinado}). \end{aligned}$$

La razón: con ≤ 2 puntos hay exactamente 7 polinomios de grado 2 (uno por cada valor posible de a_0) que pasan por ellos, así que cada valor de S sigue siendo equiprobable y la entropía condicional iguala a la incondicional: **no hay flujo de información** (la condición de Clase 9 $H(S \mid \text{obs}) < H(S)$ *no* se cumple). Al llegar a $k = 3$ sombras el polinomio queda unívocamente determinado, $H \rightarrow 0$: *ahí* ocurre el flujo y se revela S . Esto demuestra formalmente que el esquema es seguro hasta el umbral y reconstruible a partir de él.

8.5 Ejercicio 5 — Modelado de control de acceso para una organización (roles y datos)

Consigna (textual)

Para controlar el avance del deep fake, la ONU establece una organización internacional que se va a encargar de escrutinar las redes sociales y medios masivos para producir contenido serio que aporte una mirada objetiva sobre las noticias falsas que circulan. Asumiendo que son los CSO de esta incipiente organización, estructurar y modelar como podría ser el esquema para administrar permisos del sistema interno para el manejo de las publicaciones. Asignar los roles que crean pertinentes y sus competencias.

Temas. Clase 6 — Políticas y Control de Acceso (§RBAC; matriz de accesos; separación de tareas; MAC/DAC). Clase 11 — Protección de Datos (§Data Roles: Trustee / Steward / Custodian / User). Clase 8 — Principios de Diseño (§mínimo privilegio; separación de privilegios; mediación completa). Clase 6 — Clark–Wilson (§separation of duty).

Qué necesitás saber.

- **RBAC** (Clase 6): asignar permisos a **roles** y roles a usuarios; escala mejor que la matriz de accesos directa porque el cuello de botella humano (quién llena la matriz) se reduce. Tiene discrecionalidad fuerte (alguien asigna los roles).
- **Separación de tareas / separation of duty** (Clase 6 Clark–Wilson y Clase 8 principio 6): quien produce/edita un contenido no debería ser quien lo aprueba/publica (oposición de intereses, anti-fraude/anti-manipulación).
- **Data Roles** (Clase 11): jerarquía de gobernanza del dato — **Trustee** (estratégico, legal/cabinet-level) → **Steward** (aprueba accesos) → **Custodian** (otorga acceso técnico) → **User** (accede). Dominio más chico cuanto más baja la jerarquía.
- **Principios** (Clase 8): mínimo privilegio (cada rol con lo justo), mediación completa (todo paso de un estado a otro de la publicación pasa por un control), fail-safe defaults (por defecto, sin permiso).

Notas y conexiones. El ejercicio es abierto (“estructurar y modelar”): se evalúa que apliques RBAC con **separación de tareas** sobre el flujo editorial de una publicación, citando los principios de diseño y, opcionalmente, la jerarquía de Data Roles de Clase 11 para la gobernanza. El gancho es el mismo de Clark–Wilson llevado a un dominio editorial: el “CDI” es la publicación, el “TP” es el flujo de aprobación, y quien edita no puede ser quien aprueba.

Resolución. Proponemos un esquema **RBAC** con **separación de tareas** sobre el ciclo de vida de una publicación, gobernado por la jerarquía de Data Roles (Clase 11) y respetando mínimo privilegio (Clase 8).

Ciclo de vida de la publicación (estados): Borrador → En revisión → Verificado → Publicado (con posible Rechazado/Retractado). Cada transición es un punto de mediación completa (análogo a un TP de Clark–Wilson: solo ciertos roles llevan la publicación de un estado válido a otro).

Roles y competencias (RBAC):

- **Analista / Investigador** (crea): redacta borradores y recopila evidencia. Permisos: crear/editar publicaciones propias en estado Borrador; enviar a revisión. *No* puede aprobar ni publicar.
- **Fact-checker / Verificador** (verifica): contrasta fuentes y marca la publicación como Verificado o Rechazado. *Separación de tareas:* no puede ser el mismo que la redactó (oposición de intereses, anti-manipulación).
- **Editor / Aprobador** (publica): aprueba y publica el contenido verificado. *No* edita el cuerpo (para no introducir sesgo tras la verificación).
- **Auditor / Compliance** (controla): acceso de *solo lectura* a todo el historial y a la bitácora (log inmutable de quién hizo qué). Verifica integridad del proceso (rol IVP).
- **Administrador de acceso (Custodian) y Data Steward:** el Steward aprueba qué roles existen y quién los recibe; el Custodian los otorga técnicamente. Por encima, un **Data Trustee** (nivel CSO/legal) define la política internacional. *Ningún* usuario operativo administra sus propios permisos.

Decisiones de diseño justificadas:

- **Separación de tareas** entre crear, verificar y publicar (Clase 6/8): impide que una sola persona introduzca y publique desinformación; es el control central dado el dominio (deep fakes / noticias falsas).
- **Mínimo privilegio** y **fail-safe defaults** (Clase 8): cada rol solo accede a lo necesario; por defecto, sin permiso.
- **Mediación completa** + **bitácora** (Clark–Wilson, journal): toda transición de estado pasa por un control y queda registrada para auditoría/no repudio.
- **RBAC** sobre matriz directa: escala a una organización internacional grande y permite revisar permisos por rol, no usuario a usuario.

9 Recuperatorio 2025 Q1

9.1 Ejercicio 1 — Secreto compartido de Shamir (3,3) en $\mathbb{Z}_{11}$

Consigna (textual)

Considerar un esquema de secreto compartido de Shamir (3,3) en $\mathbb{Z}_{11}$, con el conjunto de sombras $C = \{(1, 2), (2, 5), (3, 5)\}$.

- Recuperar el secreto.
- ¿Cuál es el polinomio generador?

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§El secreto compartido de Shamir, en términos de entropía); base matemática de interpolación de Lagrange en un cuerpo finito $\mathbb{Z}_p$. Cruce con Clase 9 — Flujo de Información (la validez del esquema se justifica con entropía).

Qué necesitas saber. En un esquema (T, N) de Shamir el secreto S se codifica como el término independiente $f(0)$ de un polinomio de grado $T-1$ sobre $\mathbb{Z}_p$, y cada sombra es un punto $(x_i, f(x_i))$. Con **al menos** T sombras se reconstruye el polinomio (y por ende S) por **interpolación de Lagrange**; con menos de T , la entropía del secreto no cambia y no hay flujo de información (esa es la demostración de validez del esquema en Clase 8). Acá $T = 3$ y tenemos exactamente 3 sombras, así que el polinomio de grado 2 queda determinado de forma única. **Todas las operaciones son módulo 11**: las divisiones se hacen con inverso multiplicativo (por Fermat, $a^{-1} \equiv a^{p-2}$, o por inspección, ya que $p = 11$ es primo).

Notas y conexiones. Shamir aparece en Clase 8 como ejemplo de flujo de información: “con menos de T sombras no hay flujo” equivale a $H(S \mid S_1) = H(S \mid S_1, S_2) = H(S)$ y $H(S \mid S_1, S_2, S_3) = 0$. La parte de cálculo (recuperar el secreto y el polinomio) es interpolación de Lagrange en $\mathbb{Z}_p$, la misma maquinaria que en la Primera Parte. El secreto es $f(0)$, no una de las sombras.

Resolución. **Inversos en $\mathbb{Z}_{11}$ que se usan:** $2^{-1} \equiv 6$, $(-1)^{-1} \equiv 10$, $(-2)^{-1} \equiv 5$ (porque $2 \cdot 6 = 12 \equiv 1$, etc.).

(a) **Recuperar el secreto.** El secreto es $S = f(0)$. Por Lagrange,

$$f(0) = \sum_{i=1}^3 y_i L_i(0), \quad L_i(0) = \prod_{j \neq i} \frac{0 - x_j}{x_i - x_j} \quad (\text{mód } 11).$$

Con los puntos $(x_1, y_1) = (1, 2)$, $(x_2, y_2) = (2, 5)$, $(x_3, y_3) = (3, 5)$:

$$\begin{aligned} L_1(0) &= \frac{(0-2)(0-3)}{(1-2)(1-3)} = \frac{(-2)(-3)}{(-1)(-2)} = \frac{6}{2} \equiv 6 \cdot 6 \equiv 36 \equiv 3, \\ L_2(0) &= \frac{(0-1)(0-3)}{(2-1)(2-3)} = \frac{(-1)(-3)}{(1)(-1)} = \frac{3}{-1} \equiv 3 \cdot 10 \equiv 30 \equiv 8, \\ L_3(0) &= \frac{(0-1)(0-2)}{(3-1)(3-2)} = \frac{(-1)(-2)}{(2)(1)} = \frac{2}{2} \equiv 1. \end{aligned}$$

Entonces

$$S = f(0) = 2 \cdot 3 + 5 \cdot 8 + 5 \cdot 1 = 6 + 40 + 5 = 51 \equiv \boxed{7} \quad (\text{mód } 11).$$

(b) **Polinomio generador.** Como $T = 3$, $f(x) = a_2x^2 + a_1x + a_0$ con $a_0 = S = 7$. Planteando el sistema con las tres sombras (mod 11):

$$\begin{aligned} f(1) &= a_2 + a_1 + 7 \equiv 2 & \Rightarrow & a_2 + a_1 \equiv -5 \equiv 6, \\ f(2) &= 4a_2 + 2a_1 + 7 \equiv 5 & \Rightarrow & 4a_2 + 2a_1 \equiv -2 \equiv 9, \\ f(3) &= 9a_2 + 3a_1 + 7 \equiv 5 & \Rightarrow & 9a_2 + 3a_1 \equiv -2 \equiv 9. \end{aligned}$$

De la primera, $a_1 \equiv 6 - a_2$. Sustituyendo en la segunda: $4a_2 + 2(6 - a_2) = 2a_2 + 12 \equiv 9 \Rightarrow 2a_2 \equiv -3 \equiv 8 \Rightarrow a_2 \equiv 8 \cdot 6 \equiv 48 \equiv 4$. Luego $a_1 \equiv 6 - 4 = 2$. Verificación con la tercera: $9 \cdot 4 + 3 \cdot 2 = 36 + 6 = 42 \equiv 9 \checkmark$. Por lo tanto

$$f(x) = 4x^2 + 2x + 7 \pmod{11}$$

(chequeo: $f(1) = 13 \equiv 2$, $f(2) = 27 \equiv 5$, $f(3) = 49 \equiv 5 \checkmark$).

9.2 Ejercicio 2 — Verdadero/Falso (corregir una palabra)

Consigna (textual)

Determinar si los siguientes enunciados son verdaderos o falsos, corrigiendo una única palabra en los falsos que haga verdadera la oración.

- Un WAF es un modelo que no sólo realiza un análisis de tráfico sino que también analiza el intercambio de información de los protocolos a nivel aplicación.
- Los diferentes modelos de seguridad se encargan de mantener la confidencialidad, la integridad y/o la autenticidad de la información y los sistemas.
- En el modelo de seguridad Biba Low-watermark la escritura se permite a objetos de niveles inferiores mientras que la lectura es irrestricta.
- Construir software de calidad es garantía para la seguridad de una aplicación.

Temas. (a) Clase 10 — Seguridad en la Empresa (§Defense-in-depth, WAF); (b) Clase 6 — Políticas y Control de Acceso (§Modelos de seguridad) cruzado con Clase 8 (CIA/disponibilidad); (c) Clase 6 — Políticas y Control de Acceso (§Modelo Biba, variantes Ring-Policy y Low-Watermark); (d) Clase 10b — Pentesting / Clase 8 — Principios de Diseño (“nada garantiza la seguridad”).

Qué necesitas saber.

- **WAF (Web Application Firewall):** “un firewall pero para el tráfico de una aplicación web”, opera en la **capa de aplicación** e inspecciona el intercambio de los protocolos a ese nivel (Clase 10).
- **Modelos de seguridad:** están *especializados* en **confidencialidad** (Bell–LaPadula), **integridad** (Biba, Clark–Wilson) e *incluso disponibilidad*. La tríada es **C-I-A** (Confidencialidad, Integridad, Disponibilidad), *no* “autenticidad”.
- **Biba — variantes:** *Ring-Policy* = write-down ($i(s) \geq i(o)$) + **lectura irrestricta**. *Low-Watermark* = write-down + lectura libre **pero** al leer el sujeto se reclasifica $i(s) = \min(i(s), i(o))$. La frase “lectura irrestricta” (sin reclasificación) describe **Ring-Policy**.
- **Garantizar la seguridad** es imposible (Clase 10b): ni el pentesting ni el “buen diseño” la garantizan; sólo la favorecen.

Notas y conexiones. Los V/F de esta materia suelen mezclar una afirmación correcta con *una sola* palabra cambiada. Dos trampas recurrentes están acá: la tríada **CIA** (la “A” es *disponibilidad*, no autenticidad) y el verbo **garantizar** (gancho clásico de la Clase 10b: “garantizar la seguridad es algo que no se puede hacer jamás”). En (c) hay que distinguir las tres variantes de Biba: el rasgo propio de Low-Watermark es la *reclasificación dinámica* al leer; la lectura “irrestringida a secas” es de Ring-Policy.

Resolución.

- VERDADERO.** El WAF analiza el tráfico de los protocolos a nivel aplicación (capa de aplicación). (*Matiz: técnicamente es un mecanismo, no un “modelo”; pero la oración es correcta en su contenido, así que se toma como verdadera.*)
- FALSO.** Palabra a corregir: “autenticidad” → “disponibilidad”. Los modelos de seguridad mantienen **confidencialidad, integridad y/o disponibilidad** (tríada CIA).
- FALSO.** Palabra a corregir: “Low-watermark” → “Ring-Policy”. Write-down con *lectura irrestringida* es Biba **Ring-Policy**; en Low-Watermark la lectura baja el nivel de integridad del sujeto a $\min(i(s), i(o))$.
- FALSO.** Palabra a corregir: “garantía” → “ayuda” (o “no es garantía”). Construir software de calidad *favorece* la seguridad, pero **nada la garantiza**.

9.3 Ejercicio 3 — One-time-pad y pérdida de información (entropía)

Consigna (textual)

Demostrar con un cálculo de ejemplo utilizando entropía si en el one-time-pad hay pérdida de información de m a c , para una clave y mensaje de longitud 10.

Temas. Clase 9 — Flujo de Información y Malware (§Entropía de Shannon, entropía condicional, definición formal de flujo de información). Cruce con la noción de *secreto perfecto* (One-Time-Pad) de la Primera Parte.

Qué necesitás saber.

- **Entropía de Shannon:** $H(X) = -\sum_i p(x_i) \lg p(x_i)$ (en bits). Máxima con distribución uniforme ($H = \lg n$), mínima ($= 0$) con evento único.
- **Entropía condicional:** $H(X | Y) = \sum_j p(y_j) H(X | Y = y_j)$.
- **Flujo de información de m a c :** hay flujo si $H(m | c) < H(m)$ — es decir, observar c *reduce* la incertidumbre sobre m . Si $H(m | c) = H(m)$, **no hay flujo / no hay pérdida de información:** c no revela nada de m .
- **OTP:** $c = m \oplus k$ con k **aleatoria, uniforme, del mismo largo que m y usada una sola vez**. Es la condición de *secreto perfecto* de Shannon. Cuidado: OTP = *One-Time-Pad*, no confundir con OTP de contraseñas.

Notas y conexiones. “Pérdida de información de m a c ” significa que parte de la información del mensaje *se filtra* al cifrado, lo que en el marco de la Clase 9 es exactamente un *flujo de información* de m hacia c . El criterio es el mismo que se usa para “verificar que una función de cifrado no revele información de la clave”: comparar $H(m)$ con $H(m | c)$. La meta del OTP (secreto perfecto) es justamente que **no** haya flujo: $H(m | c) = H(m)$.

Resolución. Modelemos un alfabeto binario por posición (el argumento se replica por las 10 posiciones). Sea cada bit de mensaje m_i con distribución arbitraria, y cada bit de clave k_i **uniforme** ($p(k_i=0) = p(k_i=1) = \frac{1}{2}$), independiente del mensaje. El cifrado es $c_i = m_i \oplus k_i$.

Paso 1 — la clave uniformiza el cifrado. Para cualquier valor fijo de m_i ,

$$p(c_i=0) = p(k_i=m_i) = \frac{1}{2}, \quad p(c_i=1) = \frac{1}{2},$$

así que cada bit de c es uniforme: $H(c_i) = 1$ bit, y para los 10 bits $H(C) = 10$ bits (máxima).

Paso 2 — el cifrado no informa sobre el mensaje. Tomemos un ejemplo concreto con $p(m_i=0) = 0.5$ (mensaje también incierto). Antes de ver c :

$$H(m_i) = -\frac{1}{2} \lg \frac{1}{2} - \frac{1}{2} \lg \frac{1}{2} = 1 \text{ bit}.$$

Conociendo c_i , como $m_i = c_i \oplus k_i$ y k_i es uniforme e independiente, dado cualquier c_i se tiene $p(m_i=0 | c_i) = p(m_i=1 | c_i) = \frac{1}{2}$, de modo que

$$H(m_i | c_i) = -\frac{1}{2} \lg \frac{1}{2} - \frac{1}{2} \lg \frac{1}{2} = 1 \text{ bit} = H(m_i).$$

Para el mensaje completo de longitud 10 (bits independientes):

$$H(M) = \sum_{i=1}^{10} H(m_i) = 10 \text{ bits}, \quad H(M | C) = \sum_{i=1}^{10} H(m_i | c_i) = 10 \text{ bits}.$$

Conclusión. Como $H(M | C) = H(M)$ (la incertidumbre sobre el mensaje **no disminuye** al observar el cifrado), **no hay flujo de información de m a c** , es decir **no hay pérdida de información**: el OTP no filtra nada del mensaje hacia el cifrado (secreto perfecto). Si en cambio $H(M | C) < H(M)$ habría flujo y por lo tanto pérdida; eso ocurre, por ejemplo, si la clave *no* es uniforme o se reutiliza (deja de ser un OTP válido). *(Si el enunciado interpreta “pérdida” como información del mensaje que el OTP destruye/oculta, la respuesta es la misma cuenta: $H(M | C) = H(M)$ implica que el canal $m \rightarrow c$ no transmite información sobre m .)*

9.4 Ejercicio 4 — Fórmula de Anderson y almacenamiento con SHA-256

Consigna (textual)

Un modelo de amenazas opera bajo el supuesto que un atacante puede probar 10^5 contraseñas por segundo. Si las contraseñas se toman de un alfabeto de 96 caracteres, y se desea que el atacante tenga $\frac{1}{100}$ de probabilidad de éxito a lo largo de 365 días.

- Hallar la longitud mínima de una contraseña que cumpla con lo pedido.
- ¿Es válido almacenar esas contraseñas en una base de datos con SHA-256? ¿Por qué?

Temas. Clase 7 — Autenticación (§Fórmula de Anderson $P = TG/N$; §Almacenamiento seguro: Salting y PBKDF2). Cruce con Clase 9 (costo de cómputo del atacante).

Qué necesitas saber.

- **Fórmula de Anderson:** $P = \frac{T \cdot G}{N}$, con P = probabilidad de adivinar, T = tiempo del ataque, G = pruebas/segundo, N = tamaño del espacio de claves. Despejando la longitud: $N = A^L \geq \frac{TG}{P}$, luego $L \geq \log_A \left(\frac{TG}{P} \right)$ y se redondea **hacia arriba**.

- **Unidades:** pasar los 365 días a segundos ($365 \cdot 24 \cdot 3600 = 31\,536\,000$ s).
- **Almacenamiento:** guardar claves requiere una función **costosa a propósito** con **salting** (PBKDF2/bcrypt/scrypt, ≥ 100 ms por evaluación). SHA-256 es un hash de **propósito general muy rápido**, sin salt ni costo: el valor $G = 10^5/\text{s}$ es justamente la consecuencia de un hash rápido $\Rightarrow$ no es un esquema de almacenamiento adecuado.

Notas y conexiones. El ejercicio es el “Ejercicio 3” de Clase 7 (despejar la longitud mínima), ahora con $A = 96$, $G = 10^5$, $P = 1/100$ y $T = 365$ días. La parte (b) conecta con el porqué de G : con un hash rápido como SHA-256 un atacante que roba la base prueba muchísimas claves por segundo y puede paralelizar; salting + función costosa (PBKDF2) frenan la paralelización y suben el costo por intento.

Resolución. (a) **Longitud mínima.** Pasamos el tiempo a segundos:

$$T = 365 \cdot 24 \cdot 3600 = 31\,536\,000 \text{ s.}$$

Por Anderson, exigir $P \leq \frac{1}{100}$ equivale a

$$P = \frac{TG}{N} \leq \frac{1}{100} \implies N \geq \frac{TG}{P} = \frac{31\,536\,000 \cdot 10^5}{1/100} = 3.1536 \times 10^{14}.$$

Como $N = 96^L$:

$$96^L \geq 3.1536 \times 10^{14} \implies L \geq \log_{96}(3.1536 \times 10^{14}) \approx 7.31.$$

Redondeando hacia arriba, $\lceil L \geq 8 \rceil$. Verificación: $96^7 \approx 7.51 \times 10^{13} < N$ (insuficiente) y $96^8 \approx 7.21 \times 10^{15} \geq N$ (cumple) — con $L = 8$, $P = \frac{31\,536\,000 \cdot 10^5}{96^8} \approx 4.4 \times 10^{-4} < \frac{1}{100}$.

(b) **¿SHA-256 para almacenarlas? No es válido (o al menos no es buena práctica).**
Razones:

- SHA-256 es un hash de propósito general **diseñado para ser rápido**; eso es lo contrario de lo que se quiere para almacenar claves. El propio supuesto del enunciado ($G = 10^5/\text{s}$, y en GPU mucho más) es consecuencia de usar un hash rápido: el cálculo de (a) sólo “aguanta” por la longitud, no por el almacenamiento.
- SHA-256 “a secas” **no tiene salt**: dos usuarios con la misma clave producen el mismo c , y el ataque se **paraleliza** (precómputo, rainbow tables, ataque simultáneo a todas las cuentas).
- Lo correcto es usar una **función de derivación costosa con salting** — **PBKDF2**, bcrypt o scrypt (≥ 100 ms por evaluación)— que reduce drásticamente G (de $10^5/\text{s}$ a unas pocas por segundo) y, con salt por usuario, impide la paralelización. (*SHA-256 puede usarse como PRF dentro de PBKDF2-HMAC-SHA256, pero no como mecanismo de almacenamiento por sí solo.*)

10 Parcial 2025 Q2

10.1 Ejercicio 1 — Modelo de seguridad para Palantir (CSO)

Consigna (textual)

Palantir es una empresa startup que provee tecnología para control y mitigación de *swarms* de drones (entre otras cosas). Palantir necesita rediseñar todo su sistema de seguridad para poder continuar trabajando con diversos ejércitos del mundo (a quienes provee de tecnología *top secret*). Ud, Ingeniera, Ingeniero, es el nuevo CSO.

- Detallar en este escenario, como CSO de Palantir, qué modelo de seguridad establecerían, qué políticas de control de acceso y qué reglas de seguridad.
- Describir cuál es la diferencia entre modelo de seguridad, política de control de acceso y reglas de seguridad.

Temas. Clase 6 — Políticas y Control de Acceso (§Política de seguridad; §¿Qué es un modelo de seguridad?; §Bell–LaPadula; §MAC vs. DAC; §RBAC/ABAC/RuBAC).

Qué necesitás saber.

- **Modelo de seguridad:** modelo formal y lógico que define las reglas e interacciones que permiten implementar un mecanismo de control de acceso; con sustento matemático se demuestra que el sistema es seguro. Están especializados (confidencialidad, integridad, disponibilidad) y *no alcanzan por sí solos* para todo el sistema: se aplican a partes.
- **Bell–LaPadula** (confidencialidad militar): clasificaciones {TS, S, C, U} con compartimentos formando un reticulado. Reglas: *Read Down* (seguridad simple, $L(s) \succeq L(o)$), *Write Up* (propiedad *, $L(o) \succeq L(s)$) y *-fuerte (leer+escribir $\Rightarrow$ misma clasificación). Es **control de flujo**: la información “sube”, no “baja”.
- **Política de control de acceso:** la definición de qué estados consideramos seguros (una “foto en el tiempo” que caracteriza si el estado actual es seguro). Se implementa con mecanismos **MAC** (mandatorio, basado en reglas universales del sistema, p. ej. el lattice de BLP) y/o **DAC** (discrecional, basado en identidad, el dueño/admin asigna permisos). Por comprensión: **RBAC** (roles), **ABAC**, **RuBAC**.
- **Reglas de seguridad:** los enunciados concretos que el mecanismo evalúa (las reglas de BLP, las entradas de una ACL, las reglas ordenadas tipo firewall).

Notas y conexiones. El escenario (datos *top secret*, múltiples ejércitos que no deben verse entre sí) es el caso canónico de **Bell–LaPadula**: confidencialidad + compartimentos (*need-to-know*). La idea de aislar proyectos con clientes distintos es exactamente la motivación de los compartimentos del reticulado. BLP cubre la parte mandatoria; el día a día (quién accede a qué bucket/repositorio) se gestiona con DAC/RBAC, que tiene discrecionalidad fuerte (alguien asigna roles). Conecta con Clase 8 (separación de privilegios, mínimo privilegio) y Clase 10 (Defense-in-depth, ZTA).

Resolución. a) Como CSO de Palantir propondría:

- **Modelo de seguridad: Bell–LaPadula** como núcleo, por ser un modelo de *confidencialidad* pensado para el ámbito militar y manejo de información clasificada. Se definen niveles de clasificación (Top Secret / Secret / Confidential / Unclassified) y, sobre todo, **compartimentos por cliente/proyecto** (cada ejército es una categoría incomparable con las demás),

formando un reticulado. Donde BLP no alcance (sistemas de propósito general internos), se complementa con DAC/RBAC. Si además importa la *integridad* de la información de control de drones, se puede sumar Biba o Clark–Wilson sobre esos componentes.

- **Políticas de control de acceso:** mandatorias (MAC) para la información clasificada — las define el sistema y se aplican obligatoriamente a todos los accesos, cambios de atributos e intercambios entre sujetos; y discrecionales (DAC, típicamente **RBAC** por roles: analista, operador de drones, administrador, cliente militar) para la gestión cotidiana. *Need-to-know* entre compartimentos.
- **Reglas de seguridad concretas:** las tres reglas de BLP — *Read Down* ($L(s) \succeq L(o)$ para leer), *Write Up* ($L(o) \succeq L(s)$ para escribir) y **-fuerte* (lectura+escritura solo en la misma clasificación) — más la dominancia con compartimentos: $(L, C) \succeq (L', C') \iff L' \leq L \wedge C' \subseteq C$. Dos clientes en compartimentos incomparables no pueden leerse ni escribirse mutuamente.

b) La diferencia (de lo más abstracto a lo más concreto):

- El **modelo de seguridad** es el marco *formal/matemático* (p. ej. Bell–LaPadula): generaliza, demuestra propiedades y dice *qué* hay que garantizar, sin atarse a una implementación.
- La **política de control de acceso** es la *definición de qué estados son seguros* en este sistema concreto: a qué clasifico cada activo, qué sujetos existen, qué pueden hacer — el “qué se permite”. Decide entre mandatorio (MAC) y discrecional (DAC).
- Las **reglas de seguridad** son los enunciados concretos que el mecanismo evalúa para decidir cada acceso (las reglas de BLP, las entradas de una ACL, las reglas ordenadas de un firewall) — el “cómo se decide”.

En síntesis: el modelo *fundamenta* la política, y la política se *expresa* mediante reglas que el mecanismo aplica.

10.2 Ejercicio 2 — Opción múltiple (pentesting, ley 25.326, buffer overflow)

Consigna (textual)

Responder la opción más correcta.

- ¿Cuáles son los pasos para implementar un pentesting exitoso?
 - Recolección de información, Prueba de la seguridad, Generalización, Eliminación.
 - Recolección de información, Prueba formal, Generalización, Eliminación.
 - Recolección de información, Planteo de Hipótesis de falla, Prueba de la vulnerabilidad, Generalización, Eliminación.
 - Planteo de Hipótesis de falla, Prueba de la vulnerabilidad, Verificación formal.
- La ley argentina 25.326 de Protección de datos personales establece mecanismos y normativa para regular
 - El Derecho al olvido:
 - Las bases de datos públicas y privadas
 - El manejo de las cookies en los navegadores de internet.
 - Las historias clínicas.
- ¿Cuál de las siguientes estrategias no tendría sentido en un ataque buffer overflow?

- Usar RUST
- Implementar un garbage collector
- Validar todos los datos de entrada.
- Implementar un Stack Canary.

Temas. Clase 10b — Pentesting (§Metodología de hipótesis de falla); Clase 11 — Protección de Datos (§Ley 25.326); Clase 8 — Principios de Diseño y Vulnerabilidades (§CWE-787 / buffer overflow).

Qué necesitás saber.

- **Pasos del pentesting (de memoria, en orden):** Recolección de información → Planteo de hipótesis de falla → Prueba (de la vulnerabilidad) → Generalización → Eliminación (opcional). Equivocar el orden o los pasos penaliza. Notar que es *hipótesis de falla*, no “prueba formal”: la verificación formal es lo *contrario* del pentesting.
- **Ley 25.326 (argentina):** regula las bases de datos **públicas y privadas** (hábeas data: ver/rectificar/borrar tus datos). El **Derecho al Olvido** lo regula el **GDPR europeo**, NO la ley argentina. La ley no especifica tecnología.
- **Buffer overflow (CWE-787, out-of-bounds write):** escribir fuera de los límites del buffer; en el stack puede pisar la dirección de retorno. Estrategias de defensa válidas: lenguajes que eliminan la aritmética de punteros / acceso a memoria seguro (RUST, Java/C#), *stack canary*, validar la entrada. Un **garbage collector** gestiona el *ciclo de vida de memoria dinámica* (fugas, use-after-free), no impide escrituras fuera de límite en el stack.

Notas y conexiones. La primera y la última opción son las dos “trampas” que el profesor remarca: confundir “Planteo de hipótesis de falla” con “prueba formal/de la seguridad” (Clase 10b), y la distinción ley argentina vs. GDPR (Clase 11), la más preguntable de protección de datos. El buffer overflow conecta con Clase 8 (injection $\approx 90\%$; lenguajes “seguros” que mataron familias enteras de bugs) y con el principio de *validar la entrada*.

Resolución.

- **Pasos del pentesting: “Recolección de información, Planteo de Hipótesis de falla, Prueba de la vulnerabilidad, Generalización, Eliminación”** (la 3.^a opción). Las otras sustituyen el paso clave por “Prueba de la seguridad” / “Prueba formal” o lo reemplazan por “Verificación formal” (que es la técnica opuesta).
- **Ley 25.326: “Las bases de datos públicas y privadas”.** El Derecho al olvido es GDPR; cookies (consentimiento) también es más bien GDPR; las historias clínicas son un caso particular (HIPAA en EE.UU.), no el objeto de la 25.326.
- **Buffer overflow:** la estrategia que **no tiene sentido** es “Implementar un garbage collector”: un GC administra memoria dinámica, no evita la escritura fuera de los límites de un buffer (el desbordamiento clásico). Usar RUST (acceso a memoria seguro), validar la entrada y un stack canary sí mitigan.

10.3 Ejercicio 3 — Encriptación homomórfica

Consigna (textual)

La encriptación homomórfica aparece como un escenario donde es posible almacenar datos en la nube directamente encriptados y donde se pueden realizar operaciones de cálculo directamente sobre los datos, obteniendo un resultado que al desencriptarlo coincidiría con el resultado obtenido de la operación.

Por ejemplo: $f(Enc_k(x)) = Enc_k(f(x))$

- a) Explicar qué escenario de protección de datos podría ser beneficioso este esquema.

Temas. Clase 11 — Protección de Datos (§Encriptación homomórfica; §Estados del dato: *in-use*).

Qué necesitás saber. La encriptación homomórfica permite **operar sobre los datos cifrados sin descifrarlos**: $f(Enc_k(x)) = Enc_k(f(x))$. Es decir, operar sobre el texto cifrado y luego descifrar da el mismo resultado que operar sobre el texto plano. Ataca el estado más difícil de proteger: el dato *en uso* (in-use), cuando normalmente hay que descifrarlo para procesarlo.

Notas y conexiones. El control de acceso no alcanza para proteger datos: el dato fluye (RR.HH. $\rightarrow$ App $\rightarrow$ nube). Cuando delegamos cómputo a un tercero (la nube), el problema es que para procesar hay que exponer el dato en claro. La homomórfica rompe ese compromiso. Conecta con los estados del dato (Clase 11) y con *shared tenancy* / confidencialidad en la nube (Clase 9).

Resolución. El escenario beneficiado es la **computación delegada en la nube preservando la privacidad**: un tercero (proveedor cloud) procesa, analiza o agrega datos sensibles **sin verlos nunca en claro**. El cliente cifra ($Enc_k(x)$), envía el cifrado, el servidor computa f sobre el cifrado y devuelve $Enc_k(f(x))$; solo el cliente, con su clave, descifra el resultado:

$$Dec_k(Enc_k(f(x))) = f(x).$$

Ejemplos concretos: análisis estadístico/ML sobre datos médicos o financieros tercerizado a un proveedor que no debe acceder a los datos en claro (HIPAA, datos de salud anonimizados), votaciones electrónicas, o consultas a una base de datos en la nube sin revelar ni la consulta ni los registros. Resuelve la protección del dato *en uso*, que de otro modo obligaría a descifrar para procesar.

10.4 Ejercicio 4 — Secreto compartido de Shamir

Consigna (textual)

Considerar un esquema de secreto compartido de Shamir $(k, n) = (3, 5)$ en $\mathbb{Z}_7$, con el conjunto de sombras

$$C = \{(1, 2), (2, 2), (3, 3), (4, 5), (5, 1)\}.$$

- a) Recuperar el secreto.
b) Cuál es el polinomio generador.
c) Considerar el anillo de polinomios $\mathbb{F}_2[x]$ y el campo de extensión

$$\mathbb{F}_{2^3} \cong \mathbb{F}_2[x]/(p(x)), \quad p(x) = x^3 + x + 1,$$

denotando α la clase de x en $\mathbb{F}_{2^3}$ (es decir $\alpha = x \bmod p(x)$). Se define un esquema de

Shamir $(k, n) = (3, 4)$ sobre $\mathbb{F}_{2^3}$. Las sombras entregadas son

$$C = \{(1, \alpha + 1), (\alpha, 1 + \alpha + \alpha^2), (\alpha + 1, 1), (\alpha^2, \alpha + 1)\},$$

donde los puntos se escriben como (t, s_t) con $t \in \mathbb{F}_{2^3}$ y $s_t \in \mathbb{F}_{2^3}$. Detallar por qué razón el algoritmo secreto compartido de Shamir puede también plantearse de esta manera.

Temas. Clase 8 — Principios de Diseño y Vulnerabilidades (§El secreto compartido de Shamir, en términos de entropía); flujo de información (entropía). Mecánica: interpolación de Lagrange sobre un cuerpo finito.

Qué necesitás saber.

- En un esquema (T, N) de Shamir hay N sombras y se necesitan **al menos** T para recuperar el secreto S . El secreto se codifica como el término independiente de un polinomio de grado $T - 1$; cada sombra es un punto $(x_i, p(x_i))$. Con $< T$ puntos la entropía del secreto *no cambia* (no hay flujo); con $\geq T$, $H(S) \rightarrow 0$.
- Aquí $T = 3$, así que p tiene grado 2: $p(x) = a_0 + a_1x + a_2x^2$, y el secreto es $S = a_0 = p(0)$. Se recupera con **interpolación de Lagrange** usando 3 puntos, toda la aritmética módulo 7 (cuerpo $\mathbb{Z}_7$).
- La validez del esquema (parte c) depende solo de que el conjunto de coeficientes / evaluaciones viva en un **cuerpo**: lo que se necesita es poder sumar, restar, multiplicar y **dividir** (existencia de inversos) para que Lagrange funcione.

Notas y conexiones. Shamir aparece en Clase 8 como ejemplo de flujo de información medido con entropía: “con menos de T sombras no hay flujo” equivale a “ $H(S)$ no cambia”. Aquí se pide además la mecánica (Lagrange). La parte c) conecta con la Primera Parte (cuerpos finitos $\mathbb{F}_{2^n}$, aritmética polinomial módulo un irreducible, igual que en AES).

Resolución. **a) y b)** Buscamos $p(x) = a_0 + a_1x + a_2x^2$ en $\mathbb{Z}_7$. Con tres sombras, por ejemplo $(1, 2), (2, 2), (3, 3)$, planteamos el sistema:

$$\begin{aligned} a_0 + a_1 + a_2 &\equiv 2 \quad (\text{mód } 7), \\ a_0 + 2a_1 + 4a_2 &\equiv 2 \quad (\text{mód } 7), \\ a_0 + 3a_1 + 2a_2 &\equiv 3 \quad (\text{mód } 7). \end{aligned}$$

Resolviendo (eliminación de Gauss módulo 7, recordando que en $\mathbb{Z}_7$ los inversos son $2^{-1} = 4$, $3^{-1} = 5$, $4^{-1} = 2$, $5^{-1} = 3$, $6^{-1} = 6$) se obtiene

$$a_0 = 3, \quad a_1 = 2, \quad a_2 = 4.$$

Luego el **polinomio generador** es

$$p(x) = 4x^2 + 2x + 3 \quad (\text{mód } 7)$$

y el **secreto** es el término independiente:

$$S = p(0) = a_0 = 3.$$

Verificación (consistencia del esquema): $p(1) = 4 + 2 + 3 = 9 \equiv 2$, $p(2) = 16 + 4 + 3 = 23 \equiv 2$, $p(3) = 36 + 6 + 3 = 45 \equiv 3$, $p(4) = 64 + 8 + 3 = 75 \equiv 5$, $p(5) = 100 + 10 + 3 = 113 \equiv 1 \quad (\text{mód } 7)$.

Las cinco sombras caen sobre el polinomio, así que el reparto es válido y cualquier subconjunto de 3 sombras recupera el mismo secreto.

c) El algoritmo de Shamir **no depende de que trabajemos en $\mathbb{Z}_p$ con p primo**: lo único que usa es que las coordenadas vivan en un **cuerpo (campo)**. La construcción y la recuperación se basan en la **interpolación de Lagrange**, que requiere sumar, restar, multiplicar y **dividir** — y dividir solo es posible si todo elemento no nulo tiene inverso multiplicativo, que es justamente la propiedad que define a un cuerpo.

$\mathbb{F}_{2^3} \cong \mathbb{F}_2[x]/(p(x))$ con $p(x) = x^3 + x + 1$ **irreducible** sobre $\mathbb{F}_2$ es un cuerpo finito de $2^3 = 8$ elementos (las clases $\{0, 1, \alpha, \alpha + 1, \alpha^2, \dots\}$), con suma y producto de polinomios módulo $p(x)$ y, por ser cuerpo, **inversos para todo elemento no nulo**. Por lo tanto:

- Se puede definir un polinomio secreto de grado $T - 1 = 2$ con coeficientes en $\mathbb{F}_{2^3}$, evaluarlo en $N = 4$ puntos distintos $t \in \mathbb{F}_{2^3}$ para generar las sombras (t, s_t) , y
- recuperar el secreto $S = p(0)$ con la misma fórmula de Lagrange, ahora con la aritmética de $\mathbb{F}_{2^3}$ (los inversos se calculan módulo $p(x)$).

Es decir, Shamir se plantea idénticamente sobre cualquier cuerpo finito $\mathbb{F}_q$ (sea q primo como $\mathbb{Z}_7$, o potencia de primo como $\mathbb{F}_{2^3}$); usar $\mathbb{F}_{2^n}$ es natural cuando los secretos son cadenas de bits, exactamente como en AES. **TODO:** verificar numéricamente la recuperación en $\mathbb{F}_{2^3}$ no se pide explícitamente; la consigna solo pide *justificar por qué* es válido plantearlo así (la respuesta es: porque $\mathbb{F}_{2^3}$ es un cuerpo y Lagrange solo necesita un cuerpo).

10.5 Ejercicio 5 — “Garantizar la seguridad” con pentesting (Tesla)

Consigna (textual)

Tesla implementó un sistema de mejoramiento de la calidad de software nuevo. Su director, Charles Kharpaty, anunció que estarían utilizando un nuevo esquema que garantizaba la seguridad basado en estrictas pruebas de pentesting.

- a) Provea argumentos sólidos para deslizarle al oído de Elon para que lo despida o lo promueva a CSO.

Temas. Clase 10b — Pentesting (§Limitaciones: el pentesting NO garantiza la seguridad; §Verificación formal vs. pentesting).

Qué necesitás saber. **Gancho/trampa de parcial:** el pentesting es una buena estrategia pero **NO garantiza la seguridad**. “Garantizar la seguridad es algo que no se puede hacer jamás” (palabras del profesor). Si un enunciado afirma que algo “garantiza la seguridad” es incorrecto, y **hay penalización si se les escapa**. Razones:

- El pentesting prueba la **existencia** de vulnerabilidades, **nunca su ausencia** (Dijkstra: el testing muestra presencia de bugs, no su ausencia). Quien sí prueba ausencia es la *verificación formal*, pero solo en un algoritmo/ambiente acotado incluyendo todos los factores — y *tampoco* da garantía total para un sistema completo.
- No es sistemático: depende de la capacidad del tester para formular hipótesis.
- Cada test vale para *ese* sistema en *ese* momento (si cambia la API/v3, las pruebas quedan obsoletas).
- No reemplaza un buen diseño, especificación, implementación y testing; se suma *después*.

Notas y conexiones. Es el mismo gancho que el Ejercicio 2 (“Prueba formal” vs. hipótesis de falla) visto desde el lado del enunciado-trampa. Conecta con Clase 6/8 (la seguridad es un estado que se debe mantener; un buen diseño y los principios de Saltzer–Schroeder son la base) y con Clase 10 (DevSecOps: el pentesting es una capa más, no la garantía).

Resolución. **Despedirlo** (o, como mínimo, corregir el anuncio). El error de fondo es la afirmación de que el esquema “**garantiza la seguridad**”: *garantizar la seguridad es imposible*. Argumentos sólidos para el oído de Elon:

1. **El pentesting nunca prueba ausencia de vulnerabilidades, solo existencia.** Encontrar fallas demuestra que las hay; *no* encontrarlas no demuestra que no las haya (Dijkstra). Anunciar “garantía de seguridad” es técnicamente falso.
2. **No es sistemático:** su calidad depende de la habilidad del tester para plantear hipótesis de falla; dos testers distintos cubren cosas distintas.
3. **Es válido para ese sistema en ese momento:** cualquier cambio (nueva versión, nueva integración) invalida buena parte de las pruebas. No hay garantía persistente.
4. **No reemplaza el buen diseño:** la seguridad se construye con principios de diseño, especificación, implementación cuidada y cobertura de testing; el pentesting se agrega *después* para comprobar, no para garantizar.
5. Si se quisiera la mayor garantía posible, lo más cercano sería la **verificación formal** (prueba ausencia, pero solo en un dominio acotado y aun así no total), no el pentesting.

Conclusión: Kharpaty merece ser **despedido (o corregido) por prometer una garantía que no existe**; un buen CSO diría “reducimos el riesgo y aumentamos la confianza”, nunca “garantizamos la seguridad”.